

# Refinement Based Reasoning of P2P Protocols is Feasible and Effective

by  
Ankit Kumar

Submitted to the Khoury College of Computer Sciences  
in partial fulfillment of the requirements for the degree of  
DOCTOR OF PHILOSOPHY IN COMPUTER SCIENCE

at the  
NORTHEASTERN UNIVERSITY

May 2026

© 2026 Ankit Kumar. All rights reserved.

Authored by: Ankit Kumar

Northeastern University

April 2026

Certified by: Panagiotis (Pete) Manolios

Professor, Thesis Supervisor

Accepted by: Dr. Sara Wadia-Fascetti

Professor

Khoury College of Computer Sciences



To Dadaji, Dadima, Nanaji, and Nanima

Ever grateful for your blessings



**Northeastern University**  
**Khoury College of Computer Sciences**

February 20, 2026

**Dissertation Title:** Refinement Based Reasoning of P2P Protocols is Feasible and Useful.

**Author:** Ankit Kumar

Approved for Dissertation Requirements of the Doctor of Philosophy Degree.

---

**Dissertation Advisor: Panagiotis Manolios**

---

**Date**

---

**Dissertation Reader: Joseph Kiniry**

---

**Date**

---

**Dissertation Reader: Cristina Nita-Rotaru**

---

**Date**

---

**Dissertation Reader: Guevara Noubir**

---

**Date**



# Refinement Based Reasoning of P2P Protocols is Feasible and Effective

by

Ankit Kumar

Submitted to the Khoury College of Computer Sciences  
on April 2026 in partial fulfillment of the requirements for the degree of

DOCTOR OF PHILOSOPHY IN COMPUTER SCIENCE

## ABSTRACT

Gossipsub is the message dissemination protocol underlying Ethereum, IPFS, and Filecoin—systems with a combined market capitalization exceeding \$280 billion. Despite its critical role in Web3 infrastructure, Gossipsub lacks a formal specification; its security mechanisms have been evaluated primarily through ad hoc testing; and its operational parameters were selected heuristically, with limited quantitative justification.

This dissertation develops a compositional, refinement-based methodology for the rigorous analysis of peer-to-peer (P2P) publish-subscribe protocols and applies it to Gossipsub, making the following three principal contributions: (1) the first formal, executable specification of Gossipsub, now recognized as the official reference model; (2) the discovery and responsible disclosure of novel attacks against Gossipsub v1.1 based consensus layer of Ethereum (CVE-2022-47547); and (3) a systematic analysis demonstrating that Ethereum’s deployed parameter configuration for Gossipsub is Pareto-dominated by leaner alternatives under realistic network churn.

We construct a refinement hierarchy spanning four protocol layers: Broadcastsub (an abstract specification of ideal dissemination), Floodsub (a flooding-based realization), Meshsub (Gossipsub v1.0 with bounded-degree mesh overlays), and Gossipsub v1.1 (which introduces scoring-based mesh management). We prove that Meshsub refines Floodsub and Floodsub refines Broadcastsub; by compositional refinement, Meshsub refines Broadcastsub. Central to this hierarchy is a dissemination guarantee: once a message is committed at its source, every subscribed peer eventually receives it. This property holds deterministically for Broadcastsub and Floodsub, and with arbitrarily high probability for Meshsub under identified network conditions.

Applying property-based testing and counterexample generation to Gossipsub v1.1, we demonstrate that this dissemination guarantee is violated. The protocol admits executions in which committed messages are never delivered to subscribed peers. We use these counterexamples to construct concrete eclipse and network-partition attack gadgets. Simulation experiments confirm the feasibility of these attacks; following responsible disclosure, the vulnerability was assigned CVE-2022-47547 and acknowledged by the Gossipsub and Ethereum developers. These findings prompted protocol revisions and contributed to ongoing efforts to strengthen consensus-layer message dissemination in Ethereum and related systems.

Finally, we conduct a systematic exploration of the Gossipsub parameter space under network churn. Pareto analysis over delivery reliability and bandwidth cost yields three

findings: (1) Gossipsub strictly dominates Floodsub across churn regimes; (2) minimal-gossip configurations lie on the Pareto frontier and dominate Ethereum’s deployed parameters under worst-case delivery; and (3) gossip-based recovery, rather than eager-push redundancy, is the primary driver of reliable dissemination under churn. These results provide the first principled foundation for parameter selection in production Gossipsub networks.

Taken together, this work demonstrates that compositional refinement, combined with property-based testing, counter-example generation, and simulation based analyses, constitutes a practical and scalable methodology for analyzing complex, real-world P2P protocols. Beyond certifying correctness, it enables the discovery of previously unknown vulnerabilities and the principled optimization of deployed systems. All models, proofs, and simulation artifacts are publicly available and fully executable to support reproducibility and reuse.

Thesis supervisor: Panagiotis (Pete) Manolios

Title: Professor



# Acknowledgments

I like to think of my PhD journey as that of a growing child. I am deeply grateful to my advisor, Pete. Pete treats his students as his children, and like a father, he nurtured me with patience, trust, and care. Throughout my PhD, he consistently supported me in all aspects of my work and life: technical, administrative, financial, and personal. His optimism and enthusiasm are contagious, and they consistently fostered a supportive and motivating research environment. His confidence in me during moments of doubt, and his steady guidance during moments of growth, shaped not only my research but also who I am as a person. I could not have asked for a better mentor.

I would like to thank the members of my thesis committee: Pete Manolios, Cristina Nita-Rotaru, Joseph Kiniry and Guevara Noubir. Their feedback and guidance have greatly contributed to improving the quality of this dissertation.

Our research group felt like a family. Andrew Walter is my academic twin brother: we started our PhDs together under Pete during Fall 2018. Over the years, we worked side by side during internships at Amazon and Rivos Inc. and collaborated on multiple research projects. Some of my work directly builds on his contributions to ACL2S systems programming, Lisp-Z3 and ACL2S Docker infrastructure. Beyond academia, I was grateful to host Andrew at my wedding in December 2024. I cannot imagine this journey without him by my side.

Max von Hippel and I first met as collaborators on a group project for Cristina’s course on computer network systems, where we studied the correctness of the Gossipsub scoring function. That project ultimately became the foundation of this dissertation. From the very beginning, I was astounded by Max’s speed, rigor, and work ethic. Over the years, our collaboration extended well beyond research. We also lived together as roommates, and Max kindly invited me to be part of his bachelor party. Working with him pushed me to be more disciplined, to think more clearly, and to work more carefully. Our collaboration shaped not only the technical direction of this research, but also my standards for what good research should look like.

I was also fortunate to be part of a broader research group (extended family) around Pete that made graduate school both intellectually stimulating and genuinely enjoyable. I benefited greatly from working with and learning alongside Derek Egolf, Zeke Medley, Emmanuel Manolios, Angelina Manolios, Cassidy Waldrip, Shaoqi Wang, Ben Quiring, Brent (Yibo) Zhao, Joshua Wallin, Atharva Shukla, Ben Boskin, Konstantinos Athanasiou, and Justin Slepak. Whether through collaborations, classes, or informal discussions, each of them contributed to an environment that valued rigor, curiosity, and mutual support.

Students of formal methods working under professors Stavros Tripakis and Thomas Wahl

were also wonderful company and collaborators. I am especially grateful for interactions with Daniel Melcer, William Schultz, Lisa Oakley, Yuhao Zhou, Yijia Gu, and Peizun Liu. Beyond formal methods, I was equally fortunate to share my graduate years with a wider community of thoughtful and generous peers, including Girik Malik, Akshar Varma, Kashif Imteyaz, Baidik Chandra, Bryant Curto, Artem Pelenitsyan, Julia Belyakova, Sara Di Bartolomeo, Dustin Lin, Nathan Partlan, Daniel Patterson, Jethro Sun, Cameron Moy, Andrew Wagner, Ming-Ho Yee, and Lydia Zakynthinou. Their friendship, conversations, and support made Northeastern feel like a home, and my time here far richer than it would have been otherwise.

Just as it takes a village to raise a child, this journey would not have been possible without the support of many people and organizations. My two internships at Amazon with Jared Davis, Vaibhav Sharma, Andrew Gacek and Wolf Honoré were my first exposure to formal methods in an industrial setting, and significantly shaped how I think about applied research and real-world impact. I was fortunate to work with Mitesh Jain, a former student of Pete, at Rivos Inc. (now Meta), whose guidance and perspective on refinement proofs were invaluable. I sincerely thank Yiannis Psaras, Dimitris Vyzovitis, Nishant Das, Adrian Manning, Max Inden and others from ProbeLab for their thoughtful and in-depth discussions on Gossipsub, which greatly informed my understanding of the protocol’s design. I also thank the Ethereum Foundation, especially Tyler Holmes, and to the PhD Dissertation Completion Fellowship for their financial support for portions of this dissertation. I thank the ACL2 community for being welcoming and supportive. I greatly enjoyed participating in the ACL2 Workshops since 2020, and look forward to attending future workshops.

I am deeply thankful to my family for providing an unwavering foundation throughout this journey. Their sacrifices, encouragement, and belief in me allowed me to pursue this path with confidence, even during its most uncertain phases. They gave me the freedom to explore, the strength to persevere, and the reassurance that I was never walking this path alone. I am especially thankful to my sister Shilpa for sending me rakhis every year.

I am equally grateful to my friends Chandra Shukla, Vivek Maithani, Rakshit Sharma and Arpit Agarwal, who have been with me since my formative years. Beyond academic support, they generously shared guidance on many non-academic matters, including navigating life in the United States and immigration-related challenges. I cherished the opportunities to visit them during short breaks while I was in the U.S., and their enduring friendship has been a constant source of comfort, stability, and joy throughout this journey.

Above all, I am profoundly grateful to my wife, Yoshita. She has been my constant source of strength, patience, and understanding. Through long nights, failed proofs, looming deadlines, and moments of self-doubt, she stood by me with unwavering care and belief, often from across continents. Her support carried me through this journey, and it would not have been possible without her.

As this PhD comes to an end, I see it as a child finally stepping into the world, shaped by the care, guidance, and love of many. While this dissertation marks the conclusion of one chapter, it also represents a beginning, carrying forward the lessons, values, and support of all those who helped raise it. For that, I am profoundly grateful.

# Contents

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <i>List of Figures</i>                                       | 15        |
| <i>List of Tables</i>                                        | 17        |
| <b>I Introduction and Preliminaries</b>                      | <b>19</b> |
| <b>1 Introduction</b>                                        | <b>21</b> |
| 1.1 Background and Context . . . . .                         | 21        |
| 1.2 Gossipsub . . . . .                                      | 22        |
| 1.3 Problem Statement . . . . .                              | 23        |
| 1.4 Approach: Compositional Refinement . . . . .             | 23        |
| 1.5 Contributions . . . . .                                  | 24        |
| 1.6 Thesis Statement . . . . .                               | 24        |
| 1.7 Thesis Organization . . . . .                            | 24        |
| <b>2 Preliminaries</b>                                       | <b>27</b> |
| 2.1 Notations . . . . .                                      | 27        |
| 2.2 Transition Systems . . . . .                             | 28        |
| 2.3 Notions of Correctness . . . . .                         | 29        |
| 2.4 Refinement . . . . .                                     | 30        |
| 2.5 Temporal Logic . . . . .                                 | 32        |
| 2.6 The ACL2s Theorem Prover . . . . .                       | 32        |
| 2.7 Commitment Implies Delivery Property . . . . .           | 33        |
| <b>II Feasibility of Compositional Refinement</b>            | <b>35</b> |
| <b>3 Formalization and Verification of Floodsub</b>          | <b>37</b> |
| 3.1 Formal Description of the Broadcastsub model . . . . .   | 38        |
| 3.2 Formal Description of the Floodsub model . . . . .       | 40        |
| 3.3 Correctness Conditions for Floodsub . . . . .            | 41        |
| 3.3.1 Good Floodsub states . . . . .                         | 47        |
| 3.4 Choosing a refinement map . . . . .                      | 47        |
| 3.5 The Refinement Proof . . . . .                           | 48        |
| 3.6 Discussion on the Connectedness Condition . . . . .      | 57        |
| 3.6.1 Temporal Connectedness Under Empirical Churn . . . . . | 58        |

|            |                                                                          |            |
|------------|--------------------------------------------------------------------------|------------|
| <b>4</b>   | <b>Mechanized proof of Refinement of Floodsub</b>                        | <b>61</b>  |
| 4.1        | Model Descriptions . . . . .                                             | 61         |
| 4.1.1      | Broadcastnet . . . . .                                                   | 62         |
| 4.1.2      | Floodnet . . . . .                                                       | 66         |
| 4.2        | Correctness and the Refinement Theorem . . . . .                         | 70         |
| 4.3        | Proof Organization . . . . .                                             | 76         |
| 4.4        | Bibliographic Notes . . . . .                                            | 77         |
| <b>5</b>   | <b>Formalization and Verification of Meshsub</b>                         | <b>79</b>  |
| 5.1        | Formal Description of the Meshsub Model . . . . .                        | 80         |
| 5.2        | Correctness Conditions for Meshsub . . . . .                             | 86         |
| 5.3        | Correctness of Meshsub . . . . .                                         | 86         |
| 5.3.1      | Choosing a refinement map . . . . .                                      | 87         |
| 5.3.2      | High-Probability Gossip Coverage Bound . . . . .                         | 89         |
| 5.3.3      | Probabilistic Skipping Refinement . . . . .                              | 91         |
| 5.3.4      | The Refinement Proof . . . . .                                           | 91         |
| 5.4        | Discussion: Meshsub Refines Broadcastsub . . . . .                       | 103        |
| <b>6</b>   | <b>Formalization and Property-Based Testing of Gossipsub v1.1</b>        | <b>105</b> |
| 6.1        | GossipSub ACL2s Model Description . . . . .                              | 106        |
| 6.2        | Reasoning about the scoring function . . . . .                           | 115        |
| 6.3        | Discussion: Relation to Floodsub and Meshsub Correctness Arguments . . . | 118        |
| <b>III</b> | <b>Effectiveness of the Combined Methodology</b>                         | <b>121</b> |
| <b>7</b>   | <b>Model based Security Analysis of Gossipsub v1.1</b>                   | <b>123</b> |
| 7.1        | Attack Gadgets . . . . .                                                 | 124        |
| 7.2        | Eth2.0 Topologies . . . . .                                              | 125        |
| 7.3        | Experimental Setup . . . . .                                             | 125        |
| 7.3.1      | Eclipse Attacks . . . . .                                                | 127        |
| 7.3.2      | Partition Attacks . . . . .                                              | 127        |
| 7.4        | Discussion: Attacks as Refinement Counterexamples . . . . .              | 128        |
| <b>8</b>   | <b>Lean Gossip - Simulation-Based Optimization of Gossipsub</b>          | <b>129</b> |
| 8.1        | Methodology . . . . .                                                    | 130        |
| 8.1.1      | Gossipsub Overview . . . . .                                             | 131        |
| 8.1.2      | Model Simplifications . . . . .                                          | 132        |
| 8.2        | Protocol Implementation . . . . .                                        | 132        |
| 8.2.1      | Aggregate Message Workload . . . . .                                     | 132        |
| 8.2.2      | Network Model . . . . .                                                  | 133        |
| 8.2.3      | Simulation Timing . . . . .                                              | 134        |
| 8.2.4      | Bandwidth Model . . . . .                                                | 135        |
| 8.2.5      | Evaluation Metrics . . . . .                                             | 135        |
| 8.2.6      | Simulation Algorithm . . . . .                                           | 137        |

|           |                                                              |            |
|-----------|--------------------------------------------------------------|------------|
| 8.2.7     | Pareto Dominance Analysis . . . . .                          | 137        |
| 8.2.8     | Simulation Parameters . . . . .                              | 138        |
| 8.2.9     | Validation Against Published Measurements . . . . .          | 138        |
| 8.3       | Simulation Results . . . . .                                 | 142        |
| 8.3.1     | Effect of Mesh Degree . . . . .                              | 142        |
| 8.3.2     | Effect of Gossip Degree . . . . .                            | 144        |
| 8.3.3     | Best-effort Trade-offs and Dominance . . . . .               | 144        |
| 8.3.4     | Dominance Over Ethereum’s Deployed Configuration . . . . .   | 147        |
| 8.4       | Discussion . . . . .                                         | 147        |
| 8.4.1     | Why Floodsub Is Strictly Dominated . . . . .                 | 148        |
| 8.4.2     | Why Lean Gossip Dominates Ethereum’s Configuration . . . . . | 148        |
| 8.4.3     | Recovery Dominates Redundancy . . . . .                      | 148        |
| 8.4.4     | Practical Recommendations . . . . .                          | 148        |
| 8.5       | Limitations and Future Work . . . . .                        | 149        |
| 8.6       | Bibliographic Notes . . . . .                                | 150        |
| <b>IV</b> | <b>Conclusion</b>                                            | <b>153</b> |
| <b>9</b>  | <b>Conclusion</b>                                            | <b>155</b> |
| 9.1       | Future Work . . . . .                                        | 156        |
|           | <b>References</b>                                            | <b>157</b> |
|           | <b>Index</b>                                                 | <b>165</b> |
|           | <b>Vita</b>                                                  | <b>167</b> |



# List of Figures

|                                                                                |     |
|--------------------------------------------------------------------------------|-----|
| 3.1.1 Example of a message broadcast 1 . . . . .                               | 39  |
| 3.3.1 <b>Floodsub</b> network illustration . . . . .                           | 43  |
| 3.3.2 Example message broadcast 2 . . . . .                                    | 43  |
| 3.3.3 Example of message dissemination failure 1 . . . . .                     | 43  |
| 3.3.4 Example of message dissemination failure 2 . . . . .                     | 44  |
| 4.2.1 Trace comparison: Floodnet vs Broadcastnet . . . . .                     | 72  |
| 6.1.1 Topic mesh in a Gossipsub network . . . . .                              | 106 |
| 6.1.2 Eth2.02.0 community graph . . . . .                                      | 107 |
| 6.2.1 Example Gossipsub network . . . . .                                      | 115 |
| 7.1.1 Example AG <sub>2</sub> attack gadget . . . . .                          | 124 |
| 7.2.1 Ropsten topology visualization . . . . .                                 | 126 |
| 8.3.1 Effect of mesh degree $D$ under varying churn regimes . . . . .          | 143 |
| 8.3.2 Effect of gossip degree $D_{lazy}$ under varying churn regimes . . . . . | 145 |
| 8.3.3 Pareto dominance: worst-case delivery vs. normalized cost . . . . .      | 146 |





# List of Tables

|     |                                                                                                                                   |     |
|-----|-----------------------------------------------------------------------------------------------------------------------------------|-----|
| 3.1 | <code>→Broadcastsub</code> definition. . . . .                                                                                    | 39  |
| 3.2 | <code>→Floodsub</code> definition. . . . .                                                                                        | 42  |
| 5.1 | <code>→Meshsub</code> definition, Part 1 . . . . .                                                                                | 81  |
| 5.2 | <code>→Meshsub</code> definition, Part 2 . . . . .                                                                                | 83  |
| 5.3 | Auxiliary <code>Meshsub</code> Transition Definitions . . . . .                                                                   | 84  |
| 6.1 | Perturbations in topic-counters values violating Property 2 . . . . .                                                             | 118 |
| 7.1 | Eth2.0 Network Characteristics . . . . .                                                                                          | 125 |
| 7.2 | Wallclock time (mins) for attack simulation . . . . .                                                                             | 127 |
| 8.1 | Ethereum message sizes . . . . .                                                                                                  | 133 |
| 8.2 | Per-hop latency distributions (ms) by region pair . . . . .                                                                       | 134 |
| 8.3 | Simulation parameter values . . . . .                                                                                             | 139 |
| 8.4 | Validation against Protocol Labs Gossipsub evaluation (1000 nodes, $D = 8$ ,<br>$D_{lazy} = 8$ , 0% churn, 10 ms ticks) . . . . . | 140 |
| 8.5 | Gossipsub configurations dominating Floodsub at each churn level . . . . .                                                        | 147 |
| 8.6 | Representative configurations across the reliability-efficiency-latency spectrum                                                  | 149 |
| 8.7 | Delivery rate by churn level for Gossipsub . . . . .                                                                              | 150 |



# Part I

## Introduction and Preliminaries



# Chapter 1

## Introduction

Gossipsub is the message dissemination protocol underlying Ethereum, IPFS, and Filecoin—systems that collectively support billions of dollars in economic activity and a substantial fraction of today’s decentralized Web3 infrastructure. Despite its central role, Gossipsub lacks a formal specification, its security mechanisms have been evaluated primarily through ad hoc testing and scenario-specific simulation, and its operational parameters were selected heuristically with limited quantitative justification. As a result, fundamental correctness, security guarantees and optimal configurations for these economically significant systems remain unclear.

At its core, any publish-subscribe (pubsub) protocol is expected to satisfy a simple dissemination property:

**Once a message is committed at its source,  
every subscribed peer should eventually receive it.**

Whether this property holds in modern, production-scale peer-to-peer (P2P) systems—and under what conditions—has not been established formally.

This dissertation develops a compositional refinement-based methodology for rigorously analyzing P2P publish-subscribe protocols and applies it to Gossipsub. Our analysis reveals that Gossipsub v1.1 admits executions in which committed messages are never delivered to subscribed peers. We construct concrete eclipse and network partition attacks using attack gadgets that exploit this flaw; following responsible disclosure, the vulnerability was assigned MITRE CVE-2022-47547 and acknowledged by the Gossipsub and Ethereum developers. Beyond exposing this vulnerability, we demonstrate that principled refinement reasoning and systematic simulation enable both formal guarantees and practical parameter optimization for deployed P2P systems.

### 1.1 Background and Context

Peer-to-peer (P2P) systems are decentralized distributed systems in which nodes cooperate to form overlay networks over physical networks such as the Internet [2]. These systems are inherently dynamic: nodes may join or leave at any time, and network membership may

change rapidly due to churn. To achieve scalability and resilience, P2P protocols rely on self-organization and local decision-making.

Publish-subscribe (pubsub) systems constitute a widely studied class of P2P protocols that decouple information producers from consumers by routing messages according to logical topics. A publisher disseminates messages without direct knowledge of subscribers, while the network ensures delivery to interested peers. Early systems such as Scribe [8] implemented pubsub over structured overlays, demonstrating scalable topic-based dissemination.

A canonical and conceptually simple realization of pubsub is provided by *Floodsub* [74, 81]. In Floodsub, nodes maintain knowledge of their neighbors and forward published messages to all neighbors subscribed to the relevant topic. While simple and robust, naive flooding suffers from redundancy: messages may traverse multiple paths and be delivered multiple times, leading to bandwidth explosion. This phenomenon has been studied extensively, including in the context of the broadcast storm problem [67].

To mitigate redundancy, gossip-based (epidemic) dissemination protocols were developed. Seminal work by Demers *et al* [16] showed that limiting fanout to a logarithmic number of peers yields near-complete coverage within  $\mathcal{O}(\log n)$  rounds with high probability under uniform random selection. Subsequent analytical and experimental studies [5, 20] demonstrated that periodic gossip with randomly selected peers enables scalable and resilient dissemination while significantly reducing bandwidth overhead. Modern pubsub systems combine eager message forwarding with gossip-based recovery to balance efficiency and robustness.

## 1.2 Gossipsub

Gossipsub v1.1 [49, 82] represents the state of the art in gossip-based pubsub for Web3 systems. For each topic, a peer maintains a bounded mesh of neighbors to which it eagerly pushes full messages. Peers outside this mesh receive lightweight gossip notifications containing message identifiers and may request the full message if needed. This hybrid push-pull approach reduces bandwidth overhead while preserving resilience.

Gossipsub introduces additional complexity through dynamic mesh management and scoring-based security mechanisms. Peers are added to or removed from topic meshes through grafting and pruning operations. Each peer maintains a local scoring function that evaluates neighbors based on behavioral metrics, including message delivery performance and protocol compliance. Scores influence mesh membership and are parameterized by application-specific weights. These mechanisms aim to mitigate spam, prevent eclipse attacks, and maintain robust connectivity under adversarial conditions.

While these features improve efficiency and security over naive flooding, they significantly increase protocol complexity. Message dissemination, control traffic, scoring updates, and mesh maintenance interact across multiple time scales. Global behavior emerges from local heuristics, making reasoning about end-to-end guarantees difficult. Existing evaluations rely primarily on testing and simulation; no formal specification captures the intended semantics of the protocol.

## 1.3 Problem Statement

Two fundamental challenges arise in modern gossip-based pubsub protocols.

First, correctness and security guarantees remain informal. In the absence of a formal specification, it is unclear whether the intended dissemination property holds, and under what assumptions. Security mechanisms are validated through empirical testing, leaving open the possibility of subtle vulnerabilities.

Second, parameter selection lacks principled justification. Gossipsub exposes numerous parameters controlling mesh degree, gossip fanout, timers, and scoring thresholds. These parameters interact in complex ways, influencing latency, bandwidth consumption, and delivery rate. Production deployments rely on heuristics rather than systematic analysis of trade-offs.

Addressing these challenges requires both formal reasoning and empirical evaluation.

## 1.4 Approach: Compositional Refinement

This dissertation introduces a compositional refinement framework for analyzing P2P pubsub protocols. We construct a hierarchy of protocol layers:

- **Broadcastsub**, an abstract specification capturing ideal dissemination semantics;
- **Floodsub**, a flooding-based realization;
- **Meshsub** (Gossipsub v1.0), which introduces bounded-degree mesh overlays;
- **Gossipsub v1.1**, which adds scoring-based mesh management.

We prove that Floodsub refines Broadcastsub and that Meshsub refines Floodsub. By compositional refinement, Meshsub refines Broadcastsub. Within this hierarchy, we establish the central dissemination guarantee: once a message is committed at its source, every subscribed peer eventually receives it. This property holds deterministically for Broadcastsub and Floodsub, and with arbitrarily high probability for Meshsub under identified network conditions.

We then apply property-based testing and counterexample generation to Gossipsub v1.1. Our analysis demonstrates that the dissemination guarantee is violated: the protocol admits executions in which committed messages are never delivered to subscribed peers. From these counterexamples, we construct concrete eclipse and network partition attacks. Simulation experiments confirm their feasibility. Following responsible disclosure, the vulnerability was assigned CVE-2022-47547.

Finally, we conduct a systematic exploration of the Gossipsub parameter space under realistic churn. Using Pareto analysis over delivery reliability and bandwidth cost, we identify configurations that dominate Ethereum’s deployed parameters and characterize the mechanisms driving reliable dissemination. We show that gossip-based recovery, rather than eager-push redundancy, is the dominant factor under churn.

## 1.5 Contributions

The principal contributions of this dissertation are:

1. The first formal, executable specification of Gossipsub, now recognized as the official reference model.
2. A compositional refinement hierarchy establishing formal dissemination guarantees for Floodsub and Meshsub.
3. The discovery and responsible disclosure of novel attacks against Gossipsub v1.1 (CVE-2022-47547).
4. A systematic parameter analysis demonstrating that production configurations are Pareto-dominated by leaner alternatives under realistic network conditions.
5. A practical methodology combining refinement proofs, property-based testing, counterexample generation, and simulation for analyzing complex, real-world P2P protocols.

## 1.6 Thesis Statement

In this thesis, we use formal methods, specifically compositional refinement proofs, property-based testing, counter-example generation and parameter-driven simulations for exploring (1) vulnerabilities and attacks against Gossipsub networks, and (2) the protocol parameter space to find optimum performance criteria. Following is our thesis statement.

**Thesis statement:** *A compositional refinement-based approach that synergistically combines formal verification, counterexample generation, and parameter-driven simulation is both feasible and effective for the analysis of peer-to-peer (P2P) protocols, enabling the discovery of novel attacks and the systematic improvement of protocol performance metrics.*

## 1.7 Thesis Organization

This dissertation is organized into four parts, each addressing a distinct stage of the compositional refinement-based analysis of gossip-based pubsub protocols.

**Part I: Introduction and Preliminaries** This part introduces the formal framework and core abstractions used throughout the thesis. Chapter 2 presents the modeling assumptions for peer-to-peer networks and the refinement notions employed in our analysis.

**Part II: Feasibility of Compositional Refinement** This part develops formal proofs of correctness for peer-to-peer publish–subscribe protocols using refinement-based verification. Chapter 3 defines Broadcastsub as an abstract dissemination specification and Floodsub as a flooding-based realization, and establishes that Floodsub refines Broadcastsub, yielding deterministic eventual-delivery guarantees under stated assumptions. Chapter 4 shows a mechanized proof of refinement of a simpler variant of Floodsub, where pending messages are stored as sets.



Chapter 5 introduces Meshsub (Gossipsub v1.0) and proves that Meshsub refines Floodsub. By compositional refinement, Meshsub therefore refines Broadcastsub. We establish probabilistic dissemination guarantees for Meshsub under bounded churn and randomized peer selection.

**Part III: Effectiveness of the Combined Methodology** This part analyzes Gossipsub v1.1, focusing on its scoring-based mesh management and security mechanisms. Chapter 6 presents the first formal, executable specification of Gossipsub and applies property-based testing and counterexample generation to the model. In chapter 7, we demonstrate violations of the dissemination guarantee, construct concrete eclipse and network partition attacks, and describe their responsible disclosure as CVE-2022-47547. Chapter 8 explores the Gossipsub parameter space under realistic churn through systematic simulation and Pareto analysis over delivery reliability and bandwidth cost. We identify configurations that dominate production deployments and characterize the mechanisms driving robust dissemination.

**Part IV: Conclusion** Chapter 9 summarizes the results and discusses broader implications, limitations, and directions for future work.



# Chapter 2

## Preliminaries

In this chapter, we introduce the notation used throughout this dissertation and review the notions of correctness and refinement relevant to our work.

### 2.1 Notations

$\mathbb{N}$  denotes the set of natural numbers  $\{0, 1, 2, \dots\}$ . We denote tuples using angled brackets; for example, the tuple consisting of elements  $a_1, a_2, \dots, a_n$  is written as  $\langle a_1, a_2, \dots, a_n \rangle$ . We denote sequences using square brackets; for example, the sequence consisting of elements  $a_1, a_2, \dots, a_n$  is written as  $[a_1, a_2, \dots, a_n]$ . We write  $f(x)$  to denote the value of a function  $f$  applied to an argument  $x$ . The expression  $\#S$  denotes the cardinality of a set  $S$ .

We abbreviate the update  $S := S \cup P$  by  $S:\cup=P$ . Similarly, we abbreviate  $S := S \setminus P$  by  $S:\setminus=P$ . Given a record  $r = \langle \dots, \textit{field}, \dots \rangle$ , we denote the value of the *field* component of  $r$  by  $r.\textit{field}$ . The operator  $::$  denotes list cons and is used to add an element to the beginning of a list.

We use dot notation to invoke functions on structured values. For a tuple  $t = \langle x, y \rangle$ , we write  $t.\textit{first} = x$  and  $t.\textit{second} = y$ .

For a sequence or list  $l$ , we write  $l.\textit{head}$  and  $l.\textit{last}$  to denote its first and last elements, respectively. We write  $l.\textit{tail}$  for the list obtained by removing the first element, and  $l.\textit{init}$  for the list obtained by removing the last element. For a predicate  $p$ , the expression  $l.\textit{filter}(p)$  denotes the subsequence of  $l$  consisting of elements that satisfy  $p$ .

For a queue  $q$ , we write  $q.\textit{enqueue}(x)$  to denote the queue obtained by enqueueing element  $x$  into  $q$ , and  $q.\textit{dequeue}$  to denote the queue obtained by dequeuing the front element of  $q$ . We write  $q.\textit{front}$  to denote the element at the front of  $q$ .

The function  $\textit{count}(s, x)$  denotes the number of occurrences of element  $x$  in the sequence  $s$ . The operators  $\textit{extend}$  and  $\textit{delete}$  denote extension and deletion operations, whose precise behavior will be clear from context.

We use standard conditional expressions of the form  $\textit{if } G \textit{ then } A \textit{ else } B$ , and write  $\textit{otherwise}$  for the default case in piecewise definitions. The construct  $\textit{let } x = e \textit{ in } b$  denotes let-binding, and  $\textit{skip}$  denotes the identity operation.

An expression of the form  $\langle Qx : r : b \rangle$  denotes a quantified expression, where  $Q$  is a quantifier,  $x$  is the bound variable,  $r$  is the range predicate for  $x$  (taken to be  $\textit{true}$  if

omitted), and  $b$  is the body. We sometimes write  $\langle Qx \in X : r : b \rangle$  as an abbreviation for  $\langle Qx : x \in X \wedge r : b \rangle$ , where, as before,  $r$  is *true* if omitted.

For any binary relation  $R$ , we abbreviate  $\langle s, w \rangle \in R$  by  $sRw$ . For a set  $S$ , we write  $R(S)$  for the image of  $S$  under  $R$ , *i.e.*,

$$R(S) = \{ y \mid \exists x \in S : xRy \}.$$

The composition of binary relations  $R$  and  $T$  is denoted  $T \circ R$  and is defined by

$$T \circ R = \{ \langle r, t \rangle \mid \exists x : rRx \wedge xTt \}.$$

The inverse of a binary relation  $R$ , denoted  $R^{-1}$ , is defined as

$$R^{-1} = \{ \langle a, b \rangle \mid bRa \}.$$

Let  $B \subseteq X \times X$  be a binary relation. The relation  $B$  is *reflexive* if  $\langle \forall x \in X : xBx \rangle$ . It is *symmetric* if  $\langle \forall x, y \in X : xBy \Rightarrow yBx \rangle$ . It is *antisymmetric* if  $\langle \forall x, y \in X : xBy \wedge yBx \Rightarrow x = y \rangle$ . It is *transitive* if  $\langle \forall x, y, z \in X : xBy \wedge yBz \Rightarrow xBz \rangle$ .

A binary relation is a *preorder* if it is reflexive and transitive. A preorder that is also symmetric is an *equivalence relation*. A preorder that is antisymmetric is a *partial order*.

If  $\leq_1$  is a partial order on  $X$ ,  $\leq_2$  is a partial order on  $Y$ , and  $f : X \rightarrow Y$ , then  $f$  is *monotonic* if

$$a \leq_1 b \Rightarrow f(a) \leq_2 f(b)$$

for all  $a, b \in X$ .

A well-founded structure is a pair  $\langle W, \triangleleft \rangle$ , where  $W$  is a set and  $\triangleleft$  is a binary relation on  $W$  such that there are no infinitely decreasing sequences in  $W$  with respect to  $\triangleleft$ . We use  $<$  to compare natural numbers and  $\prec$  to compare ordinal numbers.

From highest to lowest binding power, the operators are ordered as follows: parentheses; function application; binary relations (*e.g.*,  $sBw$ ); equality ( $=$ ) and set membership ( $\in$ ); conjunction ( $\wedge$ ) and disjunction ( $\vee$ ); implication ( $\Rightarrow$ ); and finally, binary equivalence ( $\equiv$ ). Whitespace is used to reinforce binding strength. Greater spacing indicates lower binding power. Unless parentheses are used, expressions are parsed according to this precedence order.

## 2.2 Transition Systems

### Definition 1 (Labeled Transition System (LTS))

A labeled transition system (LTS) is a structure  $\langle S, \rightarrow, L \rangle$ , where  $S$  is a non-empty set of states,  $\rightarrow \subseteq S \times S$  is a left-total transition relation, and  $L$  is a labeling function whose domain is  $S$ .

We will often define transitions in an LTS using rules. We treat rules as functions from states to states. A rule of the form  $G \rightarrow A$  denotes the function  $\langle \lambda s :: \text{if } G \text{ then } A \text{ else } s \rangle$ , where  $s$  may occur freely in both  $G$  and  $A$ . If  $A$  itself is used as a rule, we regard it as

an abbreviation for  $\langle \lambda s :: \text{true} \rightarrow A \rangle$ , i.e., the rule that always applies  $A$ . For sequencing transitions, we write  $A \mathbin{\text{;}} B$  to denote the sequential composition of rules. Formally,

$$(A \mathbin{\text{;}} B)(s) = B(A(s)).$$

## 2.3 Notions of Correctness

A notion of correctness specifies what it means for an implementation to faithfully realize its specification. Such notions can be understood as constraints on observable behavior: an implementation is correct exactly when all of its behaviors are permitted by the specification. A useful notion of correctness must be strong enough to rule out obviously incorrect implementations, and permissive enough to accommodate for differences in observed behaviour arising from differing levels of abstraction. We define the following notions of correctness used throughout this dissertation.

### Definition 2 (Simulation)

A relation  $R$  is a *simulation relation* [65] on a transition system  $M = \langle S, \rightarrow, L \rangle$  if  $R \subseteq S \times S$  and for all  $s, w \in S$  such that  $s R w$ , the following conditions hold:

1.  $L(s) = L(w)$ ;
2. for all  $u \in S$ , if  $s \rightarrow u$ , then there exists  $v \in S$  such that  $w \rightarrow v$  and  $u R v$ .

### Definition 3 (Bisimulation)

A relation  $B$  is a *bisimulation relation* [66, 69] on a transition system  $M = \langle S, \rightarrow, L \rangle$  if  $B \subseteq S \times S$  and for all  $s, w \in S$  such that  $s B w$ , the following conditions hold:

1.  $L(s) = L(w)$ ;
2. for all  $u \in S$ , if  $s \rightarrow u$ , then there exists  $v \in S$  such that  $w \rightarrow v$  and  $u B v$ ;
3. for all  $v \in S$ , if  $w \rightarrow v$ , then there exists  $u \in S$  such that  $s \rightarrow u$  and  $u B v$ .

We say that  $s$  is *similar* to  $w$  if there exists a simulation relation  $R$  such that  $s R w$ . We say that  $s$  is *bisimilar* to  $w$  if there exists a bisimulation relation  $B$  such that  $s B w$ . If  $s$  is similar to  $w$  and  $w$  is similar to  $s$ , then it does not imply that  $s$  is bisimilar to  $w$  [70].

Stuttering simulation [56] allows an implementation to take several internal steps while remaining consistent with a single step of the specification, provided that observable behavior is preserved. This notion is intended to capture differences in granularity between abstract and concrete models, where internal behavior of the implementation may not have a direct counterpart at the specification level. Proving a stuttering simulation will require reasoning about infinite sequences, which is not amenable to mechanical verification. For mechanized proofs, it is essential to have localized proof rules. This brings us to Well Founded Simulation, which provides a complete characterization of stuttering simulation.

**Definition 4 (Well-Founded Simulation (WFS) [56])**

$B \subseteq S \times S$  is a well-founded simulation on LTS  $M = \langle S, \rightarrow, L \rangle$  iff:

- **WFS1.**  $\langle \forall s, w \in S :: sBw \Rightarrow L.s = L.w \rangle$ , and
- **WFS2.** There exists functions,  $rankt : S \rightarrow W$ ,  $rankl : S \times S \rightarrow \mathbb{N}$ , such that  $\langle W, \leq \rangle$  is well-founded, and  $\langle \forall s, u, w \in S :: sBw \wedge s \rightarrow u \Rightarrow$ 
  - (c)  $\langle \exists v :: w \rightarrow v \wedge uBv \rangle \quad \vee$
  - (c)  $\langle uBw \wedge rankt(u, w) \leq rankt(s, w) \rangle \quad \vee$
  - (c)  $\langle \exists v :: w \rightarrow v \Rightarrow (sBv \wedge rankl(v, s, u) < rankl(w, s, u)) \rangle$

Reduced well-founded skipping is a notion of correctness that characterizes when an implementation faithfully realizes a specification, while allowing the implementation to advance more rapidly than the specification and preserving observable consistency.

**Definition 5 (Reduced Well-Founded Skipping (RWFSK) [27])**

$B \subseteq S \times S$  is a well-founded skipping relation on LTS  $M = \langle S, \rightarrow, L \rangle$  iff:

- **RWFSK1.**  $\langle \forall s, w \in S :: sBw \Rightarrow L.s = L.w \rangle$ , and
- **RWFSK2.** There exists a function,  $rankt : S \times S \rightarrow W$ , such that  $\langle W, \prec \rangle$  is well-founded, and  $\langle \forall s, u, w \in S :: sBw \wedge s \rightarrow u \Rightarrow$ 
  - (c)  $\langle uBw \wedge rankt(u, w) \prec rankt(s, w) \rangle \vee$
  - (c)  $\langle \exists v :: w \rightarrow^+ v \wedge uBv \rangle$

where  $\rightarrow^+$  denotes the transitive closure of the transition relation.

## 2.4 Refinement

Refinement [1] is a specification method which involves showing that any infinite behavior of a system is related to infinite behaviors of another high-level abstract system that serves as its specification.

Having defined the background for relating states for correctness, we now apply these notions to transition systems. In the following definitions,  $M_A$  can be considered as a specification while  $M_C$  can be considered its concrete implementation.

**Definition 6 (Simulation Refinement [56])**

Let  $M_A = \langle S_A, \xrightarrow{A}, L_A \rangle$ ,  $M_C = \langle S_C, \xrightarrow{C}, L_C \rangle$ , and let  $r : S_C \rightarrow S_A$  be a refinement map. We say that  $M_C$  is a simulation refinement of  $M_A$  with respect to  $r$ , written  $M_C \sqsubseteq_r M_A$ , if there exists a relation,  $B$ , such that all of the following hold.

1.  $\langle \forall s \in S_A :: sBr(s) \rangle$  and
2.  $B$  is a WFS on the LTS  $\langle S_A \uplus S_C, \xrightarrow{C} \uplus \xrightarrow{A}, \mathcal{L} \rangle$ , where  $\mathcal{L}(s) = L_C(s)$  if  $s \in S_C$  else  $\mathcal{L}(s) = L_A(r(s))$ .

**Definition 7 (Abstractly silent suffix)**

Let  $\omega = (s_0, s_1, s_2, \dots)$  be an execution of the concrete system  $\mathcal{M}_C$ , and let  $r : S_C \rightarrow S_A$  be a refinement map. A suffix  $\omega_{\geq k} = (s_k, s_{k+1}, s_{k+2}, \dots)$  is *abstractly silent* if

$$r(s_k) = r(s_{k+1}) = r(s_{k+2}) = \dots$$

That is, the concrete system continues to transition, but the abstract image under  $r$  remains unchanged.

**Definition 8 (Skipping Refinement [27])**

Let  $M_A = \langle S_A, \xrightarrow{A}, L_A \rangle$ ,  $M_C = \langle S_C, \xrightarrow{C}, L_C \rangle$ , and let  $r : S_C \rightarrow S_A$  be a refinement map. We say that  $M_C$  is a skipping refinement of  $M_A$  with respect to  $r$ , written  $M_C \lesssim_r M_A$ , if there exists a relation  $B \subseteq S_C \times S_A$ , such that all of the following hold.

1.  $\langle \forall s \in S_C :: sBr(s) \rangle$  and
2.  $B$  is a RWFSK on the LTS  $\langle S_C \uplus S_A, \xrightarrow{C} \uplus \xrightarrow{A}, \mathcal{L} \rangle$ , where  $\mathcal{L}(s) = L_A(s)$  if  $s \in S_A$  else  $\mathcal{L}(s) = L_A(r(s))$ .

A key benefit of refinement-based correctness notions is compositionality, which supports modular reasoning about complex systems. By proving refinement between successive abstraction levels, one can compose these results to obtain end-to-end correctness. The following theorems capture this property for simulation refinement and skipping refinement.

**Theorem 2.4.1** (Compositional Simulation Refinement [56]). *If  $M_1 \sqsubseteq_p M_2$  and  $M_2 \sqsubseteq_r M_3$ , then*

$$M_1 \sqsubseteq_{rop} M_3.$$

**Theorem 2.4.2** (Compositional Skipping Refinement [29]). *If  $M_1 \lesssim_p M_2$  and  $M_2 \lesssim_r M_3$ , then*

$$M_1 \lesssim_{rop} M_3.$$

## 2.5 Temporal Logic

We use CTL\* (Computational Tree Logic) to express temporal properties over execution traces.

### Definition 9 (CTL\* Operators)

The basic CTL\* temporal operators are:

- $X\phi$  - "neXt":  $\phi$  holds in the next state
- $F\phi$  - "Finally":  $\phi$  holds eventually
- $G\phi$  - "Globally":  $\phi$  holds in all future states

Path quantifiers combine with temporal operators:

- $A$  - "for All paths"
- $E$  - "there Exists a path"

Common combinations include  $AG$  (always globally),  $AF$  (always eventually), and  $AX$  (always next).

We write  $ACTL^* \setminus X$  to denote the fragment of CTL\* with universal path quantification, excluding the next-time operator  $X$ .

## 2.6 The ACL2s Theorem Prover

The ACL2 system, due to Kaufmann and Moore [31, 33, 34], consists of a programming language, a first-order logic, and a theorem prover. ACL2 is an abbreviation for *A Computational Logic for Applicative Common Lisp*.

The term “Computational Logic” refers to ACL2, the logic, which is a first-order logic. The logic can be extended via events, such as by proving new theorems, or by defining new functions. Since the unconstrained introduction of axioms can render the logic unsound, ACL2 has a definitional principle which restricts the functions one can define. The logic also includes rules of inference, including an induction principle that is used to reason about recursively defined functions. The theorem prover, described in the book [34], is used to check proofs mechanically using previously defined axioms, rules of inference and induction principle.

The term “Applicative Common Lisp” refers to the ACL2, the language based on Lisp, which is a applicative, or purely functional programming language. This is precisely the principle of referential transparency: equals may be substituted for equals in any context. Consequently, ACL2 functions are free of side effects in the logic, which simplifies reasoning.

ACL2 has a steep learning curve; for a new user, mastering the system can resemble a learner driver attempting to control a Formula One race car. The “ACL2 Sedan” (ACL2s) saves new users from common mistakes and provides powerful and expressive macros like



`match`, helpful syntax checking, termination analysis, and testing facilities. Developed by Peter Dillinger, Pete Manolios, Harsh Chamarthi and Andrew Walter at Northeastern University, Boston, ACL2S is an extension of the ACL2 theorem prover.

In addition to the capabilities of ACL2, ACL2S provides:

- (1) A powerful type system via the `defdata` data definition framework [13] and the `definec` and `property` forms, which support typed definitions and properties.
- (2) Counterexample generation through the `cgen` framework, which synergistically integrates theorem proving, type reasoning, and testing [10–12].
- (3) Advanced termination analysis based on calling-context graphs (CCGs) [63] and ordinals [60–62].
- (4) An optional Eclipse IDE plugin [9].
- (5) The ACL2S systems programming framework (ASPF) [84], which enables the development of Common Lisp tools that use ACL2, ACL2S, and Z3 as a service [37, 85–87].

## 2.7 Commitment Implies Delivery Property

We define a uniform liveness property relating commitment of a message at its origin to delivery to subscribers.

### Definition 10 (Commitment $\Rightarrow$ Delivery Property)

The *commitment-implies-delivery property* holds for a P2P pubsub system  $\mathcal{M}$  if for every state  $s$ , message  $m$ , and peer  $i$  such that  $m.\text{Topic} \in s(i).\text{Subs}$ ,

$$\text{AF}(m \in s(m.\text{Origin}).\text{Seen}) \implies \text{AF}(m \in s(i).\text{Seen}).$$



## Part II

# Feasibility of Compositional Refinement



## Chapter 3

# Formalization and Verification of Floodsub

Floodsub is a simple, robust and popular P2P pubsub protocol, where nodes can arbitrarily leave or join the network, subscribe to or unsubscribe from topics and forward newly received messages. In this chapter, we introduce a refinement-based approach that formally relates a realistic flooding P2P protocol, Floodsub, to a simpler abstraction, Broadcastsub.

Our specification for Broadcastsub (**Broadcastsub**) comes from the description of a Publish/Subscribe system in the libp2p specification [74], modulo implementation details like peer discovery, connection and topology, and types of peering. Some of the salient features of **Broadcastsub** are:

- **Publish/Subscribe** : All messages are classified into topics. Peers can subscribe to or unsubscribe from topics.
- **Messaging** : Peers can send messages to topics. Each message gets delivered to all peers subscribed to the topic. Message transmission is instantaneous *i.e.*, subscribers receive messages they subscribe to at the same time.
- **Open Network** : Peers are free to leave or join the network. We choose to have peers leaving the network forget their state. Hence, a peer rejoining a network is identical to a completely new peer.

Broadcastsub is a very simple P2P protocol, in fact, it is so simple that it might not even be possible to implement it in the real world, where we need to consider the maximum speed of message transmission across a medium, the network topology, etc. Then what is the point in defining it? The point of having a highly abstract specification like **Broadcastsub**, is that it allows us to state and prove properties about **Floodsub** with ease. For example, if we want to prove a property that every subscribing peer of a message that is in the **Broadcastsub** receives that message, we can prove it simply by referring to the definition of a **Broadcastsub**. For the lower levels of specifications, we can prove the same property given that a refinement map from those specifications to **Broadcastsub** exists, under the conditions required for the refinement map to exist.

We prove that Floodsub implements (refines) Broadcastsub and, consequently, that any safety or liveness property established for Broadcastsub also holds for Floodsub, provided

a temporal network connectedness condition. To our knowledge, this is the first formal result transferring correctness guarantees via refinement over a dynamic network protocol where nodes are free to leave or join. In our work, we generalize the idea of nodes having adequate fanout by expressing the required connectedness of the network as a temporal connectedness condition: the graph induced by message delivery transitions must remain connected over time, even if individual message propagations steps are lossy, or the path along which the message is transmitted is disconnected. This assumption captures classical static connectivity assumptions, extends it to dynamic networks with node churn, and more importantly, satisfies known sufficient conditions from prior work. In particular, it implies the delivery path property used by Birman et al. [5] to analyze probabilistic broadcast, and is consistent with the path redundancy assumptions in Gossipsub. By phrasing this as a temporal predicate over event traces, our refinement-based formulation avoids the need for probabilistic or graph-theoretic reasoning in the correctness proof.

This chapter is organized as follows. Sections 3.1 and 3.2 formalize the Broadcastsub model and the Floodsub model respectively. Section 3.3 states correctness conditions. Section 3.4 explains our choice of refinement map. Section 3.5 develops the refinement proof, including probabilistic analysis of gossip rounds. Section 3.6 concludes with a discussion of the connectedness condition required for our refinement proof.

### 3.1 Formal Description of the Broadcastsub model

Topics are elements of a set  $\mathcal{T}$ , and message payloads are strings. A message  $m$  is a tuple  $\langle \text{Payload}, \text{Topic}, \text{Origin} \rangle$ , where  $m.\text{Payload}$  is the payload,  $m.\text{Topic} \in \mathcal{T}$  is the topic under which the message is published, and  $m.\text{Origin}$  is the peer that originated the message.

We describe a **Broadcastsub** by its state which is a partial map from peers to their states. Each peer is identified by a distinct natural number from  $\mathbb{N}$  (the set of natural numbers). The state for peer  $i \in \text{dom}(S)$  is a tuple  $\langle \text{Subs}, \text{Pubs}, \text{Seen} \rangle_i$ , where  $\text{Subs} \in 2^{\mathcal{T}}$  is a set of topics the peer subscribes to,  $\text{Pubs} \in 2^{\mathcal{T}}$  is a set of topics the peer publishes in, and  $\text{Seen}$  is a set of full message payloads that have been received.

We define the LTS  $\mathcal{M}_{\text{Broadcastsub}}$  for a **Broadcastsub** as the tuple  $\langle S_{\text{Broadcastsub}}, \rightarrow_{\text{Broadcastsub}}, L_{\text{Broadcastsub}} \rangle$  where  $S_{\text{Broadcastsub}}$  is the broadcast network state which is a map from peers to their states:  $\langle \lambda i :: \langle \text{Subs}, \text{Pubs}, \text{Seen} \rangle_i \rangle$ , and  $\rightarrow_{\text{Broadcastsub}}$  is as defined below:

| Rule                                              | Definition                                                                                                                                                                                                                                                                                                                 |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Skip</b>                                       | $\underline{\text{skip}}(s)$                                                                                                                                                                                                                                                                                               |
| <b>Broadcast</b> ( $s, m$ )                       | $m.\text{Origin} \in \text{dom}(s) \wedge m \notin \bigcup_k s(k).\text{Seen} \wedge$<br>$m.\text{Topic} \in s(m.\text{Origin}).\text{Pubs} \rightarrow$<br>$\langle \forall k \in \text{dom}(s) :: (m.\text{Topic} \in s(k).\text{Subs} \vee m.\text{Origin} = k) \Rightarrow$<br>$(s(k).\text{Seen}:\cup=\{m\}) \rangle$ |
| <b>BJoin</b> ( $s, i, \text{subs}, \text{pubs}$ ) | $i \notin \text{dom}(s) \rightarrow s(i) := \langle \text{subs}, \text{pubs}, \emptyset \rangle_i$                                                                                                                                                                                                                         |
| <b>BLeave</b> ( $s, i$ )                          | $i \in \text{dom}(s) \rightarrow \text{dom}(s) :\setminus= \{i\}$                                                                                                                                                                                                                                                          |
| <b>BSubscribe</b> ( $s, i, T$ )                   | $i \in \text{dom}(s) \rightarrow s(i).\text{Subs}:\cup=T$                                                                                                                                                                                                                                                                  |
| <b>BUnsubscribe</b> ( $s, i, T$ )                 | $i \in \text{dom}(s) \rightarrow s(i).\text{Subs}:\setminus=T$                                                                                                                                                                                                                                                             |

Table 3.1:  $\rightarrow \mathbf{Broadcastsub}$  definition.

Suppose  $B$  is a single peer in a **Broadcastsub** network. In each of the following examples, unless otherwise stated, we will assume that the message  $m = \langle p, t, B \rangle$  is carrying a payload  $p$  in topic  $t$  originating at peer  $B$ . We will also assume that each of the peers subscribe to the topic  $t$ . Figure 3.1.1 shows an example of a message broadcast in a **Broadcastsub** consisting of two nodes, 1 and 2.

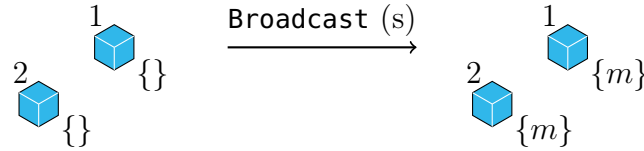


Figure 3.1.1: Example of a message broadcast in a **Broadcastsub**  $s$ . A blue hexagon represents a node in a **Broadcastsub**. Its name appears at the top-left, and the set of seen messages at this node appears at the bottom right.

Each of the network protocol abstractions we discuss in this dissertation implement interleaved semantics, *i.e.*, at each step, only one peer can execute an atomic action. **Broadcast** broadcasts a message to all of its subscribers in a single step. **BJoin** and **BLeave** allow a peer to join or leave the network. Notice that a joining peer has an empty **Seen** set. So, if a peer is rejoining, it will not have any of the messages in its **Seen** cache from before it left the network. Given a set of topics  $T$ , a peer  $i$  either subscribes to or unsubscribes from each of the  $T$  topics using the **BSubscribe** and **BUnsubscribe** transitions respectively. The labeling function  $L_{\mathbf{Broadcastsub}}$  is the **identity** function  $\langle \lambda s :: s \rangle$ . We assume the starting **Broadcastsub** state as an empty map.

We will now prove that the  $\text{Commitment} \Rightarrow \text{Delivery}$  property holds for  $\mathcal{M}_{\text{Broadcastsub}}$ .

**Theorem 3.1.1** (Broadcastsub Commitment Implies Delivery). *The  $\text{Commitment} \Rightarrow \text{Delivery}$  Property 2.7 holds for  $\mathcal{M}_{\text{Broadcastsub}}$ .*

*Proof.* Suppose

$$\text{AF}(m \in s(m.\text{Origin}).\text{Seen})$$

holds along a computation starting at  $s$ .

In Broadcastsub, the only transition that inserts  $m$  into  $s(m.\text{Origin}).\text{Seen}$  is **Broadcast**( $s, m$ ). By definition of **Broadcast**, when it occurs, for every  $k \in \text{dom}(s)$  such that  $m.\text{Topic} \in s(k).\text{Subs}$  or  $k = m.\text{Origin}$ , the successor state satisfies  $m \in s(k).\text{Seen}$ .

Thus, at the same state in which the origin first sees  $m$ , every subscriber sees  $m$  as well. Therefore,

$$\text{AF}(m \in s(i).\text{Seen}).$$

□

## 3.2 Formal Description of the Floodsub model

**Floodsub** is our most abstract specification that is also implemented in libp2p. It considers the network topology and message routing which are necessary to broadcast messages in a physical computer network. The libp2p implementation is simple: for each incoming message, a peer forwards it to all of its known peers in the topic of the message. Each peer maintains a cache of previously received messages such that they are not forwarded any further. Also, peers never forward a message back to its source or to the peer they received it from. Peers forwarding messages to all of their outgoing edges leads to flooding, hence the name **Floodsub**.

Since we assume flawless network connectivity, with no disruptions, we allow each message to be forwarded only once to each of the known peers, which we refer to as *neighbors*. We maintain an ordered (by time of arrival) cache of received messages called a *pending* queue, where each of the received messages are stored until forwarded. When a message is forwarded, it is removed from the pending queue and added to the *seen* set. Each of the above mentioned message stores as well as neighbors and their topic subscriptions are stored in the peer's state.

A **Floodsub** (FS) state is a partial map from peers to their states. Each peer is identified by a distinct natural number. If a peer rejoins, it gets assigned a new number. The state for peer  $i \in \text{dom}(S)$  is a tuple  $\langle \text{Act}, \text{Subs}, \text{Pubs}, \text{Nbrs}, \text{Pdg}, \text{Seen} \rangle_i$ , where **Act** is a boolean signifying whether the node is active in the network (*true*), or has departed (*false*). **Pubs** is a set of topics the peer publishes in, **Nbrs** is a total map of type:  $\mathcal{T} \rightarrow 2^{\mathbb{N}}$  that stores known peers subscribed to each topic, **Pdg** is an unbounded First In First Out (FIFO) queue of message payloads that have yet to be forwarded and **Seen** is a set of message payloads that have been received and forwarded.

We define the following atomic predicates:

$$\text{activep}(i, s) = i \in \text{dom}(s) \wedge s(i).\text{Act}$$



$$hasMsgp(i, m, s) = m \in s(i).Pdgs \cup s(i).Seen$$

$$mNbrp(i, j, m, s) = i \in s(j).Nbrs(m.Topic)$$

$$mSubsp(i, m, s) = m.Topic \in s(i).Subs$$

Given any **Floodsub** state  $s$ , let

**Definition 11 (Pending messages)**

$$\mathcal{P}(s) = \bigcup_j s(j).Pdgs$$

We define the existence of a path in topic  $t$  from node  $i$  to  $j$  as:

$$pathExists(i, j, t, s) = (i = j) \vee \langle \exists k \in s(i).Nbrs(t) :: pathExists(k, j, t, s) \rangle$$

We define a node as being redundant in the network, with respect to a set of topics  $T$  as follows:

$$redundant(i, s, T) \iff \langle \forall t \in T :: \langle \forall j, k : \{j, k\} \subseteq s(i).Nbrs(t) \wedge i \notin \{j, k\} : pathExists(j, k, t, s \setminus \{i\}) \rangle \rangle$$

We define the LTS for a **Floodsub** as the tuple  $\mathcal{M}_{\text{Floodsub}} = \langle S_{\text{Floodsub}}, \rightarrow_{\text{Floodsub}}, L_{\text{Floodsub}} \rangle$  where  $S_{\text{Floodsub}}$  is the **Floodsub** network state which is a function  $\langle \lambda i :: \langle Act, Subs, Pubs, Nbrs, Pdgs, Seen \rangle_i \rangle$ ,  $\rightarrow_{\text{Floodsub}}$  is as defined in Table 3.2.

**FProduce** produces a new message at peer  $i$  which was previously never seen by  $i$ . **FForward** forwards a message in the **Pdgs** set to all the neighboring subscribers, except those who have already seen it. **FJoin** and **FLeave** allow a peer to join or leave the network. **FSubscribe** and **FUnsubscribe** allow a peer to subscribe to or unsubscribe from a given set of topics, respectively. The labeling function  $L_{\text{Floodsub}}$  is the **identity** function  $\langle \lambda s :: s \rangle$ . We assume that the start state of a **Floodsub** is an empty map.

### 3.3 Correctness Conditions for Floodsub

**Broadcastsub** is a specification. In a more realistic network like **Floodsub**, where peers need to communicate on a physical network, a message is not received by its subscribers instantaneously. It might need to hop from the originating peer to its neighboring peers and so on, until it reaches its subscribers.

In the text below, we will use the following pictorial notation to represent the state of **Floodsub** nodes.

Suppose  $O, X$  and  $Y$  are peers in a **Broadcastsub** network which subscribe to the same topics. In each of the following examples, unless otherwise stated, we will assume that the message  $m = \langle p, t, O \rangle$  is carrying a payload  $p$  in topic  $t$  originating at peer  $O$ . We will also assume that each of the peers subscribe to the topic  $t$ . Figure 3.3.2 shows an example of a message broadcast in a **Broadcastsub**.

| Rule                                                        | Definition                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{FProduce}(s, m)$                                     | $\text{activep}(m.\text{Origin}, s) \wedge m.\text{Topic} \in s(m.\text{Origin}).\text{Pubs} \wedge$ $\bigwedge_{i \in \text{dom}(s)} \neg \text{hasMsgp}(i, m, s) \rightarrow s(m.\text{Origin}).\text{Pdg}.\underline{\text{enqueue}}(m)$                                                                                                                                                                                                                        |
| $\text{FForward}(s, i)$                                     | $\text{activep}(i, s) \wedge \neg s(i).\text{Pdg}.\underline{\text{empty}} \rightarrow \text{let } m = s(i).\text{Pdg}.\underline{\text{front}} \text{ in}$ $s(i).\text{Pdg}.\underline{\text{dequeue}} \wedge s(i).\text{Seen} : \cup = \{m\} \wedge$ $\langle \forall k : m.\text{Nbrp}(k, i, m, s) \wedge \neg \text{hasMsgp}(k, m, s) : s(k).\text{Pdg}.\underline{\text{enqueue}}(m) \rangle$                                                                 |
| $\text{FJoin}(s, i, \text{Subs}, \text{Pubs}, \text{Nbrs})$ | $i \notin \text{dom}(s) \wedge i \notin \text{rng}(\text{Nbrs}) \wedge$ $\langle \forall t \in \text{subs} \cup \text{pubs} : \langle \exists j \neq i : j \in \text{Nbrs}(t) \rangle \rangle \rightarrow$ $\langle \forall t \in \text{Subs} : \langle \forall k \in \text{Nbrs}(t) : s(k).\text{Nbrs}(t) : \cup = \{i\} \rangle \rangle \wedge$ $s(i) := \langle \underline{\text{true}}, \text{Subs}, \text{Pubs}, \text{Nbrs}, \emptyset, \emptyset \rangle_i$ |
| $\text{FLeave}(s, i)$                                       | $\text{activep}(i, s) \wedge s(i).\text{Pdg}.\underline{\text{empty}} \wedge \text{redundant}(i, s, s(i).\text{Subs}) \rightarrow$ $\langle \forall t \in s(i).\text{Subs} :: \langle \forall k \in s(i).\text{Nbrs}(t) : s(k).\text{Nbrs}(t) : \setminus = \{i\} \rangle \rangle$ $\wedge s(i).\text{Act} := \underline{\text{false}}$                                                                                                                            |
| $\text{FSubscribe}(s, i, T)$                                | $\text{activep}(i, s) \rightarrow s(i).\text{Subs} : \cup = T$ $\wedge \langle \forall t \in T :: \langle \forall k \in \text{rng}(s(i).\text{Nbrs}) :: s(k).\text{Nbrs}(t) : \cup = \{i\} \rangle \rangle$                                                                                                                                                                                                                                                        |
| $\text{FUnsubscribe}(s, i, T)$                              | $\text{activep}(i, s) \wedge \text{redundant}(i, s, T) \rightarrow s(i).\text{Subs} : \setminus = T$ $\wedge \langle \forall t \in T :: \langle \forall k \in \text{rng}(s(i).\text{Nbrs}) :: s(k).\text{Nbrs}(t) : \setminus = \{i\} \rangle \rangle$                                                                                                                                                                                                             |

Table 3.2:  $\rightarrow \text{Floodsub}$  definition.

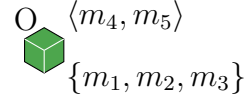


Figure 3.3.1: The green hexagon represents a single **Floodsub** node, with its name  $O$  displayed at the top. The contents of its **Pdg** and **Seen** seen sets are shown at the top right and bottom right, respectively.

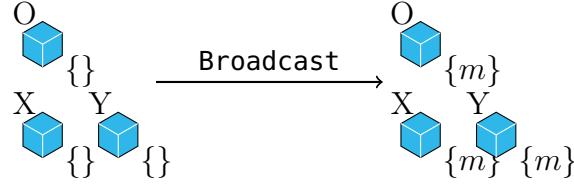


Figure 3.3.2: Example of a message broadcast in a **Broadcastsub**

We will go through a series of examples to understand the conditions under which we can show correctness.

**Example 1.** Consider two peers  $O$  and  $X$  as shown. Suppose that  $O$  and  $X$  subscribe to  $m.Topic$  but are not connected in topic  $m.Topic$  i.e.  $O.Nbrs(m.Topic) = \emptyset = X.Nbrs(t)$ .

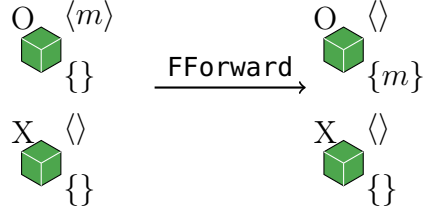


Figure 3.3.3: Example showing message dissemination failure in a disconnected **Floodsub**

In this example, we see that  $m$  was not fully propagated, with  $X$  yet to receive it. Hence it is clear that the subscribers of each topic should be connected in that topic.

**Example 2.** Suppose we have a **Floodsub** consisting of peers  $O, X$  and  $Y$  as shown, all of which are connected. However, before  $X$  could forward the message  $m$  to its neighboring peer  $Y$ , it left the network. As a result  $m$  was not fully propagated.

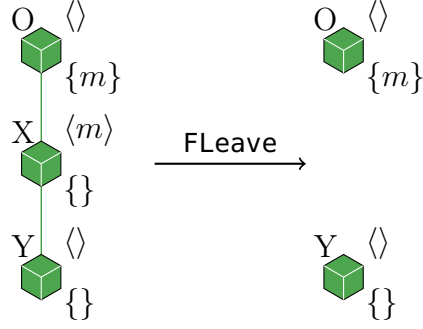


Figure 3.3.4: Example showing message dissemination failure due to a leaving peer

The issue here is that even though we started with a connected network, **Floodsub** peers are free to leave or join the network or subscribe to or unsubscribe from topics at any time, affecting network connectivity. There are four transitions that can potentially change the path of message flow: **FLeave**, **FJoin**, **FSubscribe** and **FUnsubscribe**. We need to make sure that the network remains connected in each topic, while allowing all of these transitions. This does not mean that we require that the network be connected in every topic at all times. What we need is for the network to be dynamically connected over time, in each topic *i.e.*, have temporal paths.

#### Definition 12 (Temporal Graph)

A sequence of time-indexed graphs  $[G_1, G_2, \dots]$  where each graph  $G_t = \langle V_t, E_t \rangle$  is a pair of sets of its vertices  $V_t$  and edges  $E_t$  at time  $t$ .

#### Definition 13 (Temporal Path)

Given a temporal graph  $G = [\langle V_1, E_1 \rangle, \langle V_2, E_2 \rangle, \dots]$ , a temporal path from node  $u$  to node  $v$  is a finite sequence of time-indexed edges  $P = [(v_0, v_1)_{t_0}, (v_1, v_2)_{t_1}, \dots, (v_{n-1}, v_n)_{t_n}]$  such that

- $u = v_0 \wedge v = v_n$ ,
- $\langle \forall (v_i, v_{i+1})_{t_j} \in P :: (v_i, v_{i+1}) \in E_{t_j} \rangle$  and,
- $t_0 < t_1 < \dots < t_n$

We will define temporal paths for messages in context of a **Floodsub**. A peer  $k$  can forward  $m$  to  $i$  if the following condition is satisfied:

$$\text{canForward}(k, i, m, s) = \text{activep}(k, s) \wedge \text{activep}(i, s) \wedge \text{mnbrp}(i, k, m, s) \wedge \neg \text{hasMsgp}(i, m, s)$$

We detect a **FForward** transition at peer  $i$  in state  $s$  using the following definition:

$$\text{FForwardOccurs}(i, s) = \text{let } m = s(i).\text{Pdgc.front} \text{ in } \text{X}(m \in s(i).\text{Seen} \wedge m \notin s(i).\text{Pdgc})$$

Below we define temporal reachability of nodes in a **Floodsub** indexed by distance from the originating peer. The base case  $mT\text{Reachable}_0(m, i, s)$  says that the originating peer of

a message  $m$  is reachable if after the message is produced, it eventually reaches its **Seen** set. The inductive case  $mTReachable_{n+1}(m, i, s)$  says that the message is reachable to a node  $i$  at a distance  $n + 1$  from the origin if either it is already reachable at a distance  $n$  or if there exists a peer  $k$  such that  $k$  is reachable at a distance  $n$  such that eventually  $k$  will be in a position (1) to forward  $m$  to  $i$  (structural condition defined above), (2)  $m$  will be the front message in its **Pdg** queue (3) the **FForward** transition occurs after which in the resulting state  $i$  receives  $m$ , and (4) eventually  $m$  reaches the **Seen** set of  $i$ .

$$\begin{aligned}
mTReachable_0(m, i, s) &= (i = m.\text{Origin}) \wedge \text{AG}(m \in s(i).\text{Pdg} \Rightarrow \text{AF}(m \in s(i).\text{Seen})) \\
mTReachable_{n+1}(m, i, s) &= mTReachable_n(m, i, s) \vee \\
&\quad \langle \exists k :: mTReachable_n(m, k, s) \wedge \\
&\quad \text{AF}(\text{canForward}(k, i, m, s) \wedge m \in s(k).\text{Pdg} \wedge m = s(k).\text{Pdg}.\text{front} \wedge \\
&\quad \text{FForwardOccurs}(k, s) \wedge \text{AX}(\text{hasMsgp}(i, m, s) \wedge \text{AF}(m \in s(i).\text{Seen}))) \rangle
\end{aligned}$$

#### Definition 14 (Temporal Connectedness)

A **Floodsub** state is temporally connected if for every pending message  $m$  and every active subscriber that does not already have  $m$ , there exists some distance  $n$  such that  $i$  is temporally reachable by  $m$  at distance  $n$ .

$$\begin{aligned}
TConnectedp(s) &= \langle \forall m \in \mathcal{P}(s) :: \langle \forall i \in \text{dom}(s) : msubsp(i, m, s) \wedge \\
&\quad \text{activep}(i, s) \wedge \neg \text{hasMsgp}(i, m, s) : \langle \exists n \in \mathbb{N} :: mTReachable_n(m, i, s) \rangle \rangle \rangle
\end{aligned}$$

**Theorem 3.3.1 (Message Delivery Under Temporal Connectedness).** *For any temporally connected **Floodsub** state  $s$ , message  $m$  and active subscriber  $i$ ,  $TConnectedp(s) \wedge m \in \mathcal{P}(s) \wedge msubsp(i, m, s) \wedge \text{activep}(i, s) \wedge \neg \text{hasMsgp}(i, m, s) \Rightarrow \text{AF}(m \in s(i).\text{Seen})$*

*Proof.* The proof follows by induction on the reachability index  $n$ . □

**Lemma 3.3.1.1.** *If  $s$  is temporally connected and  $s \rightarrow u$  is a **FLeave** or **FUnsubscribe** transition at node  $i$  with  $\text{redundant}(i, s, T)$ , then for every pending message  $m$ , all temporal paths witnessing  $mTReachable_n(m, j, s)$  can be reconstructed in  $u$ .*

*Proof.* Let  $s$  be a temporally connected state and suppose  $s \rightarrow u$  is either **FLeave**( $s, i$ ) or **FUnsubscribe**( $s, i, T$ ) with  $\text{redundant}(i, s, T)$  holding. Let  $m \in \mathcal{P}(s)$  be a pending message and let  $j$  be an active subscriber of  $m$  in  $s$  such that  $mTReachable_n(m, j, s)$  holds for some  $n$ , witnessed by a temporal path  $P$ .

We proceed by case analysis on whether  $P$  uses node  $i$ .

**Case 1:  $P$  does not traverse  $i$ .** The temporal path  $P$  consists of edges between nodes other than  $i$ . Since **FLeave** and **FUnsubscribe** only modify edges incident to  $i$  (specifically, removing  $i$  from its neighbors' **Nbrs** sets and vice versa), all edges in  $P$  remain present in  $u$ . Furthermore:

- All nodes on  $P$  remain active in  $u$  (only  $i$  may become inactive, and  $i \notin P$ ).
- The **Pdg** and **Seen** sets of nodes on  $P$  are unchanged by the transition.
- $\text{canForward}(k, \ell, m, u)$  for consecutive nodes  $k, \ell$  on  $P$  remain satisfied.

Hence  $P$  directly witnesses  $mT\text{Reachable}_n(m, j, u)$ .

**Case 2:  $P$  traverses  $i$ .** Let  $P = [v_0, v_1, \dots, v_k = i, v_{k+1}, \dots, v_n = j]$  where  $v_0 = m.\text{Origin}$ . Since  $i$  appears on the path, we have  $v_{k-1} \in s(i).\text{Nbrs}(m.\text{Topic})$  and  $v_{k+1} \in s(i).\text{Nbrs}(m.\text{Topic})$ .

By the guard of **FLeave** (resp. **FUnsubscribe**), we have  $\text{redundant}(i, s, T)$  where  $m.\text{Topic} \in T$  (for **FLeave**,  $T = s(i).\text{Subs}$ ; for **FUnsubscribe**,  $T$  is the unsubscription set, and by Definition 3.3,  $m.\text{Topic} \in T \subseteq s(i).\text{Subs}$  for  $i$  to be on a temporal path of  $m$ ).

By definition of *redundant*:

$$\{v_{k-1}, v_{k+1}\} \subseteq s(i).\text{Nbrs}(m.\text{Topic}) \wedge i \notin \{v_{k-1}, v_{k+1}\} \Rightarrow \text{pathExists}(v_{k-1}, v_{k+1}, m.\text{Topic}, s \setminus \{i\})$$

Let  $Q = [v_{k-1} = w_0, w_1, \dots, w_r = v_{k+1}]$  be the static path guaranteed by *pathExists* that avoids  $i$ . We construct an alternative temporal path  $P'$  by replacing the segment  $[v_{k-1}, i, v_{k+1}]$  in  $P$  with  $Q$ :

$$P' = [v_0, \dots, v_{k-1} = w_0, w_1, \dots, w_r = v_{k+1}, \dots, v_n = j]$$

We verify that  $P'$  is a valid temporal path in  $u$ :

1. **Structural connectivity.** Each edge  $(w_\ell, w_{\ell+1})$  in  $Q$  exists in  $s \setminus \{i\}$  by construction, hence in  $u$ . The edges in the prefix  $[v_0, \dots, v_{k-1}]$  and suffix  $[v_{k+1}, \dots, v_n]$  do not involve  $i$  (since  $i = v_k$  appears only once on  $P$ ), so they persist in  $u$ .
2. **Node activity.** All nodes on  $Q$  are active in  $s$  (since *pathExists* only considers edges between active nodes in the neighbor sets) and remain active in  $u$  (only  $i$  may become inactive).
3. **Temporal ordering.** The original temporal path  $P$  witnesses that  $m$  reaches  $v_{k-1}$  at some time  $t_{k-1}$ , after which forwarding to subsequent nodes occurs. In  $u$ , once  $m$  reaches  $v_{k-1}$ , the path  $Q$  provides a forwarding route to  $v_{k+1}$  that does not require  $i$ . Since all nodes on  $Q$  are active and connected, and  $m$  has not yet reached  $v_{k+1}$  (otherwise the original path would not need  $i$ ), the forwarding sequence along  $Q$  can occur in  $u$  with strictly increasing timestamps.
4. **Forwarding enabledness.** For each consecutive pair  $(w_\ell, w_{\ell+1})$  on  $Q$ :
  - $\text{activep}(w_\ell, u)$  holds (shown above).

- $w_{\ell+1} \in u(w_\ell).\text{Nbrs}(m.\text{Topic})$  holds since the edge exists in  $u$ .
- Once  $m \in u(w_\ell).\text{Pdg}$ , the transition **FForward** $(u, w_\ell)$  enqueues  $m$  into  $u(w_{\ell+1}).\text{Pdg}$  (provided  $\neg \text{hasMsgp}(w_{\ell+1}, m, u)$ , which holds as  $m$  propagates along the path for the first time).

Thus  $P'$  witnesses  $mT\text{Reachable}_{n'}(m, j, u)$  for some  $n' \leq n + r - 2$  (the path length may change but remains finite). In both cases, temporal reachability of  $j$  by  $m$  is preserved in  $u$ . Since  $m$  and  $j$  were arbitrary, we conclude that for all pending messages and all subscribers, the temporal paths witnessing  $mT\text{Reachable}_n$  in  $s$  can be reconstructed (possibly with different witnesses) in  $u$ . Therefore,  $T\text{Connectedp}(u)$  holds.  $\square$

### 3.3.1 Good Floodsub states

We define good **Floodsub** states as those which are temporally connected and well-formed with respect to message forwarding state.

**Definition 15 (Good Floodsub states)**

$$\text{goodfp}(s) = T\text{Connectedp}(s)$$

**Theorem 3.3.2.**  $\text{goodfp}(s) \wedge s \rightarrow_{\text{Floodsub}} u \Rightarrow \text{goodfp}(u)$

*Proof.* Let  $s$  be a state such that  $\text{goodfp}(s)$  holds and suppose  $s \rightarrow_{\text{Floodsub}} u$ .

By Definition 3.3.1, preservation of  $T\text{Connectedp}$  follows from the argument that among all **Floodsub** transitions, only **FLeave** and **FUnsubscribe** can modify the overlay topology, and both require the leaving peer to be redundant. From Lemma 3.3.1.1, temporal connectedness is preserved, so  $T\text{Connectedp}(u)$  holds.  $\square$

Since we want to prove the strongest notion of refinement possible, upto skipping, we are going to use skipping simulation refinement [27–29]. We will achieve this by showing a Reduced Well-Founded Skipping Refinement defined below.

## 3.4 Choosing a refinement map

A refinement proof is relative to a refinement map that maps the implementation (**Floodsub**) states to the specification (**Broadcastsub**) states. It shows us how to view a **Broadcastsub** state as a **Floodsub** state *e.g.* the refinement map needs to hide certain components of the **Floodsub** peer state such as **Nbrs** and **Pdg** sets which do not appear in a **Broadcastsub** state. Simple refinement maps with a clear relationship between implementation states and their image under the map are best [56]. We consider two refinement maps : (1) commitment refinement map and (2) flushing refinement map, both of which have been used in the verification of pipelined microprocessors [30, 55, 57–59].

A commitment refinement map relates a **Floodsub** state  $s$  to a **Broadcastsub** state  $w$  where all the partially propagated messages (that are in the **Pdg** set of any peer in  $s$ ) are removed from the **Seen** sets of all peers in  $w$ . We define **f2b** for nodes in the set of *relevant*

nodes  $A(s)$ , defined as the set of nodes in  $s$  which are either active ( $s(i).\text{Act}$ ) or which left ( $\neg s(i).\text{Act}$ ) but messages that originated from them are still pending in  $s$ . Following is the formal definition of our commitment refinement map:

**Definition 16 (Refinement map from Floodsub to Broadcastsub states)**

$$\begin{aligned} A(s) &= \{i \in \text{dom}(s) \mid \neg s(i).\text{Act} \Rightarrow i \in \{m.\text{Origin} \mid m \in \mathcal{P}(s)\}\}, \\ \text{f2b}(s) &= \langle \lambda i \in A(s) :: \underbrace{\langle s(i).\text{Subs} \cup \{m.\text{Topic} \mid \text{hasMsgp}(i, m, s) \wedge m \in \mathcal{P}(s) \wedge m.\text{Origin} \neq i\},}_{\text{subscriptions}}, \\ &\quad s(i).\text{Pubs}, \\ &\quad s(i).\text{Seen} \setminus \mathcal{P}(s) \rangle_i \rangle \end{aligned}$$

In a **Floodsub** state, message dissemination is not an atomic operation, unlike **Broadcast** in a **Broadcastsub**. Instead, it unfolds over multiple transitions, during which nodes — including those that originated the messages — may leave the network, even though their fingerprints (*i.e.*, pending messages) remain in the state. Hence, we define our refinement map in such a way that while a message is pending and does not show in the image of the refinement map, its originating node does. Similarly, topic subscriptions of nodes on the temporal path of pending messages are preserved by **f2b** until such messages are disseminated. It is clear that given a **Floodsub** state  $s$ , there are no pending messages in the **Seen** sets of **Broadcastsub** peer states of  $\text{f2b}(s)$ . The only messages present are those that have been disseminated or committed. Therefore, this is an example of the commitment approach [56].

Another approach is the flushing refinement map which maps a given **Floodsub** state to a **Broadcastsub** state after all the **Pdg** queues have been flushed. We will show why we did not choose this refinement map with an example. Suppose a message  $m$  is pending in a **Floodsub** state  $s$ . Let there be no path for  $m$  to reach all of its subscribers in  $s$ , unless another node  $p$  joins in a future state and provides a temporal path for  $m$ . If we use the flushing refinement approach, the propagation of  $m$  stalls and  $p$  will never receive  $m$  in the future. This example brings up several issues: (1) **Expressiveness**: the mapping is too coarse, matching only quiet **Floodsub** states where there are no in-flight messages, (2) **Safety reasoning**: since the mapping is too coarse, it might miss safety property violations in the intermediate states, and (3) **Liveness reasoning**: we are concerned about liveness properties like eventual delivery of messages to subscribers, which is ignored.

### 3.5 The Refinement Proof

We prove that **Floodsub** is a skipping refinement of **Broadcastsub**. **f2b** is our refinement map from good **Floodsub** states to **Broadcastsub** states. Let  $S_{\text{GOODFS}} = \{s \mid s \in S_{\text{FS}} \wedge \text{goodfp}(s)\}$ . To show that **Floodsub** is a skipping refinement of **Broadcastsub**, we need to define a relation  $B$  such that  $\langle \forall s \in S_{\text{GOODFS}} :: sB\text{f2b}(s) \rangle$  and show that  $B$  is a Well Formed Skipping simulation (WFSK) on the LTS  $\langle S, \rightarrow, \mathcal{L} \rangle$  where  $S = S_{\text{GOODFS}} \uplus S_{\text{Broadcastsub}}$ ,  $\rightarrow = \rightarrow_{\text{FS}} \uplus \rightarrow_{\text{Broadcastsub}}$  and  $\mathcal{L}(s) = L_{\text{Floodsub}}(\text{f2b}(s))$  where  $s \in S_{\text{Floodsub}}$  and



$\mathcal{L}(s) = L_{\text{Floodsub}}(s)$  otherwise. Given  $s, w \in S$ , and recognizer functions  $\text{fp}$  and  $\text{bp}$  for  $\text{Floodsub}$  and  $\text{Broadcastsub}$  states respectively, we define  $B$  as follows.

**Definition 17 (The Relation  $B$ )**

$$sBw \iff \begin{cases} s = w \\ \vee \text{goodfp}(s) \wedge \text{bp}(w) \wedge \text{f2b}(s) = w \end{cases}$$

**Theorem 3.5.1.**  $\mathcal{M}_{\text{Floodsub}} \lesssim_{\text{f2b}} \mathcal{M}_{\text{Broadcastsub}}$ .

*Proof.*  $\langle \forall s \in S_{\text{Floodsub}} :: sB(\text{f2b}(s)) \rangle$  follows from the definition of  $B$ .

To prove that  $B$  is a Well founded skipping relation on the LTS  $\langle S, \rightarrow, \mathcal{L} \rangle$ , we show the following.

**Goal 3.5.1.1. *RWFSK1*.**

*Proof.* Given  $sBw$ , we have

- if  $s = w$  then  $\mathcal{L}(s) = \mathcal{L}(w)$
- if  $\text{goodfp}(s) \wedge \text{bp}(w)$  then  $\mathcal{L}(s) = L_{\text{Broadcastsub}}(\text{f2b}(s)) = L_{\text{Broadcastsub}}(w) = \mathcal{L}(w)$

□

We will need the following lemmas to prove *RWFSK2*.

**Lemma 3.5.1.1.**  $\langle \forall s : \text{fp}(s) : \langle \forall i : i \in \text{dom}(\text{f2b}(s)) \Leftrightarrow i \in A(s) \rangle \rangle$

*Proof.* Follows from the definitions of  $\text{f2b}$  and  $A$  3.4.

□

**Lemma 3.5.1.2.**

$\langle \forall s : \text{goodfp}(s) : \langle \forall i : i \in A(s) \wedge \neg s(i).\text{Pdg.empty} :$

$$\mathcal{P}(\text{FForward}(s, i)) \neq \mathcal{P}(s) \Leftrightarrow \mathcal{P}(\text{FForward}(s, i)) = \mathcal{P}(s) \setminus \{s(i).\text{Pdg.front}\} \rangle \rangle$$

*Proof.* Let  $m = s(i).\text{Pdg.front}$ . Pong is easy since  $m \in \mathcal{P}(s)$ . We prove ping as follows: Let  $u = \text{FForward}(s, i)$ . From inspection of the  $\text{FForward}$  rule in Table 3.2, we know that the only reason why  $\mathcal{P}(u) \neq \mathcal{P}(s)$  is because  $m$  is dequeued from  $u(i).\text{Pdg}$  and there is no neighboring peer in  $u$  which receives  $m$ . Since  $i \in s(i).\text{Pdg}$ ,  $i$  will not receive its own forwarded message. Hence  $\mathcal{P}(u) = \mathcal{P}(s) \setminus \{m\}$ .

Hence if  $\mathcal{P}(\text{FForward}(s, i)) \neq \mathcal{P}(s)$  then  $\mathcal{P}(\text{FForward}(s, i)) = \mathcal{P}(s) \setminus \{m\}$ .

□

**Goal 3.5.1.2. *RWFSK2*.**

*Proof.* We define a function

$$\text{rankt}(s, w) = \langle \# \{ (i, m) \mid \langle \exists i \in \text{dom}(s) : \langle \exists m \in \mathcal{P}(s) : \text{activep}(i, s) \wedge m \notin s(i).\text{Seen} \rangle \rangle, \\ \# \{ (i, m) \mid \langle \exists i \in \text{dom}(s) : \langle \exists m \in \mathcal{P}(s) : \text{activep}(i, s) \wedge \neg \text{hasMsgp}(i, s) \rangle \rangle \} \rangle \rangle$$

over  $\langle \langle \mathbb{N}, \mathbb{N} \rangle, \prec \rangle$ , where  $\prec$  is the lexicographic ordering over pairs of natural numbers defined as follows:

$$\langle a, b \rangle \prec \langle c, d \rangle \iff a < c \vee (a = c \wedge b < d)$$

Clearly,  $\text{rankt}$  is a well-founded witness.

Given states  $s, w, u \in S \wedge sBw \wedge s \rightarrow u$ . We consider cases on the  $B$  relation.

Case 1. If  $s = w$  then for any transition  $s \rightarrow u$ , we have  $w \rightarrow v$  where  $u = v$  and hence  $uBv$ .

Case 2. Otherwise, let  $\text{goodfp}(s) \wedge \text{bp}(w) \wedge \text{f2b}(s) = w$ . Since  $\text{fp}(s)$ , let  $s \rightarrow_{FS} u$ .

We consider cases on  $\rightarrow_{FS}$ .

• Let

$$\langle \exists m : \text{activep}(m.\text{Origin}, s) \wedge m.\text{Topic} \in s(m.\text{Origin}).\text{Pubs} \\ \wedge \bigwedge_{i \in \text{dom}(s)} \neg \text{hasMsgp}(i, m, s) : u = \text{FProduce}(s, m) \rangle$$

Then,

$$\text{f2b}(u)$$

$\equiv \text{Def. f2b}$

$$\langle \lambda i \in A(u) :: \langle u(i).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(i, e, u) \wedge e \in \mathcal{P}(u) \wedge e.\text{Origin} \neq i\}, \\ u(i).\text{Pubs}, \\ u(i).\text{Seen} \setminus \mathcal{P}(u) \rangle_i \rangle$$

$\equiv \text{activep}(m.\text{Origin}, u)$ ,  $\text{dom}(u) = \text{dom}(s)$  (no node deactivated),  $\mathcal{P}(u) = \mathcal{P}(s) \cup \{m\}$

$$\langle \lambda i \in A(s) :: \langle u(i).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(i, e, u) \wedge e \in \mathcal{P}(u) \wedge e.\text{Origin} \neq i\}, \\ u(i).\text{Pubs}, \\ u(i).\text{Seen} \setminus \mathcal{P}(u) \rangle_i \rangle$$

$\equiv \text{Def. FProduce}$ ,  $\mathcal{P}(u) = \mathcal{P}(s) \cup \{m\}$

$$\langle \lambda i \in A(s) :: \langle s(i).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(i, e, s) \wedge e \in \mathcal{P}(s) \cup \{m\} \wedge e.\text{Origin} \neq i\}, \\ s(i).\text{Pubs}, \\ s(i).\text{Seen} \setminus (\mathcal{P}(s) \cup \{m\}) \rangle_i \rangle$$

$$\equiv \bigwedge_{i \in \text{dom}(s)} \neg \text{hasMsgp}(i, m, s)$$

$$\langle \lambda i \in A(s) :: \langle s(i).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(i, e, s) \wedge e \in \mathcal{P}(s) \cup \{m\} \wedge e.\text{Origin} \neq i\},$$

$$\begin{aligned}
& s(i).\text{Pubs}, \\
& s(i).\text{Seen} \setminus \mathcal{P}(s)\rangle_i \rangle \\
\equiv & \text{Def. f2b}, \quad w = \text{f2b}(s) \\
& w
\end{aligned}$$

So, we have  $\text{f2b}(u) = w$ . Let  $v = \text{Skip}(w) = w = \text{f2b}(u)$ . From Theorem 3.3.2, we have  $\text{goodfp}(u)$ , and therefore  $uBv$ .

- Let  $\langle \exists i : \text{activep}(i, s) \wedge \neg s(i).\text{Pdg.empty} : u = \text{FForward}(s, i) \rangle$ .  
Let  $m = s(i).\text{Pdg.front}$ .

Case (i): Let  $\mathcal{P}(u) = \mathcal{P}(s)$ .

$$\begin{aligned}
& \text{f2b}(u) \\
\equiv & \text{Def. f2b} \\
& \langle \lambda j \in A(u) :: \langle u(j).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(j, e, u) \wedge e \in \mathcal{P}(u) \wedge e.\text{Origin} \neq j\}, \\
& \quad u(j).\text{Pubs}, \\
& \quad u(j).\text{Seen} \setminus \mathcal{P}(u)\rangle_j \rangle \\
\equiv & \text{Def. FForward}, \text{dom}(u) = \text{dom}(s), \mathcal{P}(u) = \mathcal{P}(s) \\
& \langle \lambda j \in A(s) :: \langle s(j).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(j, e, s) \wedge e \in \mathcal{P}(s) \wedge e.\text{Origin} \neq j\}, \\
& \quad s(j).\text{Pubs}, \\
& \quad (\text{if } j = i \text{ then } s(i).\text{Seen} \cup \{m\} \\
& \quad \text{else } s(j).\text{Seen}) \setminus \mathcal{P}(s)\rangle_j \rangle \\
\equiv & \text{Lemma 3.5.1.2}, m \in \mathcal{P}(s) \\
& \langle \lambda j \in A(s) :: \langle s(j).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(j, e, s) \wedge e \in \mathcal{P}(s) \wedge e.\text{Origin} \neq j\}, \\
& \quad s(j).\text{Pubs}, \\
& \quad s(j).\text{Seen} \setminus \mathcal{P}(s)\rangle_j \rangle \\
\equiv & \text{Def. f2b}, \text{f2b}(s) = w \\
& w
\end{aligned}$$

From Theorem 3.3.2,  $\text{goodfp}(u)$ . If  $m \notin s(i).\text{Seen}$  then  $m \in u(i).\text{Seen}$  and hence,  $\text{rankt}(u, w).\text{first} < \text{rankt}(s, w)$ . Otherwise, if  $m \in s(i).\text{Seen}$  then  $\text{rankt}(u, w).\text{first} = \text{rankt}(s, w).\text{first}$  and  $\text{rankt}(u, w).\text{second} < \text{rankt}(s, w).\text{second}$ . Hence, we have  $uBw \wedge \text{rankt}(u) \prec \text{rankt}(s)$ .

Case (ii): Otherwise  $\mathcal{P}(u) \neq \mathcal{P}(s)$ . Consider the following:

- Consider the case where  $\neg s(m.\text{Origin}).\text{Act}$ . Then  $m.\text{Origin} \in \text{dom}(\text{f2b}(s))$ , because  $\text{f2b}$  preserves originating peers of pending messages, however  $m.\text{Origin} \notin$

$\text{dom}(\text{f2b}(u))$ , as  $m \notin \mathcal{P}(u)$ . Hence the single **FForward** step on the **Floodsub** side will need to skip over multiple steps on the **Broadcastsub** side:  $m$  getting broadcast and  $m.\text{Origin}$  leaving  $w$ .

- Similarly, node subscriptions under the refinement map **f2b** include topic subscriptions for pending messages that are present in the corresponding **Pdg** queue or **Seen** set. Once these messages are no longer pending, and if a **Floodsub** node is already unsubscribed from the topics of such messages, the refinement map **f2b** will ignore these topic subscriptions, due to which there may be nodes that do not subscribe to  $m.\text{Topic}$  but have  $m$  in their **Seen** set. Again, this points to the need for multiple steps on the **Broadcastsub** side where corresponding nodes will have to unsubscribe after receiving  $m$ .
- While  $m$  is pending, new nodes can join that may either be on  $m$ 's temporal path or not. There might be some nodes in  $u$  which subscribe to  $m.\text{Topic}$  such that they joined after the state  $s$  and were not on any temporal path on which  $m$  was propagating. Such nodes will not have seen  $m$  in state  $u$ . However, a **Broadcast** on **f2b**( $s$ ) will ensure that all subscribers of  $m$  receive  $m$ . Similarly, there might be some nodes that joined while  $m$  was propagating through the network, and did receive it due to being temporally connected to  $m$ . So subscription to  $m.\text{Topic}$  alone will not inform which nodes receive  $m$ . We will have to check that which nodes actually received  $m$  on the **Floodsub** side, and then **Broadcast** to them.

Due to the above considerations, we will need to take some extra steps from  $w$  to reach a state  $v$  that is related to  $u$ , as shown in the following algorithm:

---

**Algorithm 1** Algorithm to generate a state where  $m$  is broadcast

---

```

 $v \leftarrow w$ 
while  $\exists i \in \text{dom}(u) : m\text{Subsp}(i, m, u) : m \notin u(i).\text{Seen}$  do
   $v \leftarrow \text{BUnsubscribe}(v, i, \{m.\text{Topic}\})$   $\triangleright m\text{Subsp}(i, m, u) \Rightarrow m.\text{Topic} \in v(i).\text{Subs}$  from Def.f2b . These nodes should not receive  $m$ , so unsubscribing.
end while
 $v \leftarrow \text{Broadcast}(v, m)$ 
 $\triangleright$  For pending  $m$ , f2b keeps  $m.\text{Origin}$  and discards  $m$  from Seen sets of mapped nodes
if  $\neg u(m.\text{Origin}).\text{Act}$  then
   $v \leftarrow \text{BLeave}(v, m.\text{Origin})$   $\triangleright m.\text{Origin} \in \text{dom}(v)$  from Def. f2b .
end if
while  $\exists i \in \text{dom}(u) : m\text{Subsp}(i, m, u) : m \notin u(i).\text{Seen}$  do
   $v \leftarrow \text{BSubscribe}(v, i, \{m.\text{Topic}\})$   $\triangleright$  Revert previous unsubscriptions
end while
while  $\exists i \in \text{dom}(u) : \neg m\text{Subsp}(i, m, u) : m \in u(i).\text{Seen}$  do
   $v \leftarrow \text{BUnsubscribe}(v, i, \{m.\text{Topic}\})$   $\triangleright$ 
   $m\text{Subsp}(i, m, u) \wedge m \in u(i).\text{Seen} \Rightarrow m.\text{Topic} \in v(i).\text{Subs}$  from Def.f2b . These nodes should not be subscribed, as in  $u$ .
end while
return  $v$ 

```

---

Each step in Algorithm 1 is enabled by construction from Definition 3.4, and the algorithm preserves **bp** invariants. We justify guards for each of the **Broadcastsub** steps we take in the previous Algorithm 1. By inspection, the algorithm is terminating as the total number of nodes in a network is finite and each while-loop processes each node at most once. Let  $v$  be the **Broadcastsub** state returned from running Algorithm 1 on  $w$ . We will prove  $uBv$ .

From Lemma 3.5.1.2,  $\mathcal{P}(u) = \mathcal{P}(s) \setminus \{m\}$ .

$$\begin{aligned}
& \mathbf{f2b}(u) \\
& \equiv \text{Def. f2b} \\
& \quad \langle \lambda j \in A(u) :: \langle u(j).\mathbf{Subs} \cup \{e.\mathbf{Topic} \mid \text{hasMsgp}(j, e, u) \wedge e \in \mathcal{P}(u) \wedge e.\mathbf{Origin} \neq j\}, \\
& \quad \quad u(j).\mathbf{Pubs}, \\
& \quad \quad u(j).\mathbf{Seen} \setminus \mathcal{P}(u) \rangle_i \rangle \\
& \equiv \text{Def. FForward, } \text{dom}(u) = \text{dom}(s), \mathcal{P}(u) = \mathcal{P}(s) \setminus \{m\} \\
& \quad \langle \lambda j \in A(u) :: \langle s(j).\mathbf{Subs} \cup \{e.\mathbf{Topic} \mid \text{hasMsgp}(j, e, u) \wedge e \in \mathcal{P}(s) \setminus \{m\} \wedge e.\mathbf{Origin} \neq j\}, \\
& \quad \quad s(j).\mathbf{Pubs}, \\
& \quad \quad (\text{if } j = i \text{ then } s(i).\mathbf{Seen} \cup \{m\} \text{ else } s(j).\mathbf{Seen}) \setminus (\mathcal{P}(s) \setminus \{m\}) \rangle_j \rangle \\
& \equiv \text{Simplify} \\
& \quad \langle \lambda j \in A(u) :: \langle s(j).\mathbf{Subs} \cup \{e.\mathbf{Topic} \mid \text{hasMsgp}(j, e, u) \wedge e \in \mathcal{P}(s) \wedge e \neq m \wedge e.\mathbf{Origin} \neq j\}, \\
& \quad \quad s(j).\mathbf{Pubs}, \\
& \quad \quad (\text{if } (j = i \vee m \in s(j).\mathbf{Seen}) \text{ then } (s(j).\mathbf{Seen} \setminus \mathcal{P}(s)) \cup \{m\} \\
& \quad \quad \text{else } s(j).\mathbf{Seen} \setminus \mathcal{P}(s)) \rangle_j \rangle
\end{aligned}$$

Let  $\neg s(m.\mathbf{Origin}).\mathbf{Act}$  and hence  $\neg u(m.\mathbf{Origin}).\mathbf{Act}$ . Since  $m \in \mathcal{P}(s)$  but  $m \notin \mathcal{P}(u)$ , we have  $m.\mathbf{Origin} \in A(s)$  but  $m.\mathbf{Origin} \notin A(u)$  from Def. A. From Lemma 3.5.1.1, we have  $m.\mathbf{Origin} \in A(w)$  and  $m.\mathbf{Origin} \notin A(\mathbf{f2b}(u))$ . This is matched by the **BLeave** step in Algorithm 1 such that  $m.\mathbf{Origin} \notin \text{dom}(v)$ .

Otherwise if  $s(m.\mathbf{Origin}).\mathbf{Act}$  then  $m.\mathbf{Origin} \in \text{dom}(v)$  from Lemma 3.5.1.1 and from Algorithm 1. So, we have  $A(u) = \text{dom}(\mathbf{f2b}(v))$ .

Consider the following exhaustive cases on nodes:

- Let  $m \in u(j).\mathbf{Seen} \wedge m.\mathbf{Topic} \in u(j).\mathbf{Subs}$ : Then  $m \in s(j).\mathbf{Subs}$  too as **FForward** does not affect subscriptions.

$$\begin{aligned}
& \langle \lambda j \in A(u) :: s(j).\mathbf{Subs} \cup \{e.\mathbf{Topic} \mid \text{hasMsgp}(i, e, u) \wedge e \in \mathcal{P}(s) \wedge e.\mathbf{Origin} \neq i\}, \\
& \quad s(j).\mathbf{Pubs}, \\
& \quad (\text{if } (j = i \vee m \in s(j).\mathbf{Seen}) \text{ then } (s(j).\mathbf{Seen} \setminus \mathcal{P}(s)) \cup \{m\} \\
& \quad \text{else } s(j).\mathbf{Seen} \setminus \mathcal{P}(s)) \rangle_j \rangle \\
& \equiv m \in u(j).\mathbf{Seen} \Rightarrow (j = i \vee m \in s(j).\mathbf{Seen}), \text{ Simplify}
\end{aligned}$$

$$\langle \lambda j \in A(u) :: s(j).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(i, e, u) \wedge e \in \mathcal{P}(s) \wedge e.\text{Origin} \neq i\}, \\ s(j).\text{Pubs}, \\ s(j).\text{Seen} \setminus \mathcal{P}(s) \cup \{m\} \rangle_j \rangle$$

$\equiv$  Def. f2b, Algorithm 1, Def. Broadcast

$v$

For the nodes under consideration, Algorithm 1 is the **Broadcast** step, possibly followed by **BLeave** step.

- Let  $m \in u(j).\text{Seen} \wedge m.\text{Topic} \notin u(j).\text{Subs}$ : Then  $m \notin s(j).\text{Subs}$  too as **FForward** does not affect subscriptions.

$$\langle \lambda j \in A(u) :: s(j).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(i, e, u) \wedge e \in \mathcal{P}(s) \wedge e \neq m \wedge e.\text{Origin} \neq i\}, \\ s(j).\text{Pubs}, \\ (\text{if } (j = i \vee m \in s(j).\text{Seen}) \text{ then } (s(j).\text{Seen} \setminus \mathcal{P}(s)) \cup \{m\} \\ \text{else } s(j).\text{Seen} \setminus \mathcal{P}(s)) \rangle_j \rangle$$

$\equiv m \in u(j).\text{Seen} \Rightarrow (j = i \vee m \in s(j).\text{Seen}), \text{ Simplify}$

$$\langle \lambda j \in A(u) :: s(j).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(i, e, u) \wedge e \in \mathcal{P}(s) \wedge e \neq m \wedge e.\text{Origin} \neq i\}, \\ s(j).\text{Pubs}, \\ s(j).\text{Seen} \setminus \mathcal{P}(s) \cup \{m\} \rangle_j \rangle$$

$\equiv$  Def. f2b, Algorithm 1, Def. Broadcast, Def. BUnsubscribe,

$v$

For  $j$  under consideration, Algorithm 1 applies **Broadcast**( $w, m$ ) followed by unsubscription from  $m.\text{Topic}$  and possibly **BLeave**.

- Let  $m \notin u(j).\text{Seen} \wedge m.\text{Topic} \in u(j).\text{Subs}$ . Then  $m \in s(j).\text{Subs}$  too as **FForward** does not affect subscriptions.

$$\langle \lambda j \in A(u) :: s(j).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(i, e, u) \wedge e \in \mathcal{P}(s) \wedge e.\text{Origin} \neq i\}, \\ s(j).\text{Pubs}, \\ (\text{if } (j = i \vee m \in s(j).\text{Seen}) \text{ then } (s(j).\text{Seen} \setminus \mathcal{P}(s)) \cup \{m\} \\ \text{else } s(j).\text{Seen} \setminus \mathcal{P}(s)) \rangle_j \rangle$$

$\equiv m \notin u(j).\text{Seen} \Rightarrow \neg(j = i \vee m \in s(j).\text{Seen}), \text{ Simplify}$

$$\langle \lambda j \in A(u) :: s(j).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(i, e, u) \wedge e \in \mathcal{P}(s) \wedge e.\text{Origin} \neq i\}, \\ s(j).\text{Pubs}, \\ s(j).\text{Seen} \setminus \mathcal{P}(s) \rangle_j \rangle$$

$\equiv$  Algorithm 1

$v$

- Let  $m \notin u(j).\text{Seen} \wedge m.\text{Topic} \notin u(j).\text{Subs}$ : Then  $m \notin s(j).\text{Subs}$  too as **FForward** does not affect subscriptions.

$$\begin{aligned}
& \langle \lambda j \in A(u) :: s(j).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(i, e, u) \wedge e \in \mathcal{P}(s) \wedge e \neq m \wedge e.\text{Origin} \neq i\}, \\
& \quad s(j).\text{Pubs}, \\
& \quad (\underline{\text{if}} (j = i \vee m \in s(j).\text{Seen}) \underline{\text{then}} (s(j).\text{Seen} \setminus \mathcal{P}(s)) \cup \{m\} \\
& \quad \quad \underline{\text{else}} s(j).\text{Seen} \setminus \mathcal{P}(s)) \rangle_j \rangle \\
& \equiv m \notin u(j).\text{Seen} \Rightarrow \neg(j = i \vee m \in s(j).\text{Seen}), \text{Simplify} \\
& \quad \langle \lambda j \in A(u) :: s(j).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(i, e, u) \wedge e \in \mathcal{P}(s) \wedge e \neq m \wedge e.\text{Origin} \neq i\}, \\
& \quad \quad s(j).\text{Pubs}, \\
& \quad \quad s(j).\text{Seen} \setminus \mathcal{P}(s) \rangle_j \rangle \\
& \equiv \text{Def. f2b, Algorithm 1, Def. BUnsubscribe,} \\
& \quad v
\end{aligned}$$

Algorithm 1 applies **BUnsubscribe**, and possible **BLeave** transitions.

From Theorem 3.3.2, we have  $\text{goodfp}(u)$  and hence  $uBv$ .

- Let  $\langle \exists i : \text{activep}(i, s) \wedge s(i).\text{Pdg}.\text{empty} \wedge \text{redundant}(i, s, s(i).\text{Subs}) : u = \text{FLeave}(s, i) \rangle$ .

$$\begin{aligned}
& \text{f2b}(u) \\
& \equiv \text{Def. f2b} \\
& \quad \langle \lambda j \in A(u) :: \langle u(j).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(j, e, u) \wedge e \in \mathcal{P}(u) \wedge e.\text{Origin} \neq j\}, \\
& \quad \quad u(j).\text{Pubs}, \\
& \quad \quad u(j).\text{Seen} \setminus \mathcal{P}(u) \rangle_j \rangle \\
& \equiv s(i).\text{Pdg} = \emptyset \Rightarrow \mathcal{P}(u) = \mathcal{P}(s) \\
& \quad \langle \lambda j \in A(u) :: \langle u(j).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(j, e, u) \wedge e \in \mathcal{P}(s) \wedge e.\text{Origin} \neq j\}, \\
& \quad \quad u(j).\text{Pubs}, \\
& \quad \quad u(j).\text{Seen} \setminus \mathcal{P}(s) \rangle_j \rangle \\
& \equiv \text{Def. FLeave, } \neg u(i).\text{Act}, \text{f2b}(s) = w \\
& \quad \begin{cases} w & \underline{\text{if}} i \in \{e.\text{Origin} \mid e \in \mathcal{P}(s)\} \\ w \setminus \{i\} & \underline{\text{otherwise}} \end{cases}
\end{aligned}$$

From Theorem 3.3.2, we have  $\text{goodfp}(u)$ .

If  $i \in \{e.\text{Origin} \mid e \in \mathcal{P}(s)\}$  then let  $v = \text{Skip}(w) = w = \text{f2b}(u)$ . Then  $uBv$ . Otherwise, let  $v = \text{BLeave}(w, i) = w \setminus \{i\} = \text{f2b}(u)$ . Hence  $uBv$ .

- Let  $\langle \exists i \notin \text{dom}(s), \text{Pubs}, \text{Subs}, \text{Nbrs} :: i \notin \text{rng}(\text{nbrs}) \wedge \langle \forall t \in \text{subs} \cup \text{pubs} :: \langle \exists j :: j \in \text{nbrs}(t) \rangle \rangle \wedge u = \text{FJoin}(s, i, \text{Subs}, \text{Pubs}, \text{Nbrs}) \rangle$

$$\begin{aligned}
& \text{f2b}(u) \\
& \equiv \text{Def.f2b} \\
& \quad \langle \lambda j \in A(u) :: \langle u(j).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(j, e, u) \wedge e \in \mathcal{P}(u) \wedge e.\text{Origin} \neq j\}, \\
& \quad \quad u(j).\text{Pubs}, \\
& \quad \quad u(j).\text{Seen} \setminus \mathcal{P}(u) \rangle_j \rangle \\
& \equiv \text{Def.FJoin}, \mathcal{P}(u) = \mathcal{P}(s), \text{f2b}(s) = w \wedge i \notin \text{dom}(w), \text{Lemma 3.5.1.1} \\
& \quad \underline{\text{extend}}(w, i, \langle \text{Subs}, \text{Pubs}, \emptyset \rangle_i) \\
& \equiv \text{Def.BJoin} \\
& \quad \text{BJoin}(w, i, \langle \text{subs}, \text{pubs}, \emptyset \rangle)
\end{aligned}$$

Let  $v = \text{BJoin}(w, i, \langle \text{subs}, \text{pubs}, \emptyset \rangle)$ . From Theorem 3.3.2, we have  $\text{goodfp}(u)$ . Hence  $uBv$ .

- Let  $\langle \exists i, T : \text{activep}(i, s) \wedge T \in 2^T \wedge \text{redundant}(i, s, T) : u = \text{FUnsubscribe}(s, i, T) \rangle$ .

$$\begin{aligned}
& \text{f2b}(u) \\
& \equiv \text{Def.f2b} \\
& \quad \langle \lambda j \in A(u) :: \langle u(j).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(j, e, u) \wedge e \in \mathcal{P}(u) \wedge e.\text{Origin} \neq j\}, \\
& \quad \quad u(j).\text{Pubs}, \\
& \quad \quad u(j).\text{Seen} \setminus \mathcal{P}(u) \rangle_j \rangle \\
& \equiv \text{Def.FUnsubscribe}, \mathcal{P}(u) = \mathcal{P}(s), \text{dom}(u) = \text{dom}(s) \\
& \quad \langle \lambda j \in A(s) :: \langle u(j).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(j, e, s) \wedge e \in \mathcal{P}(s) \wedge e.\text{Origin} \neq j\}, \\
& \quad \quad s(j).\text{Pubs}, \\
& \quad \quad s(j).\text{Seen} \setminus \mathcal{P}(s) \rangle_j \rangle \\
& \equiv \text{Def.FUnsubscribe} \\
& \quad \langle \lambda j \in A(s) :: (\text{if } i = j \text{ then } \underline{\text{then}} \ s(j).\text{Subs} \setminus T \ \underline{\text{else}} \ s(j).\text{Subs}) \cup \\
& \quad \quad \{e.\text{Topic} \mid \text{hasMsgp}(j, e, s) \wedge e \in \mathcal{P}(s) \wedge e.\text{Origin} \neq j\}, \\
& \quad \quad s(j).\text{Pubs}, \\
& \quad \quad s(j).\text{Seen} \setminus \mathcal{P}(s) \rangle_j \rangle \\
& \equiv \text{f2b}(s) = w, \text{Def.f2b}, \text{Lemma 3.5.1.1}, \text{Def.BUnsubscribe} \\
& \quad \text{BUnsubscribe}(w, i, T \setminus \{e.\text{Topic} \mid \text{hasMsgp}(j, e, s) \wedge e \in \mathcal{P}(s) \wedge e.\text{Origin} \neq j\})
\end{aligned}$$

Let  $v = \text{BUnsubscribe}(w, i, T \setminus \{e.\text{Topic} \mid \text{hasMsgp}(j, e, s) \wedge e \in \mathcal{P}(s) \wedge e.\text{Origin} \neq j\}) = \text{f2b}(u)$ . From Theorem 3.3.2, we have  $\text{goodfp}(u)$ . Hence  $uBv$ .



- Let  $\langle \exists i, T : \text{activep}(i, s) \wedge T \in 2^T : u = \text{FSubscribe}(s, i, T) \rangle$ .

$$\begin{aligned}
& \text{f2b}(u) \\
& \equiv \text{Def.f2b} \\
& \quad \langle \lambda j \in A(u) :: \langle u(j).\text{Subs} \cup \{e.\text{Topic} \mid \text{hasMsgp}(j, e, u) \wedge e \in \mathcal{P}(u) \wedge e.\text{Origin} \neq j\}, \\
& \quad \quad u(j).\text{Pubs}, \\
& \quad \quad u(j).\text{Seen} \setminus \mathcal{P}(u) \rangle_j \rangle \\
& \equiv \text{Def.FSubscribe}, \mathcal{P}(u) = \mathcal{P}(s), \text{dom}(u) = \text{dom}(s) \\
& \quad \langle \lambda j \in A(s) :: (\text{if } i = j \text{ then } s(j).\text{Subs} \cup T \text{ else } s(j).\text{Subs}) \cup \\
& \quad \quad \{e.\text{Topic} \mid \text{hasMsgp}(j, e, s) \wedge e \in \mathcal{P}(s) \wedge e.\text{Origin} \neq j\}, \\
& \quad \quad s(j).\text{Pubs}, \\
& \quad \quad s(j).\text{Seen} \setminus \mathcal{P}(s) \rangle_j \rangle \\
& \equiv \text{f2b}(s) = w, \text{Def.f2b}, \text{Lemma 3.5.1.1}, \text{Def. BSubscribe} \\
& \quad \text{BSubscribe}(w, i, T)
\end{aligned}$$

Let  $v = \text{BSubscribe}(w, i, T) = \text{f2b}(u)$ . From Theorem 3.3.2, we have  $\text{goodfp}(u)$ . Hence  $uBv$ .

□

□

Having proved that Floodsub implements Broadcastsub, we can now prove the following property.

**Theorem 3.5.2** (Floodsub Commitment Implies Delivery). *Under temporal connectedness assumptions, the Commitment $\Rightarrow$ Delivery Property 2.7 holds for  $\mathcal{M}_{\text{Floodsub}}$ .*

*Proof.* By Theorem 3.1.1, the Commitment $\Rightarrow$ Delivery property holds for Broadcastsub. By Theorem 3.5.1,

$$\mathcal{M}_{\text{Floodsub}} \lesssim_{\text{f2b}} \mathcal{M}_{\text{Broadcastsub}}.$$

Skipping refinement preserves all  $ACTL^* \setminus X$  properties. The Commitment $\Rightarrow$ Delivery property is expressible in  $ACTL$ . Hence it is preserved under refinement. □

## 3.6 Discussion on the Connectedness Condition

Recent formal and empirical studies underscore the necessity of temporal connectedness in P2P protocols. The negation of this condition for a Floodsub corresponds to the existence of a message  $m$  and subscriber  $j$  such that  $j$  will never become temporally reachable from any forwarder of  $m$ . This failure mode precisely characterizes several well-known attacks on flooding protocols. For example, Leitao et al. [46] empirically showed that high churn rates or targeted drops can transiently break delivery paths in hybrid epidemic protocols, causing

message loss if the gossip repair mechanism is insufficient. This can further lead to disruption of structural paths of messages, possibly leading to network partition and therefore loss of correctness guarantees.

Coretti et al. [14] emphasize the importance of temporal expansion in dynamic networks: they show that for gossip protocols to tolerate Byzantine adversaries, the underlying network must provide sufficient temporal mixing, thereby ensuring that honest nodes remain connected over time. In contrast, our temporal connectedness condition represents the minimal structural assumption needed for reliable flooding: it suffices that each pending message remains temporally reachable to each intended subscriber. Both approaches reinforce the broader principle that correctness in dynamic networks depends on connectivity over time, not static topology alone. Haeupler et al. [23] show that robust gossip aggregation under adversarial corruption requires temporal redundancy — sufficiently many temporal paths between honest nodes over time to resist Byzantine behavior. In our setting, reliable message flooding requires at least one such path per subscriber. Together, these works demonstrate that enforcing the temporal connectedness predicate is essential for guaranteeing reliable dissemination in real-world, adversarial environments.

By making this structural and temporal connectedness requirement explicit in the form of  $mTReachable_n$ , our formulation provides a clear condition under which correctness fails, and thus a target for both attack mitigation and runtime monitoring.

### 3.6.1 Temporal Connectedness Under Empirical Churn

The refinement proof for Floodsub assumes the temporal connectedness predicate  $TConnectedp(s)$ , which requires that for every pending message  $m$  and every active subscriber  $i$  that has not yet received  $m$ , there exists a temporal path along which  $m$  can reach  $i$ . In this section, we examine whether this assumption is realistic under empirically observed churn patterns.

**Churn Model.** Measurements of peer-to-peer networks [35] report that peer session durations follow heavy-tailed distributions, with median online times on the order of minutes and significant fractions of short-lived peers. Following standard practice, we approximate this behavior with an exponential model calibrated to match observed churn rates. Specifically, we assume peers have independent exponentially distributed online durations with mean  $\tau_{on} = 10$  seconds, yielding a per-second departure rate of

$$\lambda = \frac{1}{\tau_{on}} = 0.1/\text{sec}.$$

This parameterization is deliberately conservative: it captures the behavior of the least stable peers in empirical measurements, while many deployed networks exhibit substantially longer median session times.

**Message Propagation Time.** Let  $D$  denote the mesh degree and  $\Delta$  the overlay diameter. Empirical overlays of the size considered here typically have bounded degree  $D \approx 6$  and diameter  $\Delta \approx 5$ . Message propagation latency per hop is on the order of 100 ms, yielding a

total end-to-end dissemination time of approximately

$$T_{\text{prop}} \approx 0.5 \text{ seconds.}$$

Crucially,  $T_{\text{prop}} \ll \tau_{\text{on}}$ : message propagation completes within a small fraction of the average peer lifetime.

**Single-Path Survival.** Consider a temporal path of length  $\ell \leq \Delta$  from a publisher to a subscriber. For the message to traverse this path, each intermediate node must remain online during the interval in which the message passes through it. Under the exponential model, the probability that a node departs during an interval of length  $\delta$  is  $1 - e^{-\lambda\delta}$ . Since per-hop latency is approximately  $T_{\text{prop}}/\Delta \approx 0.1$  s, each intermediate node must survive only this brief window rather than the entire propagation time.

For a conservative analysis, we assume each of the  $\ell$  intermediate nodes must survive the full propagation time  $T_{\text{prop}}$ . The probability that a single path of length  $\Delta$  survives is then at least

$$p_{\text{path}} = (e^{-\lambda T_{\text{prop}}})^{\Delta} = e^{-\lambda T_{\text{prop}} \Delta} = e^{-0.1 \cdot 0.5 \cdot 5} \approx 0.78.$$

**Redundancy Near the Source.** Flooding-based overlays do not rely on a single path; each node forwards messages to multiple neighbors, creating redundant dissemination paths. The critical observation is that path failures are not independent across subscribers. Instead, dissemination in Floodsub proceeds outward from the publisher in waves, so failures affecting different subscribers are dominated by early structural failures near the source.

If the publisher has degree  $D = 6$ , the probability that at least two neighbors survive the first hop interval ( $\approx 0.1$  s) is

$$\Pr[\text{at least 2 of 6 survive}] = 1 - \sum_{j=0}^1 \binom{6}{j} q^j (1-q)^{6-j} > 0.999,$$

where  $q = e^{-\lambda \cdot 0.1} \approx 0.99$  denotes the probability that a node remains online during one hop interval. Similarly, survival of second-hop neighbors remains high under the exponential churn model.

**Extension to Multiple Subscribers.** The predicate  $TConnected_p$  requires temporal paths to *all* active subscribers. A naive union bound over subscribers would suggest that the failure probability grows linearly in the number of subscribers  $N$ . However, such a bound is overly pessimistic for flooding-based protocols, as it ignores the correlated structure of dissemination.

Let  $E_{\text{early}}$  denote the event that dissemination fails within the first two hops—that is, the message fails to reach a sufficient number of nodes to sustain the flooding process. As shown above, the probability that the publisher retains at least two active neighbors during the first hop exceeds 0.999, and similar reasoning applies to the second hop. Therefore,  $\Pr[E_{\text{early}}]$  is well below 0.01 under the conservative churn parameters used here.

Conditioned on  $\neg E_{\text{early}}$ , the message reaches a substantial fraction of the overlay before significant churn occurs. Since propagation time ( $\approx 0.5$  s) is an order of magnitude smaller

than average session duration (10 s), the vast majority of nodes that are reachable at the start of dissemination remain reachable throughout. Once the message successfully propagates beyond the first few hops, it reaches a large connected component of the overlay with high probability.

Thus, the dominant contribution to  $\Pr[\neg TConnectedp]$  arises from rare early failures near the source, not from independent downstream path failures. The failure probability remains on the order of a few percent even for moderate subscriber sets ( $N \leq 100$ ), under the conservative churn parameters adopted here.

**Implications.** These calculations demonstrate that temporal connectedness holds with high probability under realistic churn conditions. Several factors further reinforce this conclusion:

- **Conservative parameters.** Our churn model uses session times at the pessimistic end of empirical observations. Networks with longer median session times exhibit substantially higher reliability.
- **Retransmission mechanisms.** Protocols such as GossipSub incorporate heartbeat-based state reconciliation and IWANT/IHAVE exchanges, which can recover from transient disconnections not modeled here.
- **Timescale separation.** The order-of-magnitude gap between propagation time and session duration ensures that the overlay remains essentially static during message dissemination.

This probabilistic argument is orthogonal to the semantic refinement proof. The correctness of Floodsub is established assuming  $TConnectedp$ , while the analysis above demonstrates that this assumption is satisfied with high probability under empirically observed churn. Consequently, the refinement result is not predicated on an unrealistic network model, but rather on conditions commonly observed in deployed peer-to-peer systems.

# Chapter 4

## Mechanized proof of Refinement of Floodsub

This chapter presents the mechanized refinement proof of a simplified model of Floodsub introduced in the previous chapter. In this simplified model pending messages are represented using sets rather than queues, which removes explicit ordering and scheduling behavior from the system. As a result, we adopt a different notion of correctness from: instead of skipping refinement, we prove simulation refinement, which is stronger.

Within this setting, we develop a fully mechanized proof in ACL2s showing that Floodsub (with set-based messages) implements Broadcastsub. These proofs provide a clear and rigorous validation of the protocol structure at a semantic level and establish a foundation for the more detailed executable models and refinement-based analyses presented in subsequent chapters. Our ACL2s models and proofs are publicly available in our repository [38]. Overall, our code consists of 476 theorems proved, and 10277 lines of lisp code.

This chapter is organized as follows. Sections 4.1 formalizes the set based message models of Broadcastsub and Floodsub in ACL2s. Section 4.2 states the refinement map and theorem. Section 4.3 explains the refinement proof structure and Section 4.4 concludes with a discussion of related work.

**Chapter provenance.** This chapter is based on work previously published in [40]. The presentation has been revised and extended for inclusion in this dissertation.

### 4.1 Model Descriptions

We model Floodsub and its specification Broadcastsub in ACL2s using transition systems consisting of states and transition relations (boolean functions on 2 states) that depend on transition functions. We call our ACL2s models Floodnet and Broadcastnet respectively. In this section we will explain each of our transition system models in a top-down fashion. The state models are self explanatory. The interesting parts of the following code listings are the transition relations, where we place conditions, not only as sanity checks or for cases, but also as guards to disallow illegal behaviour, like requiring that a peer leaving a network is already in the network. We will discuss such conditions in detail.

### 4.1.1 Broadcastnet

The state of a Broadcastnet is stored in a map from peers to their corresponding peer-states. We represent peers by natural numbers. A Broadcastnet peer state is a record consisting of (i) **pubs** : the set of topics in which a peer publishes, (ii) **subs** : the set of topics to which a peer subscribes to, and (iii) **seen** : the set of messages a peer has already processed. We use sorted ACL2 lists containing unique elements to represent sets. Hence, set equality is reduced to list equality. Messages are records consisting of (i) **pld** : a message payload of type string (ii) **tp** : the topic in which this message was published, and (iii) **or** : the originating peer for this message.

```
(defdata s-bn (map peer ps-bn))
```

```
(defdata-alias peer nat)
```

```
(defdata ps-bn (record
  (pubs . lot)
  (subs . lot)
  (seen . lom)))
```

```
(defdata lot (listof topic))
```

```
(defdata-alias topic var)
```

```
(defdata lom (listof mssg))
```

```
(defdata mssg (record
  (pld . string)
  (tp . topic)
  (or . peer)))
```

We will now define the transition relation for Broadcastnet. The relation **rel-step-bn** relates 2 Broadcastnet states **s** and **u** iff **s** transitions to **u**. **rel-step-bn** is an OR of all the possible ways **s** can transition to **u**. **rel-skip-bn** represents a transition where **s** chooses to skip, hence **u** is **s**.

```
(definec rel-step-bn (s u :s-bn) :bool
  (v (rel-skip-bn s u)
    (rel-broadcast-bn s u)
    (rel-broadcast-partial-bn s u)
    (rel-subscribe-bn s u)
    (rel-unsubscribe-bn s u)
    (rel-leave-bn s u)
    (rel-join-bn s u)))
```

```
(definecd rel-skip-bn (s u :s-bn) :bool
  (== u s))
```

**rel-broadcast-bn** defines a relation between **s** and **u**, where **u** represents the state resulting from broadcasting a message in **s**. The broadcast is modeled as an atomic operation in

which all subscribers receive the message simultaneously. The function (`br-mssg-witness s u`) is a witness finding function that calculates the message that was broadcast, if one exists. Since `seen` is a set of messages implemented as an ordered list with unique elements, `br-mssg-witness` utilizes this ordering of unique messages to find the broadcasted message.

The boolean function `broadcast-bn-pre` is a conjunction of the following preconditions: (i) the broadcast message is new, *i.e.*, it is not already found in the `seen` set of any of the peers in `s`, (ii) the originating peer of the message exists in `s`, and (iii) the topic of the broadcast message is one in which the originating peer publishes messages. `broadcast-bn-pre` also appears as an input contract (`:ic`) in the definition of the `broadcast` transition function. `(= u (broadcast (br-mssg-witness s u) s))` in the definition of `rel-broadcast-bn` ensures that the message found by `br-mssg-witness` was the sole message broadcast in `s`. We use the `insert-unique` function within `broadcast-help` to add new messages while preserving order and uniqueness of the `seen` set.

In the following code snippets, we use some ACL2S syntax described as follows. `^`, `v` and `!` are macros for `and`, `or` and `not` respectively. In a `property` form, the form following the keyword `:h` is the hypothesis, while the form following the keyword `:b` is the body. `(nin a x)` stands for `(not (in a x))`. `:match` is a powerful ACL2S pattern matching capability which supports predicates, including recognizers automatically generated by `defdata`, disjunctive patterns and patterns containing arbitrary code [64]. For more expressive pattern matching, `!` is used for literal match while `&` is used as a wildcard. In a function definition form, `:ic` and `:oc` abbreviate `:input-contract` and `:output-contract` respectively.

```
(definecd rel-broadcast-bn (s u :s-bn) :bool
  (^ (br-mssg-witness s u)
     (broadcast-bn-pre (br-mssg-witness s u) s)
     (= u (broadcast (br-mssg-witness s u) s))))

(definec broadcast-bn-pre (m :mssg s :s-bn) :bool
  (b* ((origin (mget :or m))
       (origin-st (mget origin s)))
    (^ (new-bn-mssgp m s)
       origin-st
       (in (mget :tp m)
           (mget :pubs origin-st)))))

(definec br-mssg-witness (s u :s-bn) :maybe-mssg
  (cond
    ((v (endp s) (endp u)) nil)
    ((= (car s) (car u)) (br-mssg-witness (cdr s) (cdr u)))
    (t (car (set-difference-equal (mget :seen (cdar u))
                                  (mget :seen (cdar s)))))))

(defdata maybe-mssg (v nil mssg))

(definecd new-bn-mssgp (m :mssg s :s-bn) :bool
  (v (endp s)
    (^ (nin m (mget :seen (cdar s))
```

```

        (new-bn-mssgp m (cdr s))))))

(definecd broadcast (m :mssg s :s-bn) :s-bn
  :ic (broadcast-bn-pre m s)
  (broadcast-help m s))

(definecd broadcast-help (m :mssg st :s-bn) :s-bn
  (match st
    (() nil)
    (((p . pst) . rst)
     (cons '(', p . , (if (v (in (mget :tp m) (mget :subs pst))
                             (== p (mget :or m))))
           (mset :seen
                 (insert-unique m (mget :seen pst))
                 pst)
           pst)))
    (broadcast-help m rst))))))

(definec insert-unique (a :all x :tl) :tl
  (match x
    (() (list a))
    ((!a . &) x)
    ((e . es) (if (< a e) (cons a x) (cons e (insert-unique a es))))))

```

We will explain `rel-broadcast-partial-bn`, and its necessity when discussing the proof of correctness later in the paper. `rel-subscribe-bn` and `rel-unsubscribe-bn` relate states `s` and `u` where `u` represents the state obtained after a peer in `s` subscribes to or unsubscribes from a set of topics, respectively. `bn-topics-witness` calculates the peer and the set of topics it subscribes to or unsubscribes from, if there exists such peer. Notice that we reuse `bn-topics-witness` in the definition of `rel-unsubscribe-bn`, with the arguments reversed, so as to find the topics that are subscribed to in `s`, but not in `u`. The calculated set of topics are unioned with or removed from the existing set of peer topic subscriptions of the calculated peer, based on whether it is subscribing or unsubscribing. The definition of `unsubscribe-bn` is analogous to that of `subscribe-bn` and is hence omitted.

```

(definecd rel-subscribe-bn (s u :s-bn) :bool
  (^ (bn-topics-witness s u)
     (mget (car (bn-topics-witness s u)) s)
     (== u (subscribe-bn (car (bn-topics-witness s u))
                         (cdr (bn-topics-witness s u))
                         s))))))

(definecd rel-unsubscribe-bn (s u :s-bn) :bool
  (^ (bn-topics-witness u s)
     (mget (car (bn-topics-witness u s)) s)
     (== u (unsubscribe-bn (car (bn-topics-witness u s))
                           (cdr (bn-topics-witness u s))
                           s))))))

```



```

(definec bn-topics-witness (s u :s-bn) :maybe-ptops
  (cond
    ((v (endp s) (endp u)) nil)
    ((= (car s) (car u)) (bn-topics-witness (cdr s) (cdr u)))
    ((^ (= (caar s) (caar u))
      (set-difference-equal (mget :subs (cdar u))
                           (mget :subs (cdar s)))))
    (cons (caar s)
      (set-difference-equal (mget :subs (cdar u))
                           (mget :subs (cdar s)))))
    (t nil)))

```

```

(defdata maybe-ptops (v nil (cons peer lot)))

```

```

(definecd subscribe-bn (p :peer topics :lot s :s-bn) :s-bn
  :ic (mget p s)
  (let ((pst (mget p s)))
    (mset p (mset :subs (union-equal (mget :subs pst) topics) pst) s)))

```

rel-join-bn and rel-leave-bn relate states *s* and *u* where *u* is obtained after a peer joins *s* or leaves *s*, respectively. **bn-join-witness** calculates the peer and its peer-state, if there exists a peer that joins *s*. Its definition depends on the keys of our Broadcastnet state being in order, which is guaranteed by ACL2S **maps**. A peer joins a Broadcastnet state when a new default Broadcastnet peer state is set for the corresponding peer. A peer leaves a Broadcastnet state when the entry corresponding to the leaving peer is removed from the state.

```

(definecd rel-join-bn (s u :s-bn) :bool
  (^ (bn-join-witness s u)
    (b* ((p (car (bn-join-witness s u)))
      (pst (cdr (bn-join-witness s u)))))
    (^ (! (mget p s))
      (= u (join-bn p (mget :pubs pst) (mget :subs pst) s))))))

```

```

(definecd rel-leave-bn (s u :s-bn) :bool
  (^ (bn-join-witness u s)
    (mget (car (bn-join-witness u s)) s)
    (= u (leave-bn (car (bn-join-witness u s)) s))))

```

```

(definec bn-join-witness (s u :s-bn) :maybe-ppsbns
  (match (list s u)
    ((()) ((q . qst) . &)) '(),q . ,qst))
    (((p . pst) . rs1) ((q . qst) . rs2))
    (cond
      ((= '(),p . ,pst) '(),q . ,qst)) (bn-join-witness rs1 rs2))
      ((!= q p) '(),q . ,qst)) ;; Joining peer found
    (t nil)))

```

```

(& nil)))

(defdata maybe-ppsb (v nil (cons peer ps-bn)))

(definecd join-bn (p :peer pubs subs :lot s :s-bn) :s-bn
  :ic (! (mget p s)) ;; Join only if peer does not already exist in state
  (mset p (ps-bn pubs subs '()) s))

(definecd leave-bn (p :peer s :s-bn) :s-bn
  :ic (mget p s) ;; Leave only if peer already exists in state
  (match s
    (((!p . &) . rst) rst)
    ((r . rst) (cons r (leave-bn p rst)))))

```

### 4.1.2 Floodnet

A Floodnet peer-state is a record consisting of sets **pubs**, **subs** and **seen** which we described previously in context of Broadcastnet peer-states. It also consists of **pending**, which is a set of messages that have not yet been processed, and **nsubs**, a map from topics to list of peers. **nsubs** stores topic subscriptions for neighboring peers.

```

(defdata s-fn (map peer ps-fn))

(defdata ps-fn
  (record (pubs . lot)
    (subs . lot)
    (nsubs . topic-top-map)
    (pending . lom)
    (seen . lom)))

```

When a message has not been forwarded to neighboring subscribers (processed) it remains in the **pending** set. Once it is processed, it is added to the **seen** set. In our Floodnet model, **pending** and **seen** are sets of messages, instead of queues. This simplifies the model and allows us to not worry about the order in which messages are received. Related states have equal sets of **seen** messages.

We define the transition relation **rel-step-fn** which relates two Floodnet states **s** and **u** iff **s** transitions to **u**. It encodes all the possible ways **s** can transition to **u**. **rel-skip-fn** represents a transition where **s** chooses to skip, hence **u** is **s**.

```

(definecd rel-step-fn (s u :s-fn) :bool
  (v (rel-skip-fn s u)
    (rel-produce-fn s u)
    (rel-forward-fn s u)
    (rel-subscribe-fn s u)
    (rel-unsubscribe-fn s u)
    (rel-leave-fn s u)
    (rel-join-fn s u)))

```

```
(defnecd rel-skip-fn (s u :s-fn) :bool
  (= u s))
```

`rel-produce-fn` relates `s` and `u` where `u` represents the state obtained after a new message has been produced in `s`. The newly produced message is one of the pending messages in `u`. The boolean function `produce-fn-pre` is a conjunction of the following preconditions: (i) the produced message is new *i.e.*, it is not already found in the `seen` or `pending` sets of any of the peers in `s`, (ii) the originating peer of the message exists in `s`, and (iii) the topic of the produced message is one in which the originating peer publishes messages. The new message is added to the set of pending messages of the originating peer.

```
(defnecd rel-produce-fn (s u :s-fn) :bool
  (rel-produce-help-fn s u (fn-pending-mssgs u)))
```

```
(defnec fn-pending-mssgs (s :s-fn) :lom
  (match s
    (()) '())
    (((& . pst) . rst) (union-set (mget :pending pst) (fn-pending-mssgs rst))))
  )
```

```
(defnec rel-produce-help-fn (s u :s-fn ms :lom) :bool
  (match ms
    (()) nil)
    ((m . rst) (v (^ (produce-fn-pre m s)
                      (= u (produce-fn m s)))
                  (rel-produce-help-fn s u rst)))))
```

```
(defnec produce-fn-pre (m :mssg s :s-fn) :bool
  (b* ((origin (mget :or m))
       (origin-st (mget origin s)))
    (^ (new-fn-mssgp m s)
       origin-st
       (in (mget :tp m) (mget :pubs origin-st)))))
```

```
(defnecd new-fn-mssgp (m :mssg s :s-fn) :bool
  (v (endp s)
    (^ (nin m (mget :seen (cdar s)))
       (nin m (mget :pending (cdar s)))
       (new-fn-mssgp m (cdr s)))))
```

```
(defnecd produce-fn (m :mssg s :s-fn) :s-fn
  :ic (produce-fn-pre m s)
  (mset (mget :or m)
    (add-pending-psfn m (mget (mget :or m) s)) s))
```

```
(defnecd add-pending-psfn (m :mssg pst :ps-fn) :ps-fn
  (if (v (in m (mget :pending pst))
        (in m (mget :seen pst))))
```

```

    pst
    (mset :pending (cons m (mget :pending pst)) pst)))

```

`rel-forward-fn` relates states `s` and `u` where `u` represents the state obtained after a peer in `s` forwards a pending message. Notice that any of the pending messages in `s` are eligible to be forwarded. Notice also that there can be several peers with a given message pending, and the Floodnet can take several possible transitions to states related to the current state by `rel-forward-fn`. To model this in a constructive and deterministic way, we introduce `find-forwarder` as a skolem function which returns the first peer in the state where a given message is pending. It produces a concrete peer `p` in the call to `forward-fn`. Its output contract `(:oc)` specifies that the message forwarding peer it returns (i) is a peer in `s`, (ii) possesses the given message in its `pending` set, and (iii) that message is not new in `s`.

The `forward-fn` transition function simultaneously updated the state of the peer that forwards the message, using `update-forwarder-fn` and updates the `pending` sets of the neighboring subscribers by inserting the forwarded message using `forward-help-fn`. Note that messages are forwarded to all the peers subscribing to the topic of the message in the `:nsubs` map. If the forwarding peer records its own subscriptions in `:nsubs`, it can lead to `rel-forward-fn` being infinitely enabled. We will ensure that a peer does not include itself in this map, by considering *good* Floodnet states later in the paper.

```

(definecd rel-forward-fn (s u :s-fn) :bool
  (rel-forward-help-fn s u (fn-pending-mssgs s)))

(definec rel-forward-help-fn (s u :s-fn ms :lom) :bool
  (match ms
    (() nil)
    ((m . rst)
     (v (^ (in m (fn-pending-mssgs s))
            (== u (forward-fn (find-forwarder s m) m s)))
         (rel-forward-help-fn s u rst)))))

(definec find-forwarder (s :s-fn m :mssg) :peer
  :ic (in m (fn-pending-mssgs s))
  :oc (^ (mget (find-forwarder s m) s)
         (in m (mget :pending (mget (find-forwarder s m) s)))
         (! (new-fn-mssgp m s)))
  (match s
    (((p . &)) p)
    (((p . pst) . rst)
     (if (in m (mget :pending pst)) p (find-forwarder rst m)))))

(definecd forward-fn (p :peer m :mssg s :s-fn) :s-fn
  :ic (^ (mget p s)
         (in m (mget :pending (mget p s)))))
  (b* ((tp (mssg-tp m))
       (pst (mget p s))
       (nsubs (mget :nsubs pst))
       (fwdnbrs (mget tp nsubs))))

```

```

(forward-help-fn (update-forwarder-fn p m s) fwdnbrs m)))

(definec update-forwarder-fn (p :peer m :mssg s :s-fn) :s-fn
  (match s
    (() '())
    (((!p . pst) . rst) (cons '(',p . ,(forwarder-new-pst pst m)) rst))
    ((r . rst) (cons r (update-forwarder-fn p m rst)))))

(definecd forwarder-new-pst (pst :ps-fn m :mssg) :ps-fn
  (mset :seen
    (insert-unique m (mget :seen pst))
    (mset :pending
      (remove-equal m (mget :pending pst))
      pst)))

(definecd forward-help-fn (s :s-fn nbrs :lop m :mssg) :s-fn
  (match s
    (() '())
    (((q . qst) . rst)
      (cons (if (in q nbrs)
        '(',q . ,(add-pending-psfn m qst))
        '(',q . ,qst))
      (forward-help-fn rst nbrs m)))))

```

`rel-subscribe-fn` and `rel-unsubscribe-fn` relate states `s` and `u` where `u` represents the state obtained after a peer in `s` subscribes to or unsubscribes from a set of topics, respectively. They are very similar to their Broadcastnet counterparts and hence we omit their definitions.

`rel-join-fn` and `rel-leave-fn` relate states `s` and `u` where `u` represents the state obtained after a peer joins `s` or leaves `s`, respectively. `fn-join-witness` calculates the peer and its peer-state, if there exists a peer that joins `s`, and is analogous to `bn-join-witness`. `rel-join-fn` requires that the joining peer (i) is not already in `s`, and (ii) does not exist in its own `:nsubs` map. The second condition is necessary to prevent peers from endlessly forwarding messages to themselves. `join-fn` depends on `new-joinee-st-fn` which returns the state for a newly joined peer, and on `set-subs-sfn` which updates the `:nsubs` map for each of the neighboring peers of the joining node. For the sake of brevity, we omit the definitions of these helper functions. The `rel-leave-fn` relation, similar to `rel-leave-bn` requires that there is a leaving peer, as calculated by `(fn-join-witness u s)` and that it already exist in the state. Notice that there is another requirement, that a leaving peer has no pending messages. This condition allows for graceful exit of leaving peers, guaranteeing that no pending messages are lost along with them.

```

(definecd rel-join-fn (s u :s-fn) :bool
  (^ (fn-join-witness s u)
    (b* ((p (car (fn-join-witness s u)))
      (pst (cdr (fn-join-witness s u)))
      (nbrs (topic-lop-map->lop (mget :nsubs pst)))))
    (^ (! (mget p s))
      (nin p nbrs)

```

```

      (== u (join-fn p (mget :pubs pst) (mget :subs pst) nbrs s))))))

(definecd join-fn (p :peer pubs subs :lot nbrs :lop s :s-fn) :s-fn
  :ic (^ (! (mget p s))
    (nin p nbrs))
  (set-subs-sfn nbrs
    subs
    p
    (mset p (new-joiner-st-fn pubs subs nbrs s) s)))

(definecd rel-leave-fn (s u :s-fn) :bool
  (^ (fn-join-witness u s)
    (mget (car (fn-join-witness u s)) s)
    (endp (mget :pending (mget (car (fn-join-witness u s)) s)) s))
    (== u (leave-fn (car (fn-join-witness u s)) s))))

(definecd leave-fn (p :peer s :s-fn) :s-fn
  :ic (mget p s)
  (match s
    (() '())
    (((!p . &) . rst) rst)
    ((r . rst) (cons r (leave-fn p rst)))))

```

## 4.2 Correctness and the Refinement Theorem

We consider simulation refinement [56] as the notion of correctness for Floodnet and show that Floodnet is a simulation refinement of Broadcastnet. The key idea of a simulation refinement is to show that every behavior of the concrete system (Floodnet) is allowed by the abstract system (Broadcastnet). If we prove a WFS refinement then we know that for any infinite computation tree starting from some Floodnet state, we can find a related computation tree in Broadcastnet after applying the refinement map. Another consequence is that we preserve any branching time properties, excluding next time, for example, all properties in  $ACTL^* \setminus X$  [7].

The refinement map needs to map Floodnet states to “related” Broadcastnet states. Why do we require a refinement map? Because states in different levels of abstractions may represent data differently, or some implementation details from the lower abstraction may simply be missing in the higher level specification. For example, `:nsubs` and `:pending` appear only in Floodnet peer states, not in Broadcastnet peer states. Using a refinement map is like putting on glasses that let us “see” lower-level concrete states as their corresponding abstract specification states. The refinement map that we use is `f2b` shown below. It maps Floodnet states to Broadcastnet states where pending messages have not yet been broadcasted. This is called as the commitment approach to refinement, since we are mapping to states consisting of only those messages that have been fully propagated in Floodnet and are thus considered committed.

```

(definecd f2b (s :s-fn) :s-bn

```

```

(f2b-help s (fn-pending-mssgs s)))

(definec f2b-help (s :s-fn ms :lom) :s-bn
  (if (endp s)
      '()
      (cons '(. (caar s) . (f2b-st (cdar s) ms))
            (f2b-help (cdr s) ms))))

(definecd f2b-st (ps :ps-fn ms :lom) :ps-bn
  (ps-bn (mget :pubs ps)
        (mget :subs ps)
        (set-difference-equal (mget :seen ps) ms)))

```

To gain a better understanding of our refinement map, we examine example traces of Floodnet and Broadcastnet in Figure 4.2.1. On the left side, we have a trace of a Floodnet, consisting of 3 green colored nodes, numbered 1, 2 and 3. The node numbered 3 is connected to nodes 1 and 2. We show pending messages on the top left of a node, and seen messages on the bottom right. So, in the second Floodnet state shown, node 1 has a pending message  $m$ , after a **produce-fn** transition. On the right side, we have Broadcastnet states such that for each Floodnet state on the left, we have its refinement map on the right and for each transition on the left, we show a corresponding matching transition on the right.

The transitions on the Floodnet side are as follows : (i) Node 1 produces message  $m$ ; (ii) Node 1 forwards its pending message  $m$  to its connected neighboring peer 3; (iii) Node 1 leaves the network (iv) Node 2 unsubscribes from (**mssg-tp**  $m$ ), which is the message topic (iv) Node 3 unsubscribes from (**mssg-tp**  $m$ ), and finally (v) Node 3 forwards  $m$  to node 2. Notice that **f2b** is a clear refinement map where events like joining and leaving are not masked. Hence, in the corresponding Broadcastnet states, leave and unsubscribe transitions are matched with leave and unsubscribe transitions. However, when the message  $m$  can no longer be forwarded, and is no longer pending, it needs to be matched by a **broadcast**. But notice that on the Broadcastnet side (a) the originating peer (Node 1) is no longer present, which is required for a **broadcast**, due to **broadcast-bn-pre**, and (b) there are no subscribers of  $m$  left in the network! This issue arises from the fact that broadcasting a message in Floodnet is a highly fragmented operation, taking place over several message hops during which peers are free to leave, join, subscribe or unsubscribe. With so many moving parts in the network, it becomes impossible to specify which nodes will receive the broadcasted message in the Broadcastnet under the refinement map at the time the message is produced. To solve this problem, we generalize the Broadcastnet specification by adding another transition relation: **rel-broadcast-partial-bn** which allows us to relate 2 states  $s$  and  $u$  where  $u$  represents the state obtained after broadcasting a message in  $s$ , but only partially. This relation is defined using the **broadcast-partial** transition function, which given a message and a list of peers, sends the message to those peers.

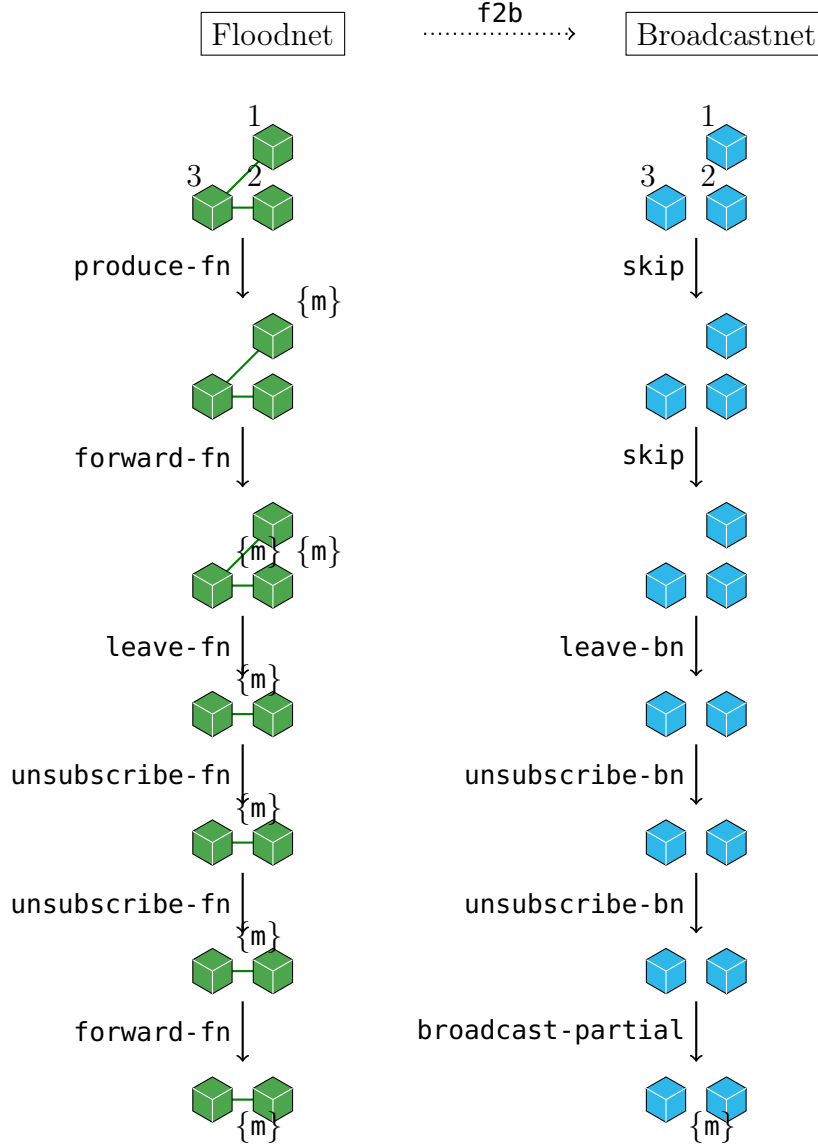


Figure 4.2.1: On the left is an example Floodnet trace. Broadcastnet states on the right are refinement maps of the Floodnet states on the left, and every step taken by the Broadcastnet states matches each step taken by the Floodnet states.

When we consider a static configuration where nodes do not change their subscriptions, no existing nodes are leaving, no new nodes are joining, and the network is connected in each topic, the recipients of the message  $m$  in **broadcast-partial** can be shown to be exactly those in **broadcast** *i.e.*, the subscribers of  $(mssg-tp\ m)$ . This aligns with Dijkstra’s notion of self-stabilizing distributed systems [17], where the system is guaranteed to reach a legitimate configuration regardless of the initial state. In our case, once the system stabilizes (*i.e.*, peer churn and subscription/unsubscription ceases), the **broadcast-partial** step effectively becomes indistinguishable from a **broadcast** step, reinforcing the view that **broadcast-partial** is a generalization that accommodates transient perturbations while preserving desirable behavior in steady state.



```

(definecd rel-broadcast-partial-bn (s u :s-bn) :bool
  (^ (br-mssg-witness s u)
    (new-bn-mssgp (br-mssg-witness s u) s)
    (== u (broadcast-partial (br-mssg-witness s u)
      (brd-receivers-bn (br-mssg-witness s u) u)
      s))))))

(definecd broadcast-partial (m :mssg ps :lop s :s-bn) :s-bn
  :ic (new-bn-mssgp m s)
  (broadcast-partial-help m ps s))

(definecd broadcast-partial-help (m :mssg ps :lop st :s-bn) :s-bn
  (match st
    (() nil)
    (((p . pst) . rst)
      (cons '(', p . ,(if (== p (car ps))
        (mset :seen (insert-unique m (mget :seen pst)) pst)
        pst))
      (broadcast-partial-help m (if (== p (car ps)) (cdr ps) ps) rst))))))
;; broadcast message receivers in a Broadcastnetwork
(definecd brd-receivers-bn (m :mssg s :s-bn) :lop
  (match s
    (() ())
    (((p . pst) . rst) (if (in m (mget :seen pst))
      (cons p (brd-receivers-bn m rst))
      (brd-receivers-bn m rst)))))

```

We define **rel-B**, which holds for related states. **rel-B** is defined over a set of states combining both Floodnet and Broadcastnet states, which we define as **borf**. Notice that **rel-B** depends on Floodnet states satisfying the **good-s-fnp** predicate. This predicate ensures that each Floodnet state in the trace satisfies certain invariants. Given space constraints, we only list the invariant properties that hold true for **good-s-fnp** states at the end of the following listing.

```

(defdata borf (v s-bn s-fn))

(definecd rel-B (x y :borf) :bool
  (v (rel-wf x y)
    (== x y)))

(definecd rel-wf (x y :borf) :bool
  (^ (s-fnp x)
    (s-bnp y)
    (good-s-fnp x)
    (== y (f2b x))))

(definecd rel-> (s u :borf) :bool
  (v (^ (s-fnp s) (s-fnp u) (good-rel-step-fn s u))

```

```

    (^ (s-bnp s) (s-bnp u) (rel-step-bn s u))))

(definec good-rel-step-fn (s u :s-fn) :bool
  (^ (good-s-fnp s)
     (good-s-fnp u)
     (rel-step-fn s u)))

;; A good-s-fnp state satisfies 2 predicates
(definec good-s-fnp (s :s-fn) :bool
  (^ (p!in-nsubs-s-fn s) (ordered-seenp s)))

;; Invariant 1: A peer p does not track its own subscriptions in the
;; :nsubs map. So, it can not forward a message to itself.
(property prop=p!in-nsubs-s-fn (p :peer tp :topic s :s-fn)
  :h (^ (mget p s) (p!in-nsubs-s-fn s))
  :b (nin p (mget tp (mget :nsubs (mget p s)))))

;; Invariant 2: :seen components of Floodnet peers are ordered.
(property prop=ordered-seenp-cdar (s :s-fn)
  :h (^ s (ordered-seenp s))
  :b (orderedp (mget :seen (cdar s))))

(definec orderedp (x :tl) :bool
  (match x
    (() t)
    ((&) t)
    ((a . (b . &)) (^ (<< a b) (orderedp (cdr x))))))

```

Proving the WFS refinement requires proving the following three theorems: (i) WFS1 states that concrete Floodnet states are related to their corresponding Broadcastnet states under the refinement map (by relation **rel-B**), (ii) WFS2 states that the labelling function labels related states equally, and (iii) WFS3 states that given related states **s** and **w**, and given **s** steps to **u** under the transition relation, there exists a state, say **v**, such that **u** is matched by a step from **w** going to **v** such that **w** is related to **v**.

```

;; WFS1
(property b-maps-f2b (s :s-fn)
  :h (good-s-fnp s)
  :b (rel-B s (f2b s)))

;; WFS2. L is the labelling functions of our combined transition system
(definec L (s :borf) :borf
  (match s
    (:s-bn s)
    (:s-fn (f2b s))))

(property wfs2 (s w :borf)
  :h (rel-B s w)

```

```

:b (== (L s) (L w)))

;; WFS3
(defun-sk exists-v-wfs (s u w)
  (exists (v)
    (^ (rel-> w v)
      (rel-B u v))))

(property wfs3 (s w u :borf)
  :h (^ (rel-B s w)
    (rel-> s u))
  :b (exists-v-wfs s u w))

;; Witness generating function for v
(definec exists-v (s u w :borf) :borf
  :ic (^ (rel-B s w)
    (rel-> s u))
  (if (null s)
    (if (null u)
      nil
      (exists-nil-v u))
    (exists-cons-v s u w)))

;; when w is nil
(definec exists-nil-v (u :borf) :borf
  :ic (^ u (rel-> nil u))
  (match u
    (:s-fn (exists-v1 nil u))
    (:s-bn u)))

;; when w is not nil
(definec exists-cons-v (s u w :borf) :borf
  :ic (^ s (rel-B s w) (rel-> s u))
  (cond
    ((^ (s-bnp s) (s-bnp w)) u)
    ((^ (s-fnp s) (s-bnp w)) (exists-v1 s u))
    ((^ (s-fnp s) (s-fnp w)) u)))

;; when s and u are Floodnet states
(definec exists-v1 (s u :s-fn) :s-bn
  :ic (good-s-fnp s)
  (cond
    ((rel-skip-fn s u) (f2b s))
    ((^ (rel-forward-fn s u)
      (!= (f2b s) (f2b u)))
      (broadcast-partial (br-mssg-witness (f2b s) (f2b u))
        (brd-receivers-bn (br-mssg-witness (f2b s) (f2b u))
          (f2b u)))))

```

```

                                (f2b u))
      (f2b s)))
(t (f2b u))))

```

### 4.3 Proof Organization

Given that we define our models using transition functions from states to states, whereas the refinement theorem is expressed in terms of transition relations, proving the monolithic refinement theorem can be a daunting task. In this section, we describe how we approached the mechanization of the refinement proof. The entire codebase can be logically partitioned into four stages:

- **State models and transition functions:** State models are described using `defdata`, and transition functions on the state models are described using `definec` and appear in files `bn-trx.lisp` and `fn-trx.lisp`. Apart from the input state, functions may accept additional arguments. For example `forward-fn` 4.1.2 accepts a peer `p` along with a message `m` that `p` forwards. These functions usually have input contracts to ensure that the extra arguments satisfy certain properties, for example, `p` should be a peer in the state `s`, and it should have `m` in its `pending` set of messages.
- **Properties of functions under the refinement map:** Given that we have defined functions that accept states and output states, we then prove theorems relating Floodnet states to their corresponding Broadcastnet states under the refinement map in file `f2b-commit.lisp`. For example, here is one such theorem:

```

(property prop=forward-fn (p :peer m :mssg s :s-fn)
  :h (^ (mget p s)
    (in m (mget :pending (mget p s)))
    (== (fn-pending-mssgs (forward-fn p m s))
      (fn-pending-mssgs s))))
  :b (== (f2b (forward-fn p m s))
    (f2b s)))

```

Notice that these theorems still depends on variables that have not yet been skolemized, so as to ease the theorem proving process.

- **Transition relations:** We define the transitions relations for both Broadcastnet and Floodnet in `trx-rels.lisp`. Transition relations are boolean functions over two state variables, and hence, we also define witness functions for non-state variables appearing in the transition functions. In our running example, we instantiate `p` with `(find-forwarder s m)` and `m` could be any one of the pending messages in the state.
- **Combined states, transitions relations and correctness theorems:** Finally we prove the WFS theorems and their helper properties in `f2b-sim-ref.lisp`.

During development, some of the previous iterations of our model deviated from the metatheory. For example, in one iteration of the final theorem, the restrictions on the

transition relations, which serve as guards against illegal behaviors, emerged as part of the hypotheses of WFS3. It forced us to understand the nature of good states, and to derive the required hypotheses from the invariants of the good states. In another iteration of our models, the transition relations corresponding to each of the transitions a model can make, was augmented with natural numbers, such that transitions on the Floodnet states were matched by transitions on the Broadcastnet states bearing the same number. Eventually, this arrangement seemed unnecessary because even without the natural numbers, the theorem prover was able to pick the required transition based on theorems proved on them, and because of their definitions being disabled. Hence we would recommend to stick to the metatheory when implementing proofs of refinement, and always write the top level theorems, before embarking on proving lower-level theorems.

## 4.4 Bibliographic Notes

Proof mechanization in context of P2P systems has been explored previously. Azmy et. al. [3] formally verified a safety property of Pastry, a P2P Distributed Hash Table (DHT), in TLA+ [45]. The safety property is that of correct delivery, which states that at any point in time, there is at most one node that answers a lookup request for a key, and this node must be the closest live node to that key. Their proof assumes that nodes never fail, which is a likely event in any P2P system. Zave [88] utilized the Alloy tool [26] to produce counter-examples to show that no published version of Chord is correct w.r.t. the liveness property of the Chord ring-maintenance protocol: that the protocol can eventually repair all disruptions in the ring structure, given ample time and no further disruptions while it is working. Kumar et. al. modeled the Gossipsub [82] P2P protocol in ACL2S, formulated safety properties for its scoring function and showed using counter-examples that for some applications like Ethereum which configure Gossipsub in a particular way, it is possible for Sybil nodes to violate those properties, thereby creating large scale partition or eclipse attacks on the network [41, 43]. To the best of our knowledge, none of the previous work has attempted to prove the correctness of a P2P system by showing it as a refinement of a higher level specification.

There exist several provers to formally check properties of distributed systems, such as Dafny [24], TLA+, Ivy [68] and DistAlgo [78]. They operate by reducing a given specification to a decidable logic formula expressed entirely in First Order Logic. The basic tactic involves forming a conjunction of protocol invariants, invert it, and then using an SMT solver to (possibly) search for a counterexample. The issue in such systems is a lack of expressivity, which does not allow capturing properties over infinite traces. Another issue is that our models can be arbitrary, with nodes leaving and joining and with arbitrary pending messages in transit across a network. And we are reasoning about all possible behaviors of the protocol, which could not be done if we were to be limited to a decidable fragment of logic.

We wrote our models and proved our theorems in ACL2S. The ACL2 Sedan (ACL2S) [9, 18] is an extension of the ACL2 theorem prover [31, 33, 34]. On top of the capabilities of ACL2, ACL2S provides the following: (1) A powerful type system via the **defdata** data definition framework [13] and the **definec** and **property** forms, which support typed definitions and properties. (2) Counterexample generation capability via the **cgen** framework,

which is based on the synergistic integration of theorem proving, type reasoning and testing [10–12]. (3) A powerful termination analysis based on calling-context graphs [63] and ordinals [60–62]. (4) An (optional) Eclipse IDE plugin [9]. (5) The ACL2S systems programming framework (ASPF) [84] which enables the development of tools in Common Lisp that use ACL2, ACL2S and Z3 as a service [37, 85–87].

## Chapter 5

# Formalization and Verification of Meshsub

In this chapter, we describe Gossipsub v1.0 [48], which we refer to as *Meshsub* throughout this dissertation. **Meshsub** extends **Floodsub** with differentiated peering types that reduce bandwidth overhead while preserving efficient message propagation. At a high level, protocols in the libp2p pubsub family organize peers into an *overlay network*, where message dissemination occurs over a logical graph of peer connections rather than the underlying physical topology. In such overlays, peers exchange application messages and control information according to protocol-defined routing rules, enabling scalable, decentralized dissemination without requiring full connectivity among all nodes. Mesh-based pubsub protocols such as Gossipsub combine structured overlay mesh construction with gossip-based metadata exchange to balance low-latency delivery and bandwidth efficiency.

Within this framework, **Meshsub** defines distinct peering types that allow neighboring peers to negotiate the nature of their connections. Broadly, peering relationships fall into two categories:

- **Full-message peerings.** In these connections, peers maintain links over which complete messages are relayed. In protocols such as Gossipsub, full-message links form a bounded-degree mesh overlay for each topic, where peers push messages proactively to their selected neighbors, enabling low-latency dissemination across the network. To prevent excessive flooding, the outbound degree of such connections is bounded, yielding a sparsely connected overlay dedicated to full-message propagation.
- **Metadata-only peerings.** Gossip links constitute a more densely connected overlay through which peers exchange metadata about recently seen messages (*e.g.*, message identifiers) and request missing messages. Because metadata is a small fraction of a full message, this gossip exchange imposes comparatively low bandwidth overhead while improving robustness to loss and enabling peers outside the mesh to discover messages they have missed.

In addition, **Meshsub** supports *fanout* peerings, where a publishing peer maintains full-message links to a set of downstream nodes without itself subscribing to the corresponding

topic. This unidirectional peering allows messages to enter the mesh from outside peers efficiently and subsequently be propagated through the mesh overlay.

The combination of these peering types aligns with the design of gossip-based pubsub protocols: a bounded mesh overlay supports eager push of messages among a core set of peers, while a dense metadata-exchange overlay ensures wider, lazy pull recovery and awareness. This duality enables effective bandwidth management and resilient propagation in large decentralized networks.

Since peers may join or leave the network at any time, such churn can disrupt the structure of mesh overlay networks. To address this, **Meshsub** employs explicit mesh maintenance mechanisms that operate periodically to restore and preserve desired connectivity properties. These mechanisms adjust mesh membership by selectively adding and removing peers, enforcing degree bounds and repairing broken links caused by peer departures or failures. In particular, mesh maintenance is driven by recurring control actions that rebalance the overlay, ensuring that each topic mesh remains well connected and capable of supporting efficient message dissemination despite continual network dynamics.

The remainder of this chapter is organized as follows. Section 5.1 formalizes the **Meshsub** model. Section 5.2 states correctness conditions. Section 5.3 develops the refinement proof, including probabilistic analysis of gossip rounds. Section 5.4 concludes with a discussion of the main theorems.

## 5.1 Formal Description of the Meshsub Model

A **Meshsub** (MS) state is a partial map from peers to their states, where each peer is identified by a distinct natural number. As in **Floodsub**, if a peer rejoins the network, it is assigned a fresh identifier. The state for a peer  $i \in \text{dom}(S_{\text{Meshsub}})$  is a tuple

$$\langle \text{Act}, \text{Subs}, \text{Pubs}, \text{Nbrs}, \text{Mesh}, \text{Fanout}, \text{Pdg}, \text{Ctrl}, \text{WReqs}, \text{RSeen}, \text{Seen} \rangle_i.$$

The components **Act**, **Subs**, **Pubs**, **Nbrs**, **Pdg**, and **Seen** have the same interpretation as in **Floodsub**. In addition, **Mesh** maps each subscribed topic to the set of peers forming the mesh overlay for that topic, while **Fanout** records topic-specific forwarding peers used by publishers that are not subscribed to the topic. The queue **Ctrl** stores control messages exchanged for mesh maintenance and gossip, **WReqs** is a set of message hashes corresponding to outstanding **IWANT** requests, and **RSeen** records recently seen messages used to support gossip emission.

A **Meshsub** is parameterized by the following variables:

- $D$ : the target degree of each node in the mesh overlay,
- $D_{lo}$ : the low-degree threshold below which mesh maintenance adds peers ( $D_{lo} \leq D$ ),
- $D_{hi}$ : the high-degree threshold above which mesh maintenance prunes peers ( $D_{hi} \geq D$ ),
- $D_{lazy}$ : the number of non-mesh neighbors selected for gossip in each heartbeat round, and
- $W$ : the length of the recently seen window.



| Rule                            | Definition                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MProduce</b> ( $s, m$ )      | $activep(m.\text{Origin}, s) \wedge m.\text{Topic} \in s(m.\text{Origin}).\text{Pubs} \wedge$ $\bigwedge_{i \in \text{dom}(s)} \neg hasMsgp(i, m, s) \rightarrow s(m.\text{Origin}).\text{Pdg.enqueue}(m)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>MForward</b> ( $s, i, D$ )   | $activep(i, s) \wedge \neg s(i).\text{Pdg.empty} \rightarrow \text{let } m = s(i).\text{Pdg.front} \text{ in}$ $s(i).\text{RSeen}[0]:\cup=\{m\} \wedge s(i).\text{Pdg.dequeue} \wedge s(i).\text{Seen}:\cup=\{m\} \wedge$ $\text{if } m.\text{Topic} \in s(i).\text{Subs} \text{ then}$ $\langle \forall k \in s(i).\text{Mesh}(m.\text{Topic}) : activep(k, s) \wedge \neg hasMsgp(k, m, s) : s(k).\text{Pdg.enqueue}(m) \rangle$ $\text{else}$ $(\text{if } s(i).\text{Fanout} = \emptyset \text{ then}$ $s(i).\text{Fanout}(m.\text{Topic}):\cup= \text{select}(D, s(i).\text{Nbrs}(m.\text{Topic}))\rangle$ $\langle \forall k \in s(i).\text{Fanout}(m.\text{Topic}) : activep(k, s) \wedge \neg hasMsgp(k, m, s) : s(k).\text{Pdg}:\cup=\{m\} \rangle$ |
| <b>MPldReq</b> ( $s, j, i, m$ ) | $activep(j, s) \wedge \neg s(j).\text{Ctrl.empty} \wedge \langle \text{IHAVE}, \text{hash}(m), i \rangle =$ $s(j).\text{Ctrl.front} \rightarrow$ $s(j).\text{Ctrl.dequeue} \wedge$ $\text{if } \text{hash}(m) \notin \{\text{hash}(r)   r \in s(j).\text{Seen}\} \wedge activep(i, s) \text{ then}$ $s(j).\text{WReqs}:\cup=\{\text{hash}(m)\} \wedge s(i).\text{Ctrl.enqueue}(\langle \text{IWANT}, \text{hash}(m), j \rangle)$                                                                                                                                                                                                                                                                                                                             |
| <b>MPldSnd</b> ( $s, i, j, m$ ) | $activep(i, s) \wedge \neg s(i).\text{Ctrl.empty} \wedge \langle \text{IWANT}, \text{hash}(m), j \rangle = s(i).\text{Ctrl.front} \rightarrow$ $s(i).\text{Ctrl.dequeue} \wedge (\text{if } activep(j, s) \wedge \neg hasMsgp(j, m, s) \text{ then}$ $s(j).\text{Pdg.enqueue}(m) \wedge s(j).\text{WReqs}:\setminus=\{\text{hash}(m)\})$                                                                                                                                                                                                                                                                                                                                                                                                                     |

Table 5.1:  $\rightarrow_{\text{Meshsub}}$  definition, Part 1

Typical configurations satisfy  $D_{lo} < D < D_{hi}$ , ensuring hysteresis in mesh maintenance to avoid oscillation.

We model the behavior of **Meshsub** as a labeled transition system

$\mathcal{M}_{\text{Meshsub}} = \langle S_{\text{Meshsub}}, \rightarrow_{\text{Meshsub}}, L_{\text{Meshsub}} \rangle$ , where  $S_{\text{Meshsub}}$  is the set of **Meshsub** states,  $\rightarrow_{\text{Meshsub}}$  is the transition relation defined in Tables 5.1 to 5.3, and  $L_{\text{Meshsub}}$  is the identity labeling function.

We begin by defining the core message dissemination and gossip transitions of **Meshsub**. These transitions capture the fundamental behaviors through which messages are produced, forwarded, advertised, and requested among active peers. In particular, they formalize how peers enqueue newly produced messages, forward messages to mesh or fanout neighbors, exchange metadata via gossip, and respond to explicit requests for missing messages. Together, these rules define the end-to-end flow of messages through the overlay network, abstracting away mesh maintenance and membership dynamics, which are handled separately.  $\text{select}(k, S)$  randomly chooses  $\min(k, |S|)$  elements from a set  $S$ .

We next define transitions governing peer membership, subscription management, and mesh maintenance. These rules describe how peers join and leave the network, subscribe to or unsubscribe from topics, and adjust mesh connectivity through explicit control messages. In

contrast to the previous transitions, which focus on message flow, these rules are responsible for constructing and repairing the mesh overlay in the presence of churn, enforcing degree constraints, and ensuring that topic-specific meshes remain sufficiently connected over time.

Finally, we introduce a collection of auxiliary transitions that are invoked periodically as part of the protocol's maintenance routines. These transitions formalize gossip emission, fanout adjustment, and mesh rebalancing, and are typically triggered by heartbeat events. Although auxiliary in nature, these mechanisms play a critical role in preserving the structural invariants of the overlay network and in supporting robustness against message loss, peer failures, and dynamic changes in connectivity.

We call **MPldReq**, **MPldSnd**, **MPrune** and **MGraft** as *control transitions* as they are enabled based on the front message in the **Ctrl** queue.

**Definition 18 (Control transition relation)**

The control transition relation  $\rightarrow_{\text{Ctrl}} \subseteq S_{\text{Meshsub}} \times S_{\text{Meshsub}}$  is defined by:

$$\begin{aligned} w \rightarrow_{\text{Ctrl}} v = & \exists i, j, m : \neg w(i).\text{Ctrl.empty} \wedge \\ & ((w(i).\text{Ctrl.front} = \text{IHAVE} \wedge v = \text{MPldReq}(w, i, j, m)) \\ & \vee (w(i).\text{Ctrl.front} = \text{IWANT} \wedge v = \text{MPldSnd}(w, i, j, m)) \\ & \vee (w(i).\text{Ctrl.front} = \text{PRUNE} \wedge v = \text{MPrune}(w, i, j)) \\ & \vee (w(i).\text{Ctrl.front} = \text{GRAFT} \wedge v = \text{MGraft}(w, i, j))). \end{aligned}$$

---

**Algorithm 2** DRAIN-CTRL-Q

---

**Require:** Meshsub state  $w$ , finite peer set  $P \subseteq \text{dom}(w)$

**Ensure:** State  $w'$  such that  $\forall i \in P. w'(i).\text{Ctrl.empty}$

```

1: while  $\exists i \in P$  such that  $\neg w(i).\text{Ctrl.empty}$  do
2:   Choose  $i \in P$  such that  $\neg w(i).\text{Ctrl.empty}$ 
3:    $c \leftarrow w(i).\text{Ctrl.front}$ 
4:   Execute the corresponding Ctrl transition producing  $w'$ 
5:    $w \leftarrow w'$ 
6: end while
   return  $w$ 

```

---

**Theorem 5.1.1** (Termination of DRAIN-CTRL-Q). *For any Meshsub state  $w$  and finite peer set  $P$ , the algorithm DRAIN-CTRL-Q terminates.*

*Proof.* We define the following measure for DRAIN-CTRL-Q.

|                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MGraft</b> ( $s, j, i$ )                          | $activep(j, s) \wedge \neg s(j).Ctrl.empty \wedge \langle GRAFT, t, i \rangle = s(j).Ctrl.front \rightarrow$<br>$s(j).Ctrl.dequeue \wedge$<br>$\text{if } t \in s(j).Subs \text{ then } s(j).Mesh(t) : \cup = \{i\}$<br>$\text{else if } activep(i, s) \text{ then } s(i).Ctrl.enqueue(\langle PRUNE, t, j \rangle)$                                                                                                                                                                                                                                                                  |
| <b>MPrune</b> ( $s, j, i$ )                          | $activep(j, s) \wedge \neg s(j).Ctrl.empty \wedge \langle PRUNE, t, i \rangle = s(j).Ctrl.front \rightarrow$<br>$s(j).Ctrl.dequeue \wedge s(j).Mesh(t) : \setminus = \{i\}$                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>MSubscribe</b> ( $s, i, T, D$ )                   | $activep(i, s) \rightarrow s(i).Subs : \cup = T \wedge \text{dom}(s(i).Fanout) : \setminus = T \wedge$<br>$\langle \forall t \in T :: \langle \forall k \in \text{rng}(s(i).Nbrs) :: s(k).Nbrs(t) : \cup = \{i\} \rangle \rangle \wedge$<br>$\langle \forall t \in T :: \text{let } A =$<br>$\text{select}(D -  s(i).Fanout(t) , s(i).Nbrs(t) \setminus s(i).Fanout(t)) \text{ in}$<br>$s(i).Mesh(t) : \cup = s(i).Fanout(t) \uplus A;$<br>$\langle \forall a \in A :: s(a).Ctrl : \cup = \langle GRAFT, t, i \rangle \rangle \rangle$                                                |
| <b>MUnsubscribe</b> ( $s, i, T$ )                    | $activep(i, s) \rightarrow s(i).Subs : \setminus = T \wedge \text{dom}(s(i).Mesh) : \setminus = T \wedge$<br>$\langle \forall t \in T :: \langle \forall k \in \text{rng}(s(i).Nbrs) :: s(k).Nbrs(t) : \setminus = \{i\} \rangle \rangle \wedge$<br>$\langle \forall t \in T :: \langle \forall k \in s(i).Mesh(t) :: s(k).Ctrl : \cup = \{ \langle PRUNE, t, i \rangle \} \rangle \rangle$                                                                                                                                                                                           |
| <b>MJoin</b> ( $s, i, Subs,$<br>$Pubs, Nbrs, Mesh$ ) | $i \notin \text{dom}(s) \wedge i \notin \text{rng}(Nbrs) \wedge \langle \forall t \in Subs \cup Pubs ::$<br>$\langle \exists j :: j \in Nbrs(t) \rangle \rangle \rightarrow$<br>$\langle \forall t \in Subs :: \langle \forall k \in Nbrs(t) :: s(k).Nbrs(t) : \cup = \{i\} \rangle \rangle \wedge$<br>$\langle \forall t \in \text{dom}(s(i).Mesh) :: \langle \forall k \in s(i).Mesh(t) ::$<br>$s(k).Ctrl.enqueue(\langle GRAFT, t, i \rangle) \rangle \rangle \wedge$<br>$s(i) := \langle Subs, Pubs, Nbrs, Mesh, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle_i$ |
| <b>MLeave</b> ( $s, i$ )                             | $activep(i, s) \rightarrow \langle \forall t \in s(i).Subs :: \langle \forall k \in s(i).Nbrs(t) ::$<br>$s(k).Nbrs(t) : \setminus = \{i\} \wedge s(k).Mesh(t) : \setminus = \{i\} \wedge s(k).Fanout(t) : \setminus =$<br>$\{i\} \rangle \rangle \wedge$<br>$s(i).Act := \text{false}$                                                                                                                                                                                                                                                                                                |
| <b>MHBM</b> ( $s, i, D, W, c$ )                      | $activep(i, s) \rightarrow \text{MGossip}(s, i, D, W) \wedge \text{MFanoutMnt}(s, i, D, c) \wedge$<br>$\text{MMeshMnt}(s, i, D)$                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

Table 5.2:  $\rightarrow_{\text{Meshsub}}$  definition, Part 2

| Auxiliary Rule                                | Definition                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MGossip</b> ( $s, i, D, W$ )               | $\langle \forall m \in \bigcup_{k=0,1,\dots,W-1} s(i).R\text{Seen}[k],$ $\langle \forall j \in \text{select}(D_{\text{lazy}}, s(i).\text{Nbrs}(m.\text{Topic}) \setminus$ $(s(i).\text{Mesh}(m.\text{Topic}) \cup s(i).\text{Fanout}(m.\text{Topic}))) ::$ $\text{if } \text{activep}(j, s) \text{ then } s(j).\text{Ctrl}.\text{enqueue}(\langle \text{IHAVE}, \text{hash}(m), i \rangle) \rangle \ ;$ $s(i).R\text{Seen} := \emptyset :: \text{init}(s(i).R\text{Seen})$                                                                                                                                                                                                                                                   |
| <b>MFanoutMnt</b> ( $s, i, D, c$ )            | $\langle \forall t \in \text{dom}(s(i).\text{Fanout}) :: \text{if } c \text{ then } \text{dom}(s(i).\text{Fanout}) := \{t\}$ $\text{else if }  s(i).\text{Fanout}(t)  < D \text{ then let } k = D -  s(i).\text{Fanout}(t)  \text{ in}$ $s(i).\text{Fanout}(t) := \text{select}(k, s(i).\text{Nbrs}(t) \setminus s(i).\text{Fanout}(t)) \rangle$                                                                                                                                                                                                                                                                                                                                                                             |
| <b>MMeshMnt</b> ( $s, i, D, D_{hi}, D_{lo}$ ) | $\langle \forall t \in \text{dom}(s(i).\text{Mesh}) :: \text{if }  s(i).\text{Mesh}(t)  < D_{lo} \text{ then}$ $\text{let } R = \text{select}(D -  s(i).\text{Mesh}(t) , s(i).\text{Nbrs}(t) \setminus s(i).\text{Mesh}(t)) \text{ in}$ $s(i).\text{Mesh}(t) := R \ ;$ $\langle \forall k \in R : \text{activep}(k, s) : s(k).\text{Ctrl}.\text{enqueue}(\langle \text{GRAFT}, t, i \rangle) \rangle$ $\text{else if }  s(i).\text{Mesh}(t)  > D_{hi} \text{ then}$ $\text{let } R = \text{select}( s(i).\text{Mesh}(t)  - D, s(i).\text{Mesh}(t)) \text{ in}$ $s(i).\text{Mesh}(t) := R \ ;$ $\langle \forall k \in R : \text{activep}(k, s) : s(k).\text{Ctrl}.\text{enqueue}(\langle \text{PRUNE}, t, i \rangle) \rangle$ |

Table 5.3: Auxiliary Transition Definitions

**Definition 19 (DRAIN-CTRL-Q-MEASURE)**

For a Meshsub state  $w$ , let  $C_i = w(i).\text{Ctrl}$ , let  $\text{count}_m(C)$  denote the number of times  $m$  occurs in control queue  $C$ , and let  $-$  denote a wildcard. Define

$$\begin{aligned} \text{DRAIN-CTRL-Q-MEASURE}(w) = \sum_{\text{activep}(i,w)} & \left( 2 \cdot \text{count}_{(\text{GRAFT}, -, -)}(C_i) + \text{count}_{(\text{PRUNE}, -, -)}(C_i) \right. \\ & \left. + 2 \cdot \text{count}_{(\text{IHAVE}, -, -)}(C_i) + \text{count}_{(\text{IWANT}, -, -)}(C_i) \right). \end{aligned}$$

Observe that:

1.  $\text{DRAIN-CTRL-Q-MEASURE}(w) \in \mathbb{N}$ .
2.  $\text{DRAIN-CTRL-Q-MEASURE}(w) \geq 0$ .
3.  $\text{DRAIN-CTRL-Q-MEASURE}(w) = 0$  iff  $\forall i \in P. w(i).\text{Ctrl}.\text{empty}$ .

Each iteration of the loop in DRAIN-CTRL-Q executes exactly one **Ctrl** transition. We verify that any newly enqueued message has strictly smaller weight than the dequeued message, by case analysis on the front control message:

- **IHAVE (weight 2, MPldReq)**. Dequeues an **IHAVE** message (weight 2). May enqueue one **IWANT** message (weight 1) into another peer's queue. Net change:  $-2+1 = -1 < 0$ . ✓
- **IWANT (weight 1, MPldSnd)**. Dequeues an **IWANT** message (weight 1). Enqueues nothing into any **Ctrl** queue (only enqueues  $m$  into **Pdg**, which is not counted by the measure). Net change:  $-1 < 0$ . ✓
- **GRAFT (weight 2, MGraft)**. Dequeues a **GRAFT** message (weight 2). May enqueue one **PRUNE** message (weight 1) into another peer's queue. Net change:  $-2+1 = -1 < 0$ . ✓
- **PRUNE (weight 1, MPrune)**. Dequeues a **PRUNE** message (weight 1). Enqueues nothing. Net change:  $-1 < 0$ . ✓

Hence, for any successor state after an iteration,  $w'$ , we have

$$\text{DRAIN-CTRL-Q-MEASURE}(w') < \text{DRAIN-CTRL-Q-MEASURE}(w),$$

Since  $\text{DRAIN-CTRL-Q-MEASURE}(w)$  is a natural number and strictly decreases on each iteration, DRAIN-CTRL-Q must terminate.  $\square$

## 5.2 Correctness Conditions for Meshsub

In **Meshsub**, message dissemination relies on a combination of direct forwarding through mesh and fanout connections, and indirect delivery through gossip and request mechanisms. Unlike **Floodsub**, where messages are forwarded eagerly to all neighbors, **Meshsub** employs selective forwarding to reduce bandwidth while maintaining delivery guarantees. The correctness of this approach relies on the following assumptions.

**(C1) Weak fairness of heartbeat transitions.** We assume weak fairness of heartbeat maintenance transitions: for every node  $i$ , if the heartbeat transition  $\text{MHBM}(s, i, D, W, c)$  is enabled infinitely often along an execution, then it is taken infinitely often. This assumption ensures that gossip rounds are not indefinitely starved by other transitions, and is necessary to guarantee eventual progress of **IHAVE/IWANT** dissemination.

**(C2) Non-empty recently seen window.** We require the recently seen window size to satisfy  $W \geq 1$ . This guarantees that every forwarded message remains eligible for gossip for at least one heartbeat round, preventing messages from being lost before lazy peers can be notified.

## 5.3 Correctness of Meshsub

Having defined the operational model of **Meshsub**, we now establish its correctness by relating it to the simpler specification **Floodsub**. Our goal is to show that **Meshsub** refines **Floodsub**: every behavior of the concrete protocol corresponds to a behavior of the abstract flooding specification under an appropriate abstraction map.

A natural first question is whether the two systems are bisimilar. We show that this is not the case. The failure of bisimulation is not incidental, but stems from a structural difference in how the two protocols handle message forwarding.

Bisimulation does not hold between **Meshsub** and **Floodsub** due to a fundamental mismatch in their control and forwarding semantics. Consider a state in which a node has received two messages  $m_1$  and  $m_2$  that have not yet been forwarded. In **Floodsub**, the node may nondeterministically forward either message in the next step. In **Meshsub**, however, forwarding is preceded by **IHAVE/IWANT** control exchanges, and the order of forwarding is determined by the scheduling of control messages in the node's **Ctrl** queue. Suppose the **IWANT** corresponding to  $m_2$  is processed before the **IWANT** for  $m_1$ . Then the next step in **Meshsub** may fully forward  $m_2$ , whereas **Floodsub** may take a **FForward** step from the related state that forwards  $m_1$ . Because this **Floodsub** transition cannot be matched by a corresponding **Meshsub** transition from the related state, no bisimulation relation can exist. The mismatch arises from the control-plane ordering constraints imposed by **Meshsub**.

Although bisimulation fails, the observable behaviors of **Meshsub** are still consistent with **Floodsub** up to skipping. **Meshsub** refines **Floodsub** at the level of eventual dissemination, but not at the level of individual transition granularity. One might attempt to use stuttering simulation with a well-founded rank, requiring the rank to decrease at every abstractly silent step of **Meshsub**. This fails because gossip-based neighbor selection via select is probabilistic:

in any given heartbeat round, a particular non-mesh neighbor may or may not receive an **IHAVE** message, so there is no deterministic measure that is guaranteed to decrease at each concrete step. Progress towards full dissemination is only assured with high probability over sufficiently many rounds. Skipping refinement, lifted probabilistically, captures this precisely: the concrete system is permitted to take arbitrarily many internal steps before matching an abstract step, with the guarantee that the matching occurs with probability at least  $1 - \varepsilon$ , rather than deterministically at every step. This motivates the use of Skipping Simulation Refinement [27–29] lifted to a probabilistic setting.

We will establish this via a Reduced Well-Founded Skipping Simulation (RWFSK) Refinement. Proving RWFSK refinement ensures that for any infinite computation tree starting from some **Meshsub** state, there exists a related computation tree in **Floodsub** after applying the refinement map. As a consequence, all branching-time properties excluding next-time, for example those in  $ACTL^* \setminus X$  [7], are preserved.

### 5.3.1 Choosing a refinement map

Let  $C$  be a concrete **Meshsub** state and let  $A = \mathbf{m2f}(C)$  be the corresponding abstract **Floodsub** state. For each node  $i$ , the abstraction preserves the **Subs**, **Pubs**, **Nbrs**, and **Seen** components. The key difference lies in the treatment of pending queues: in the concrete system, messages may exist in intermediate states of dissemination from a node to its neighbors, whereas the abstract model exposes only messages that are fully committed to delivery. Accordingly, partially forwarded messages must be suppressed by the abstraction.

The refinement map  $\mathbf{m2f}$  is therefore defined as a commitment refinement map: it projects a concrete state to the abstract state representing only those effects that are irrevocably committed in the concrete execution. Transient dissemination artifacts, such as partially forwarded messages that may still be dropped or redirected, are intentionally omitted. This choice ensures that the abstract state reflects stable, externally observable behavior rather than intermediate protocol mechanisms.

We will need the following predicates to define our refinement mapping.

#### Definition 20 (Fully-forwarded predicate)

For a message  $m$  forwarded from node  $i$  in state  $s$ , we define:

$$\begin{aligned} \text{fullyFwd}(i, m, s) &:= m \in s(i).\text{Seen} \Rightarrow \\ &\langle \forall j \in s(i).\text{Nbrs}(m.\text{Topic}) :: \text{activep}(j, s) \Rightarrow \text{hasMsgp}(j, m, s) \rangle \end{aligned}$$

Intuitively,  $m$  is *fully forwarded* when every neighbor that is supposed to receive  $m$  has already received it (in **Pdg** or in **Seen**).

### Definition 21 (Ctrl-depth and ctrl-order)

For a reachable **Meshsub** state  $s$ , active peer  $i$ , and message  $m \in s(i).\text{Seen}$  with  $\neg \text{fullyFwd}(i, m, s)$ , define the *ctrl-depth* of  $m$  at  $i$  in  $s$  as

$$\text{ctrlDepth}(i, m, s) = \max_{j \in s(i).\text{Nbrs}(\text{m.Topic}), \text{activep}(j, s), \neg \text{hasMsgp}(j, m, s)} \text{qpos}(\langle \text{IWANT}, \text{hash}(m), j \rangle, s),$$

where  $\text{qpos}(\langle \text{IWANT}, \text{hash}(m), j \rangle, s)$  is the zero-indexed position of that **IWANT** in its **Ctrl** queue, i.e., the number of **Ctrl** messages ahead of it. If the **IWANT** for  $j$  has not yet been generated in state  $s$  (because peer  $j$  has not yet processed the corresponding **IHAVE**), we set  $\text{qpos}(\langle \text{IWANT}, \text{hash}(m), j \rangle, s) = \infty$ . We set  $\text{ctrlDepth}(i, m, s) = 0$  if no active neighbor still needs  $m$ , i.e.,  $\text{fullyFwd}(i, m, s)$  holds.

The use of  $\infty$  for ungenerated **IWANT**s ensures that messages requiring further gossip rounds before their **IWANT**s are generated sort *after* messages whose **IWANT**s are already queued.

The *ctrl-order*  $\text{ctrlOrd}(i, s)$  is the total order on  $\{m \in s(i).\text{Seen} \mid \neg \text{fullyFwd}(i, m, s)\}$  defined by  $\text{ctrlDepth}(i, \_, s)$  ascending (with  $\infty > k$  for all  $k \in \mathbb{N}$ ). Within a single FIFO **Ctrl** queue all positions are distinct, so no ties arise between messages whose **IWANT**s reside in the same queue. When two messages have **IWANT**s in *different* **Ctrl** queues at the same finite depth, ties are broken by the peer id of the originating peer, ascending. This tiebreaker is not arbitrary: under the refinement constraint  $sBw$  we have  $s = \text{m2f}(w)$  and  $s(i).\text{Pdg} = \text{APdg}(w, i)$ , so the ordering of the abstract Floodsub pending queue  $s(i).\text{Pdg}$  is determined by the arrival order of messages from their source peers, which is itself peer-id ordered. The peer-id tiebreaker in  $\text{ctrlOrd}$  therefore precisely mirrors the queue ordering that  $sBw$  forces on the abstract side.

### Definition 22 (Abstract pending queues)

We define the abstract pending queue for node  $i$  in **Meshsub** state  $s$  as:

$$\begin{aligned} \text{APdg}(s, i) := & \text{ctrlOrd}(i, s)(\{m \in s(i).\text{Seen} \mid \neg \text{fullyFwd}(i, m, s)\}) \\ & \uplus s(i).\text{Pdg}.\text{filter}(\langle \lambda m :: \bigwedge_{j \in \text{rng}(s(i).\text{Nbrs})} \text{fullyFwd}(j, m, s) \rangle) \end{aligned}$$

The design of **APdg** is motivated by the refinement constraint. When  $sBw$  holds,  $s = \text{m2f}(w)$  and  $s(i).\text{Pdg} = \text{APdg}(w, i)$ : the abstract Floodsub pending queue must equal the concrete abstract pending queue. Floodsub processes  $s(i).\text{Pdg}$  in FIFO order, where messages arrive from their source peers in peer-id order. The first component of  $\text{APdg}(s, i)$  therefore uses  $\text{ctrlOrd}(i, s)$ , whose tiebreaker is precisely the source peer-id order in order to ensure that the front of  $\text{APdg}(w, i)$  always matches the front of  $s(i).\text{Pdg}$ . The second component retains committed pending messages, i.e., whose source peers have already fully forwarded.



### Definition 23 (Abstract Seen set)

We define the abstract **Seen** set for node  $i$  in **Meshsub** state  $s$  as:

$$\mathbf{ASeen}(s, i) := s(i).\mathbf{Seen}.\text{filter}(\langle \lambda m :: \text{fullyFwd}(i, m, s) \rangle)$$

Thus, any message  $m$  that has not yet been forwarded to *all* neighbors is removed from the seen set.

All other fields not present in **Floodsub** (mesh maintenance, gossip queues, fanout sets, control queues, **WReqs**, and **RSeen**) are ignored in the abstract state.

### Definition 24 (Refinement map from **Meshsub** to **Floodsub** states)

$$\mathbf{m2f}(s) = \langle \lambda i \in \text{dom}(s) :: \langle s(i).\mathbf{Act}, s(i).\mathbf{Subs}, s(i).\mathbf{Pubs}, s(i).\mathbf{Nbrs}, \mathbf{APdg}(s, i), \mathbf{ASeen}(s, i) \rangle_i \rangle$$

### Definition 25 (Message frontier)

The *message frontier* of a **Meshsub** state  $s$ , denoted  $\mathcal{F}(s)$ , is the set of messages that are still actively propagating through the network:

$$\mathcal{F}(s) = \mathcal{P}(s) \cup \bigcup_{\substack{i \in \text{dom}(s) \\ \text{activep}(i, s)}} \bigcup_{k=0}^{W-1} s(i).\mathbf{RSeen}[k]$$

where  $\mathcal{P}(s) = \bigcup_{j \in \text{dom}(s)} s(j).\mathbf{Pdg}$  is the set of pending messages.

Messages in  $\mathcal{F}(s)$  have not yet completed dissemination: they may still be forwarded to mesh neighbors or advertised via **IHAVE** messages to lazy peers. Once a message leaves the frontier—that is, it has been forwarded by all holders and has aged out of all recently seen windows—it is considered fully committed and becomes visible in the abstract state under the refinement map  $\mathbf{m2f}$ .

## 5.3.2 High-Probability Gossip Coverage Bound

We analyze the number of gossip rounds required for a **Meshsub** node to deliver **IHAVE** messages to all non-mesh neighbors. Let  $N$  denote the total number of neighbors,  $D$  the number of mesh neighbors, and  $D_{\text{lazy}}$  the number of non-mesh neighbors selected uniformly at random in each gossip round. Define  $M = N - D$  as the number of non-mesh neighbors.

**Lemma 5.3.0.1** (State-local high-probability exposure bound). *Let  $w$  be any reachable **Meshsub** state. Let  $M$  denote the number of active non-mesh neighbors of peer  $i$  that have not yet received an **IHAVE** message for  $m$  in  $w$ . Let  $T_w$  denote the number of subsequent gossip rounds until all such neighbors have received an **IHAVE**.*

Under uniform random peer selection via select and fair heartbeat scheduling, for every  $0 < \varepsilon < 1$ ,

$$\Pr_w[T_w \leq t(\varepsilon)] \geq 1 - \varepsilon,$$

where

$$t(\varepsilon) = \frac{M}{D_{\text{lazy}}} (\ln M + \ln(1/\varepsilon)).$$

*Proof.* We derive the tail bound using a union bound over uncovered neighbors.

**Probability a single neighbor is uncovered.** Fix a neighbor  $j$ . In a single round, the probability that  $j$  is *not* selected is

$$\Pr[j \text{ not selected in one round}] = \frac{\binom{M-1}{D_{\text{lazy}}}}{\binom{M}{D_{\text{lazy}}}} = \frac{M - D_{\text{lazy}}}{M} = 1 - \frac{D_{\text{lazy}}}{M}.$$

After  $t$  independent rounds, the probability that  $j$  has never been selected is

$$\Pr[j \text{ uncovered after } t \text{ rounds}] = \left(1 - \frac{D_{\text{lazy}}}{M}\right)^t \leq e^{-tD_{\text{lazy}}/M},$$

using the inequality  $1 - x \leq e^{-x}$  for  $x \geq 0$ .

**Union bound over all neighbors.** By the union bound, the probability that *any* neighbor remains uncovered after  $t$  rounds is

$$\Pr[T > t] \leq \sum_{j=1}^M \Pr[j \text{ uncovered after } t \text{ rounds}] \leq M \cdot e^{-tD_{\text{lazy}}/M}.$$

**Solving for  $t(\varepsilon)$ .** We require  $\Pr[T > t] \leq \varepsilon$ . Setting

$$M \cdot e^{-tD_{\text{lazy}}/M} \leq \varepsilon$$

and solving for  $t$ :

$$\begin{aligned} e^{-tD_{\text{lazy}}/M} &\leq \frac{\varepsilon}{M} \\ -\frac{tD_{\text{lazy}}}{M} &\leq \ln\left(\frac{\varepsilon}{M}\right) = \ln \varepsilon - \ln M \\ t &\geq \frac{M}{D_{\text{lazy}}} (\ln M - \ln \varepsilon) = \frac{M}{D_{\text{lazy}}} (\ln M + \ln(1/\varepsilon)). \end{aligned}$$

Therefore, setting

$$t(\varepsilon) = \frac{M}{D_{\text{lazy}}} (\ln M + \ln(1/\varepsilon))$$

guarantees  $\Pr[T \leq t(\varepsilon)] \geq 1 - \varepsilon$ .

**Numerical example.** For  $M = 50$  non-mesh neighbors,  $D_{\text{lazy}} = 6$ , and  $\varepsilon = 10^{-4}$ :

$$t(10^{-4}) = \frac{50}{6}(\ln 50 + \ln 10^4) \approx 8.33 \cdot (3.91 + 9.21) \approx 109 \text{ rounds.}$$

With heartbeat intervals of 1 second, this corresponds to approximately 2 minutes to achieve 99.99% confidence that all non-mesh neighbors have received gossip.  $\square$

**Discussion.** The preceding lemma provides a state-local, high-probability bound on the number of gossip rounds required to expose a message to all relevant neighbors. Together with the deterministic termination of control-queue draining, this yields a bound on the finite stuttering required to match each abstract **Floodsub** forwarding step. This bound forms the probabilistic core of the skipping refinement from **Meshsub** to **Floodsub**.

### 5.3.3 Probabilistic Skipping Refinement

To handle the inherent randomness in gossip-based neighbor selection, we introduce probabilistic lifting of refinement.

#### Definition 26 ( $\varepsilon$ -probabilistic skipping refinement)

Let  $M_C$  be a probabilistic transition system and  $M_A$  a deterministic transition system. For  $\varepsilon \in [0, 1]$ , we write

$$\begin{aligned} M_C &\lesssim_r^\varepsilon M_A \\ &\text{iff for every reachable state } s \in S_C, \\ &\Pr_s[\text{the RWFSK obligations for } \lesssim_r \text{ are satisfied}] \geq 1 - \varepsilon. \end{aligned}$$

### 5.3.4 The Refinement Proof

We prove that **Meshsub** is a probabilistic skipping refinement of **Floodsub**.  $\text{m2f}$  is our refinement map from **Meshsub** states to **Floodsub** states. Let  $S_{\text{Meshsub}} = \{s \mid s \in S_{MS} \wedge \text{mp}(s)\}$ . To show that **Meshsub** is a skipping refinement of **Floodsub**, we define a relation  $B$  such that  $\langle \forall s \in S_{\text{Meshsub}} :: s B \text{m2f}(s) \rangle$  and show that  $B$  is a Reduced Well-Founded Skipping (RWFSK) relation on the LTS  $\langle S, \rightarrow, \mathcal{L} \rangle$ , where  $S = S_{\text{Meshsub}} \uplus S_{FS}$ ,  $\rightarrow = \rightarrow_{\text{Meshsub}} \uplus \rightarrow_{FS}$ , and  $\mathcal{L}(s) = L_{\text{Floodsub}}(\text{m2f}(s))$  if  $s \in S_{\text{Meshsub}}$ ,  $\mathcal{L}(s) = L_{\text{Floodsub}}(s)$  otherwise. Given  $s, w \in S$  and recognizer functions  $\text{mp}$  and  $\text{fp}$  for **Meshsub** and **Floodsub** states respectively, we define  $B$  as follows.

#### Definition 27 (The Relation $B$ for **Meshsub**-**Floodsub**)

$$s B w \text{ iff } \begin{cases} s = w \\ \vee \text{ fp}(s) \wedge \text{mp}(w) \wedge \text{m2f}(w) = s \end{cases}$$

**Lemma 5.3.0.2** (**IWANT** no-op under repeated exposure). *Let  $w$  be a reachable **Meshsub** state,  $m$  a message,  $j$  an active peer, and  $k$  a peer holding  $\langle \text{IWANT}, \text{hash}(m), j \rangle \in w(k).\text{Ctrl}$ . If  $\text{hasMsgp}(j, m, w)$ , then  $\text{MPldSnd}(w, k, j, m)$  is a no-op:*

$$\text{m2f}(\text{MPldSnd}(w, k, j, m)) = \text{m2f}(w).$$

*In particular, any duplicate **IWANT** for  $m$  at peer  $j$ —generated because  $m$  remained in peer  $k$ 's recently-seen window across multiple heartbeat rounds—is a no-op once the first **IWANT** has been processed and  $m$  delivered to  $j.\text{Pdg}$ .*

*Proof.* By the **MPldSnd** transition (Table 5.1), processing  $\langle \text{IWANT}, \text{hash}(m), j \rangle$  enqueues  $m$  into  $j.\text{Pdg}$  only if  $\text{activep}(j, w) \wedge \neg \text{hasMsgp}(j, m, w)$ . Since  $\text{hasMsgp}(j, m, w)$  holds by hypothesis, the body of the conditional is not taken. The transition only dequeues from  $k.\text{Ctrl}$  and (conditionally) removes  $\text{hash}(m)$  from  $j.\text{WReqs}$ . Neither **Ctrl** nor **WReqs** appear in the **m2f** projection, so  $\text{m2f}(\text{MPldSnd}(w, k, j, m)) = \text{m2f}(w)$ .  $\square$

**Lemma 5.3.0.3** (**ctrlOrd** front is fully forwarded first). *Let  $s, w \in S$  with  $sBw$ ,  $\text{fp}(s)$ ,  $\text{mp}(w)$ , and let  $m = \text{APdg}(w, i).\text{front}$  be the **ctrlOrd**-front message for active peer  $i$ . Under any FIFO **Ctrl**-step sequence  $w = w_0 \rightarrow_{\text{Ctrl}} w_1 \rightarrow_{\text{Ctrl}} \dots$  that, when choosing between queues of equal depth, processes queues in ascending peer-id order,  $\text{fullyFwd}(w_k, i, m)$  becomes true strictly before  $\text{fullyFwd}(w_{k'}, i, m_2)$  for any  $m_2 \neq m$  with  $m_2 \in \mathcal{F}(w)$ .*

*Proof.* By Def. **ctrlOrd**,  $\text{ctrlDepth}(i, m, w) \leq \text{ctrlDepth}(i, m_2, w)$  for all  $m_2$  with  $\neg \text{fullyFwd}(i, m_2, w)$ , with ties broken by ascending peer id of the queue holding the relevant **IWANT**.

The peer-id scheduling strategy is not arbitrary: since  $s = \text{m2f}(w)$ , the ordering of  $s(i).\text{Pdg} = \text{APdg}(w, i)$  is inherited from **Floodsub**, where messages are enqueued by their source peers in peer-id arrival order. Processing **Ctrl** queues in ascending peer-id order when depths tie is therefore exactly the strategy that realizes the same delivery ordering as the abstract **Floodsub** queue.

**FIFO tail-insertion invariant.** A key property used throughout is that FIFO queues insert new messages at the *tail*. Therefore, any **Ctrl** transition that enqueues a new message (e.g., **MPldReq** enqueueing a new **IWANT**, or **MMeshMnt** enqueueing a **GRAFT**) does *not* change the position *qpos* of any message already in the queue. Consequently, **ctrlDepth** values of already-queued messages are non-increasing across **Ctrl** steps: the depth of an existing **IWANT** can only decrease (as messages ahead of it are processed and dequeued) or stay the same (if a new message is appended behind it).

**Case 1: same **Ctrl** queue.** FIFO ensures that **IWANT**s for  $m$  and  $m_2$  in the same queue have distinct positions, hence  $\text{ctrlDepth}(i, m, w) \neq \text{ctrlDepth}(i, m_2, w)$ . The one with smaller depth is processed first and its message is fully forwarded first.

**Case 2: different **Ctrl** queues at the same depth.** The peer-id scheduling strategy processes  $m$ 's queue before  $m_2$ 's (since  $m$ 's source peer has lower peer id, by the tiebreaker). Hence  $m$  is fully forwarded strictly before  $m_2$ .

In both cases,  $\text{fullyFwd}(w_k, i, m)$  holds strictly before  $\text{fullyFwd}(w_{k'}, i, m_2)$  for any  $m_2 \neq m$ .  $\square$

**Theorem 5.3.1** (Control transitions under  $B$ -relatedness). *Let  $s, w \in S$  with  $sBw$ ,  $fp(s)$ , and  $mp(w)$ . Let  $m = \mathbf{APdg}(w, i).front$  for some active peer  $i$  with  $\neg \mathbf{APdg}(w, i).empty$ . For any  $\mathbf{Ctrl}$  transition  $w \rightarrow_{\mathbf{Ctrl}} w'$ , one of the following holds:*

- (a) (**Abstractly silent.**)  $m2f(w') = m2f(w)$ ; or
- (a) (**Delivers  $m$ .**) The transition is  $\mathbf{MPldSnd}(w, k, j, m)$  for some peers  $k, j$ , which enqueues  $m$  into  $w'(j).Pdg$ , making  $hasMsgp(j, m, w')$  true.

When  $\mathbf{APdg}(w, i).empty$  for all active  $i$ , every  $\mathbf{Ctrl}$  transition falls into case (a).

*Proof.* We classify each  $\mathbf{Ctrl}$  transition type.

- **IHAVE (MPldReq).** Modifies only  $\mathbf{Ctrl}$  and  $\mathbf{WReqs}$ , both outside the  $m2f$  projection. Case (a).
- **GRAFT (MGraft) and PRUNE (MPrune).** Modify only  $\mathbf{Mesh}$  and  $\mathbf{Ctrl}$ , outside the  $m2f$  projection. Case (a).
- **IWANT for  $m$  (MPldSnd).** Dequeues  $\langle \mathbf{IWANT}, \text{hash}(m), j \rangle$  from  $w(k).\mathbf{Ctrl}$ . If  $activep(j, w)$  and  $\neg hasMsgp(j, m, w)$ , enqueues  $m$  into  $w(j).Pdg$ . Case (b). If  $j$  is inactive or  $hasMsgp(j, m, w)$  already holds (e.g. a duplicate **IWANT** arising from a long recently-seen window), then by Lemma 5.3.0.2 the transition is a no-op and  $m2f(w') = m2f(w)$ . Case (a).
- **IWANT for  $m_2 \neq m$  (MPldSnd).** Delivers  $m_2$  to some peer  $j$ . This may change  $m2f(w')$  if it completes  $fullyFwd(i, m_2, w')$ . By Lemma 5.3.0.3, since  $m$  is the  $\mathbf{ctrlOrd}$ -front ( $\mathbf{ctrlDepth}(i, m, w) \leq \mathbf{ctrlDepth}(i, m_2, w)$ ),  $m_2$  cannot be fully forwarded before  $m$  is. Therefore  $m2f(w') = m2f(w)$ . Case (a).  $\square$

**Lemma 5.3.1.1** (Deterministic skipping under bounded exposure). *Suppose that for every reachable state  $w \in S_{\mathbf{Meshsub}}$ , every message  $m \in \mathcal{F}(w)$ , and every active peer  $i$  with  $m \in w(i).Seen$  and  $\neg fullyFwd(i, m, w)$ , there exists a finite concrete path  $w \xrightarrow{+}_{\mathbf{Meshsub}} v$  such that  $fullyFwd(v, i, m)$ . Then  $B$  is a RWFSK relation on  $\langle S, \rightarrow, \mathcal{L} \rangle$ , and hence  $\mathcal{M}_{\mathbf{Meshsub}} \lesssim_{m2f} \mathcal{M}_{\mathbf{Floodsub}}$ .*

*Proof.*  $\langle \forall w \in S_{\mathbf{Meshsub}} :: m2f(w) B w \rangle$  follows from the definition of  $B$ .

To prove that  $B$  is a RWFSK relation on the LTS  $\langle S, \rightarrow, \mathcal{L} \rangle$ , we show the following. The hypothesis ensures that from every reachable state the RWFSK2(b) obligation can always be discharged: for any seen-but-not-fully-forwarded message  $m$  at peer  $i$ , the hypothesis directly supplies a finite concrete path to a state where  $fullyFwd(i, m, v)$  holds, without reference to any particular algorithm or execution path.

#### Goal 5.3.1.1. RWFSK1.

*Proof.* Given  $sBw$ , we have

- if  $s = w$  then  $\mathcal{L}(s) = \mathcal{L}(w)$

- if  $\text{mp}(w) \wedge \text{fp}(s)$  then  $\mathcal{L}(w) = L_{\text{Floodsub}}(\text{m2f}(w)) = L_{\text{Floodsub}}(s) = \mathcal{L}(s)$

□

### Goal 5.3.1.2. *RWFSK2*.

*Proof.* Given states  $s, w, u \in S \wedge sBw \wedge s \rightarrow u$ . We consider cases on the  $B$  relation.

Case 1. If  $s = w$  then for any transition  $s \rightarrow u$ , we have  $w \rightarrow v$  where  $u = v$  and hence  $uBv$ .

Case 2. Otherwise, let  $\text{fp}(s) \wedge \text{mp}(w) \wedge \text{m2f}(w) = s$ . Since  $\text{fp}(s)$ , let  $s \rightarrow_{FS} u$ .

We consider cases on  $\rightarrow_{FS}$ .

- Let

$$\langle \exists m :: m.\text{Origin} \in \text{dom}(s) \wedge s(m.\text{Origin}).\text{Act} \wedge m.\text{Topic} \in s(m.\text{Origin}).\text{Pubs} \\ \wedge \bigwedge_{i \in \text{dom}(s)} \neg \text{hasMsgp}(i, m, s) : u = \text{FProduce}(s, m) \rangle$$

If  $\bigwedge_{i \in \text{dom}(s)} \neg \text{hasMsgp}(i, m, s)$  then  $\bigwedge_{i \in \text{dom}(s)} \neg \text{hasMsgp}(i, m, w)$  because  $\text{m2f}$  does not add any new message to mapped **Pdg** and **Seen** sets. Since all peers and their publishing topics are preserved in the  $\text{m2f}$  mapping, we have  $m.\text{Origin} \in \text{dom}(w) \wedge w(m.\text{Origin}).\text{Act} \wedge m.\text{Topic} \in w(m.\text{Origin}).\text{Pubs}$ . So, let  $v = \text{MProduce}(w, m)$ . Since  $\bigwedge_{i \in \text{dom}(s)} \neg \text{hasMsgp}(i, m, w)$ , no **IHAVE** for  $m$  has been generated and hence no **IWANT** for  $m$  exists in any **Ctrl** queue. By *Def. ctrlDepth*, this means  $\text{ctrlDepth}(m.\text{Origin}, m, v) = \infty$ , so  $m$  sorts last in  $\text{ctrlOrd}$  and is appended at the back of  $\text{APdg}(v, m.\text{Origin})$ , i.e.,  $\text{APdg}(v, m.\text{Origin}) = \text{APdg}(w, m.\text{Origin}).\text{enqueue}(m)$ . Then,

$$\begin{aligned} & \text{m2f}(v) \\ \equiv & \text{Def. m2f} \\ & \langle \lambda i \in \text{dom}(v) :: \langle v(i).\text{Act}, v(i).\text{Subs}, v(i).\text{Pubs}, v(i).\text{Nbrs}, \text{APdg}(v, i), \text{ASeen}(v, i) \rangle \rangle \\ \equiv & \text{dom}(v) = \text{dom}(w) \\ & \langle \lambda i \in \text{dom}(w) :: \langle v(i).\text{Act}, v(i).\text{Subs}, v(i).\text{Pubs}, v(i).\text{Nbrs}, \text{APdg}(v, i), \text{ASeen}(v, i) \rangle \rangle \\ \equiv & \text{Def. MProduce} \\ & \langle i \in \text{dom}(w) :: \langle w(i).\text{Act}, w(i).\text{Subs}, w(i).\text{Pubs}, w(i).\text{Nbrs}, \\ & \quad \text{if } i == m.\text{Origin} \text{ then } \text{APdg}(w, i).\text{enqueue}(m) \text{ else } \text{APdg}(w, i), \text{ASeen}(w, i) \rangle \rangle \\ \equiv & \text{Def. FProduce} \\ & \text{FProduce}(\text{m2f}(w), m) \\ \equiv & \text{m2f}(w) = s, \text{FProduce}(s, m) = u \\ & u \end{aligned}$$

Hence  $uBv$ .

- Let  $\langle \exists i \in \text{dom}(s) :: s(i).\text{Act} \wedge \neg s(i).\text{Pdg}.\underline{\text{empty}} \wedge u = \text{MForward}(s, i) \rangle$  and let  $m = s(i).\text{Pdg}.\underline{\text{front}}$ .

Consider the following cases:

- **Case** ( $m \notin w(i).\text{Seen}$ ): Since  $s = \text{m2f}(w)$ , we have  $s(i).\text{Pdg} = \text{APdg}(w, i)$  and so,  $m = \text{APdg}(w, i).\underline{\text{front}}$ . From *Def. m2f*, *Def. APdg*, and from the fact that  $m \notin w(i).\text{Seen}$ , we have  $m = w(i).\text{Pdg}.\underline{\text{front}}$ .

Then let  $v = \text{MForward}(w, i, D)$ . Consider the following cases:

1. **Case**  $\text{fullyFwd}(v, i, m)$ : From the transition semantics of *Meshsub*, this is possible if either  $v(i).\text{Mesh}(m.\text{Topic}) = w(i).\text{Nbrs}(m.\text{Topic})$  or  $v(i).\text{Fanout}(m.\text{Topic}) = w(i).\text{Nbrs}(m.\text{Topic})$ . In both these cases we have  $uBv$  because  $m$  gets forwarded to all of the subscribing neighbors of  $i$ , just like in *Floodsub*.
2. **Otherwise**  $\neg \text{fullyFwd}(v, i, m)$ , *i.e.*, only a subset of  $w(i).\text{Nbrs}(m.\text{Topic})$  receive  $m$ . From our correctness condition,  $|\text{RSeen}| > 0$ , and hence  $m \in v(i).\text{RSeen}[0]$  (from *Def. MForward*). From *Def. F*, we get  $m \in \mathcal{F}(v)$ . Therefore,  $\text{ASeen}(v, i) = w(i).\text{Seen}$  (partially forwarded  $m$  gets removed), and for all neighbors  $j$ ,  $v(j).\text{APdg} = w(j).\text{Pdg}$ . Since  $m$  is partially forwarded in  $v$  and  $m \in v(i).\text{Seen}$ , from *Def. APdg*, we have that  $\text{APdg}(v, i) = w(i).\text{Pdg}$ . Hence  $\text{m2f}(v) = \text{m2f}(w)$ .

Since  $s = \text{m2f}(w)$ , we get  $s = \text{m2f}(v)$  and hence  $sBv$ . This brings us into the case  $sBv \wedge (m \in v(i).\text{Seen})$ , which we handle next. This reduction is well-founded by the following direct argument. Define

$$K(v) = |\{j \in v(i).\text{Nbrs}(m.\text{Topic}) \mid \text{activep}(j, v) \wedge \neg \text{hasMsgp}(j, m, v)\}|,$$

the number of active neighbors of  $i$  that have not yet received  $m$ . Each application of  $\text{MForward}(w, i, D)$  enqueues  $m$  into the **Pdg** of every active mesh (or fanout) neighbor that does not yet have  $m$ , making *hasMsgp* true for each such neighbor. Since at least one neighbor receives  $m$  per **MForward** step (otherwise *fullyFwd* would already hold),  $K$  strictly decreases by at least one. As  $K \geq 0$ , after at most  $|\text{Nbrs}(m.\text{Topic})|$  applications of **MForward** we reach  $K = 0$ , *i.e.*,  $m \in v(i).\text{Seen}$ , entering the next case.

- **Case** ( $m \in v(i).\text{Seen}$ ): We construct a concrete path using the following algorithm.

---

**Algorithm 3** EVENTUAL-FULL-FORWARD

---

**Require:** Meshsub state  $w$ , active peer  $i$ , message  $m \in w(i).$ Seen

**Ensure:** A state  $v$  such that  $\text{fullyFwd}(v, i, m)$

```
1:  $v \leftarrow w$ 
2: while  $\neg \text{fullyFwd}(v, i, m)$  do
3:    $U \leftarrow \{j \mid \text{activep}(v, j) \wedge j \in v(i).\text{Nbrs}(m.\text{Topic}) \wedge \neg \text{hasMsgp}(j, m, \text{m2f}(v))\}$ 
4:   if  $\exists j \in U$  such that  $\langle \text{IHAVE}, \text{hash}(m), - \rangle \notin v(j).\text{Ctrl}$  then
5:      $v \leftarrow \text{MHBm}(v, i, D, W, c)$   $\triangleright$  Phase 1: heartbeat gossip round
6:   else
7:     Choose any  $j \in v(i).\text{Nbrs}(m.\text{Topic})$  with  $\neg v(j).\text{Ctrl.empty}$ 
8:     Execute the Ctrl transition for  $v(j).\text{Ctrl.front}$ , producing  $v'$   $\triangleright$  Phase 2: one
9:      $v \leftarrow v'$ 
10:   end if
11: end while
    return  $v$ 
```

---

*Remark 5.3.1.* Phase 2 processes *one* **Ctrl** transition per outer-loop iteration, rather than draining all **Ctrl** queues at once. This is essential: FIFO queues may contain  $\langle \text{IWANT}, \text{hash}(m_2), - \rangle$  messages for  $m_2 \neq m$  ahead of  $\langle \text{IWANT}, \text{hash}(m), - \rangle$ . Processing one step at a time and re-checking  $\text{fullyFwd}(v, i, m)$  after each step ensures the loop exits exactly when  $m$  is fully forwarded, without committing any  $m_2 \neq m$  beyond what  $\text{ctrlOrd}$  permits.

By Theorem 5.3.3 and Theorem 5.3.4, EVENTUAL-FULL-FORWARD produces a finite path  $w \xrightarrow{+}_{\text{Meshsub}} v$  with  $\text{fullyFwd}(v, i, m)$  and  $uBv$ , thereby satisfying the hypothesis of Lemma 5.3.1.1 and RWFSK2(b).

- Let  $\langle \exists i : \text{activep}(i, s) \wedge s(i).\text{Pdg.empty} \wedge \text{redundant}(i, s, T) : u = \text{FLeave}(s, i) \rangle$ . Then, under the  $\text{m2f}$  mapping we have  $\text{activep}(i, w) \wedge \text{APdg}(w, i).\text{empty}$ .

We first establish that every **Ctrl** transition from  $w$  is abstractly silent.

Since  $\text{APdg}(w, i).\text{empty}$ , there is no front message  $m$  in  $\text{APdg}(w, i)$ , so by Theorem 5.3.1 (the  $\text{APdg}(w, i).\text{empty}$  case), every **Ctrl** transition falls into case (a) ( $\text{m2f}(w') = \text{m2f}(w)$ ), and thus  $sBw'$  for every  $w \rightarrow_{\text{Ctrl}} w'$ .

We now construct the matching concrete path using the following algorithm.

---

**Algorithm 4** CTRL-DRAIN-THEN-LEAVE

---

**Require:** Meshsub state  $w$ , departing peer  $i$

**Ensure:** State  $v$  with  $\text{m2f}(v) = \text{FLeave}(\text{m2f}(w), i)$

```
1:  $v \leftarrow \text{DRAIN-CTRL-Q}(w, \{i\})$ 
2:  $v \leftarrow \text{MLeave}(v, i)$  return  $v$ 
```

---

**Theorem 5.3.2** (Termination of CTRL-DRAIN-THEN-LEAVE). *CTRL-DRAIN-THEN-LEAVE terminates and produces a state  $v$  reachable from  $w$  via  $\xrightarrow{+}_{\text{Meshsub}}$  such that  $\text{m2f}(v) = \text{FLeave}(\text{m2f}(w), i)$ .*



*Proof. Termination.* DRAIN-CTRL-Q( $w, \{i\}$ ) terminates by Theorem 5.1.1. The subsequent **MLeave** is a single transition, so the algorithm terminates overall.

**Reachability.** Each iteration of DRAIN-CTRL-Q executes one **Ctrl** transition of **Meshsub**, so its output  $v_0$  satisfies  $w \rightarrow_{\text{Meshsub}}^* v_0$ . Since  $\text{APdg}(w, i).empty$  and DRAIN-CTRL-Q does not enqueue payload messages,  $\text{APdg}(v_0, i).empty$  and  $activep(i, v_0)$ , so **MLeave**( $v_0, i$ ) is enabled. Hence  $w \rightarrow_{\text{Meshsub}}^+ v$ .

**Abstract state correspondence.** Let  $v_0 = \text{DRAIN-CTRL-Q}(w, \{i\})$  and  $v = \text{MLeave}(v_0, i)$ . Since every **Ctrl** step from  $w$  to  $v_0$  is abstractly silent,  $m2f(v_0) = m2f(w)$ . We then compute:

$$\begin{aligned}
& m2f(v) \\
& \equiv \text{Def. } m2f \\
& \quad \langle \lambda j \in \text{dom}(v) :: \langle v(j).Act, v(j).Subs, v(j).Pubs, v(j).Nbrs, \text{APdg}(v, j), \text{ASeen}(v, j)) \rangle \rangle \\
& \equiv \text{dom}(v) = \text{dom}(v_0) \\
& \quad \langle \lambda j \in \text{dom}(v_0) :: \langle v(j).Act, v(j).Subs, v(j).Pubs, v(j).Nbrs, \text{APdg}(v, j), \text{ASeen}(v, j)) \rangle \rangle \\
& \equiv \text{Def. MLeave, APdg, ASeen} \\
& \quad \langle j \in \text{dom}(v_0) :: \langle \text{if } j=i \text{ then false else } v_0(j).Act, v_0(j).Subs, v_0(j).Pubs, \\
& \quad \quad \langle \forall t \in v_0(i).Subs :: \text{if } j \in v_0(i).Nbrs(t) \text{ then } v_0(j).Nbrs(t) := \{i\}, \\
& \quad \quad \text{APdg}(v_0, j), \text{ASeen}(v_0, j) \rangle \rangle \rangle \\
& \equiv m2f(v_0) = m2f(w), \text{ Def. } m2f, \text{ Def. FLeave} \\
& \quad \text{FLeave}(m2f(w), i) \\
& \equiv m2f(w) = s, \text{ FLeave}(s, i) = u \\
& \quad u
\end{aligned}$$

□

By Theorem 5.3.2,  $w \rightarrow_{\text{Meshsub}}^+ v$  and  $m2f(v) = u$ . Since  $u \in S_{\text{Floodsub}}$ , we have  $uBv$  by Def. B. This satisfies RWFSK2(b):  $\langle \exists v :: w \rightarrow^+ v \wedge uBv \rangle$ .

Hence  $uBv$ .

- Let  $\langle \exists i \notin \text{dom}(s), \text{Pubs}, \text{Subs}, \text{Nbrs} :: i \notin \text{rng}(nbrs) \wedge \langle \forall t \in \text{subs} \cup \text{pubs} :: \langle \exists j :: j \in nbrs(t) \rangle \rangle \wedge u = \text{FJoin}(s, i, \text{Subs}, \text{Pubs}, \text{Nbrs}) \rangle$   
Then, under the  $m2f$  mapping we have  $i \notin \text{dom}(w) \wedge i \notin \text{rng}(\text{Nbrs}) \wedge \langle \forall t \in \text{Subs} \cup \text{Pubs} :: \langle \exists j :: j \in \text{Nbrs}(t) \rangle \rangle$ .

Let  $v = \text{MJoin}(s, i, \text{Subs}, \text{Pubs}, \text{Nbrs}, \text{Mesh})$ . Then,

$$\begin{aligned}
& \mathbf{m2f}(v) \\
& \equiv \text{Def. } \mathbf{m2f} \\
& \quad \langle \lambda j \in \text{dom}(v) :: \langle v(j).\text{Act}, v(j).\text{Subs}, v(j).\text{Pubs}, v(j).\text{Nbrs}, \text{APdg}(v, j), \text{ASeen}(v, j)) \rangle \\
& \equiv \text{dom}(v) = \text{dom}(w) \cup \{i\} \\
& \quad \underline{\text{extend}}(\langle \lambda j \in \text{dom}(w) :: \\
& \quad \quad \langle v(j).\text{Act}, v(j).\text{Subs}, v(j).\text{Pubs}, v(j).\text{Nbrs}, \text{APdg}(v, j), \text{ASeen}(v, j)) \rangle, \\
& \quad \quad i, \\
& \quad \quad \langle \text{Subs}, \text{Pubs}, \text{Nbrs}, \text{Mesh}, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle_i \\
& \equiv \text{Def. } \mathbf{MJoin}, \mathbf{APdg}, \mathbf{ASeen} \\
& \quad \underline{\text{extend}}(\langle \lambda j \in \text{dom}(w) :: \\
& \quad \quad \langle w(j).\text{Act}, w(j).\text{Subs}, w(j).\text{Pubs}, w(j).\text{Nbrs}, \text{APdg}(w, j), \text{ASeen}(w, j)) \rangle, \\
& \quad \quad i, \\
& \quad \quad \langle \text{Subs}, \text{Pubs}, \text{Nbrs}, \text{Mesh}, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle_i \\
& \equiv \text{Def. } \mathbf{m2f}, \text{Def. } \mathbf{FJoin} \\
& \quad \mathbf{FJoin}(\mathbf{m2f}(w), i, \text{Subs}, \text{Pubs}, \text{Nbrs}) \\
& \equiv \mathbf{m2f}(w) = s, \mathbf{FJoin}(s, i, \text{Subs}, \text{Pubs}, \text{Nbrs}) = u \\
& \quad u
\end{aligned}$$

Hence  $uBv$ .

- Let  $\langle \exists i, T : \text{activep}(i, s) \wedge T \in 2^{\mathcal{T}} : u = \mathbf{FSubscribe}(s, i, T) \rangle$ . Then, under the  $\mathbf{m2f}$  mapping we have  $\text{activep}(i, w)$ . Let  $v = \mathbf{MSubscribe}(s, i, T, D)$ . Then,

$$\begin{aligned}
& \mathbf{m2f}(v) \\
& \equiv \text{Def. } \mathbf{m2f} \\
& \quad \langle \lambda j \in \text{dom}(v) :: \langle v(j).\text{Act}, v(j).\text{Subs}, v(j).\text{Pubs}, v(j).\text{Nbrs}, \text{APdg}(v, j), \text{ASeen}(v, j)) \rangle \\
& \equiv \text{dom}(v) = \text{dom}(w) \\
& \quad \langle \lambda j \in \text{dom}(w) :: \langle v(j).\text{Act}, v(j).\text{Subs}, v(j).\text{Pubs}, v(j).\text{Nbrs}, \text{APdg}(v, j), \text{ASeen}(v, j)) \rangle \\
& \equiv \text{Def. } \mathbf{MSubscribe}, \mathbf{APdg}, \mathbf{ASeen} \\
& \quad \langle \lambda j \in \text{dom}(w) :: \langle v(j).\text{Act}, \\
& \quad \quad \underline{\text{if}} \ j == i \ \underline{\text{then}} \ w(j).\text{Subs} : \cup = T \ \underline{\text{else}} \ w(j).\text{Subs}, \\
& \quad \quad w(j).\text{Pubs}, \\
& \quad \quad \langle \forall t \in T :: \underline{\text{if}} \ i \in \text{rng}(w(j).\text{Nbrs}) \ \underline{\text{then}} \ w(j).\text{Nbrs}(t) : \cup = \{i\} \ \underline{\text{else}} \ s(j).\text{Nbrs} \rangle,
\end{aligned}$$

$$\begin{aligned}
& \text{APdg}(w, j), \\
& \text{ASeen}(w, j) \rangle \rangle \rangle \\
\equiv & \text{Def. m2f, Def. FSubscribe} \\
& \text{FSubscribe}(\text{m2f}(w), i, T) \\
\equiv & \text{m2f}(w) = s, \text{FSubscribe}(s, i, T) = u \\
& u
\end{aligned}$$

Hence  $uBv$ .

- Let  $\langle \exists i, T : \text{activep}(i, s) \wedge T \in 2^{\mathcal{T}} \wedge \text{redundant}(i, s, T) : u = \text{FUnsubscribe}(s, i, T) \rangle$ . Then, under the  $\text{m2f}$  mapping we have  $\text{activep}(i, w)$ . Let  $v = \text{MUnsubscribe}(s, i, T)$ . Then,

$$\begin{aligned}
& \text{m2f}(v) \\
\equiv & \text{Def. m2f} \\
& \langle \lambda j \in \text{dom}(v) :: \langle v(j).\text{Act}, v(j).\text{Subs}, v(j).\text{Pubs}, v(j).\text{Nbrs}, \text{APdg}(v, j), \text{ASeen}(v, j) \rangle \rangle \\
\equiv & \text{dom}(v) = \text{dom}(w) \\
& \langle \lambda j \in \text{dom}(w) :: \langle v(j).\text{Act}, v(j).\text{Subs}, v(j).\text{Pubs}, v(j).\text{Nbrs}, \text{APdg}(v, j), \text{ASeen}(v, j) \rangle \rangle \\
\equiv & \text{Def. MUnsubscribe, APdg, ASeen} \\
& \langle \lambda j \in \text{dom}(w) :: \langle v(j).\text{Act}, \\
& \quad \underline{\text{if}} \ j == i \ \underline{\text{then}} \ w(j).\text{Subs} : \underline{\neq} \ T \ \underline{\text{else}} \ w(j).\text{Subs}, \\
& \quad w(j).\text{Pubs}, \\
& \quad \langle \forall t \in T :: \underline{\text{if}} \ i \in \text{rng}(w(j).\text{Nbrs}) \ \underline{\text{then}} \ w(j).\text{Nbrs}(t) : \underline{\neq} \ \{i\} \ \underline{\text{else}} \ s(j).\text{Nbrs} \rangle, \\
& \quad \text{APdg}(w, j), \\
& \quad \text{ASeen}(w, j) \rangle \rangle \rangle \\
\equiv & \text{Def. m2f, Def. FUnsubscribe} \\
& \text{FUnsubscribe}(\text{m2f}(w), i, T) \\
\equiv & \text{m2f}(w) = s, \text{FUnsubscribe}(s, i, T) = u \\
& u
\end{aligned}$$

Hence  $uBv$ .

□

□

**Theorem 5.3.3** (Probabilistic Termination of EVENTUAL-FULL-FORWARD). *Let  $w$  be a reachable **Meshsub** state,  $i$  an active peer, and  $m \in w(i).\text{Seen}$ . Under the refinement conditions and the probabilistic semantics of **MHBM**, for every  $\varepsilon > 0$  there exists  $t(\varepsilon) = O(n \log n)$  such that  $\text{EVENTUAL-FULL-FORWARD}(w, i, m)$  terminates with probability at least  $1 - \varepsilon$ .*

*Proof.* The algorithm consists of two phases.

**Phase 1 (Gossip Exposure).** While there exists an active neighbor  $j$  of  $i$  that has not yet received  $m$  and does not have an **IHAVE** for  $m$  in its **Ctrl** queue, the algorithm executes **MHBM** transitions. Peer selection in **MHBM** is performed via  $\text{select}(k, S)$ , which samples uniformly at random and independently across rounds, so the coupon-collector bound applies. By Lemma 5.3.0.1, Phase 1 completes within  $t(\varepsilon)$  gossip rounds with probability at least  $1 - \varepsilon$ .

**Phase 2 (One-step Ctrl processing).** Once all relevant neighbors have **IHAVE** for  $m$ , each outer-loop iteration executes one **Ctrl** transition from  $P$ 's queues. By Lemma 5.3.0.3,  $m$  has minimum **ctrlDepth** among all frontier messages, so  $m$  is fully forwarded before any  $m_2 \neq m$ . The DRAIN-CTRL-Q measure (Theorem 5.1.1) strictly decreases at each **Ctrl** step, so Phase 2 terminates deterministically.

Therefore, with probability at least  $1 - \varepsilon$ , **EVENTUAL-FULL-FORWARD** terminates.  $\square$

**Lemma 5.3.3.1** (APdg invariant under **EVENTUAL-FULL-FORWARD**). *Let  $w$  be a reachable Meshsub state,  $i$  an active peer, and  $m \in w(i).\text{Seen}$ . Let  $w'$  be any intermediate state reached during **EVENTUAL-FULL-FORWARD**( $w, i, m$ ). Then for every peer  $k$  and every message  $m_2 \neq m$ :*

- (i)  $w'(k).\text{Seen} = w(k).\text{Seen}$ ,
- (i)  $w'(k).\text{Nbrs} = w(k).\text{Nbrs}$ ,
- (i)  $\text{fullyFwd}(k, m_2, w') = \text{fullyFwd}(k, m_2, w)$ .

Consequently,

$$\text{APdg}(w', k) \setminus \{m\} = \text{APdg}(w, k) \setminus \{m\}$$

for all peers  $k$ .

*Proof.* **EVENTUAL-FULL-FORWARD** executes two kinds of transitions:

**Phase 1 (MHBM transitions)**  $\text{MHBM}(s, i, D, W, c)$  is the sequential composition of three auxiliary transitions:

- **MGossip**: enqueues **IHAVE** messages into **Ctrl** queues and resets **RSeen**[0]. Modifies only **Ctrl** and **RSeen**.
- **MFanoutMnt**: adds or removes entries from **Fanout**. Modifies only **Fanout**.
- **MMeshMnt**: adds or removes peers from **Mesh** and enqueues **GRAFT/PRUNE** into **Ctrl**. Modifies only **Mesh** and **Ctrl**.

None of these sub-transitions touches **Seen**, **Nbrs**, or **Pdg** of any peer. Therefore conditions (i)–(iii) hold after every **MHBM** step.

**Phase 2 (one-step Ctrl processing)** Each iteration executes one **Ctrl** transition from some  $j \in P$ . By Lemma 5.3.0.3,  $m$  is the **ctrlOrd**-front, so  $m$  is fully forwarded before any  $m_2 \neq m$ . Until the loop terminates (when  $\text{fullyFwd}(v, i, m)$  holds), no  $m_2 \neq m$  has been fully forwarded. Therefore for  $m_2 \neq m$ :

- **IHAVE/GRAFT/PRUNE Ctrl transitions:** modify only **Ctrl**, **Mesh**, **WReqs** — no change to **Seen**, **Nbrs**, **Pdg**. Conditions (i)–(iii) preserved.
- **IWANT-for- $m$  Ctrl transitions (MPldSnd):** enqueue  $m$  into  $w'(j).$ Pdg only; **Pdg** unchanged for  $m_2 \neq m$ . Conditions (i)–(iii) preserved for  $m_2$ .
- **IWANT-for- $m_2$  Ctrl transitions (MPldSnd):** may enqueue  $m_2$  into  $w'(j).$ Pdg, changing  $m_2$ 's **Pdg** membership. However,  $hasMsgp(j, m_2, w')$  thereby becomes true for peer  $j$ . For  $fullyFwd(k, m_2, w')$  to become true, *all* active neighbors of  $k$  would need to satisfy  $hasMsgp$ . By Lemma 5.3.0.3,  $m$  is fully forwarded before  $m_2$ , so the loop exits before  $fullyFwd(k, m_2, \_)$  can become true. Hence condition (iii) is preserved for  $m_2$ .

**Consequence.** Condition (iii) directly gives  $fullyFwd(k, m_2, w') = fullyFwd(k, m_2, w)$  for all  $k$  and  $m_2 \neq m$ . Since  $ctrlDepth(k, m_2, \_)$  is defined by **IWANT** queue positions, and by the FIFO tail-insertion invariant (Lemma 5.3.0.3) no existing **IWANT** changes position, we have  $ctrlOrd(k, w') = ctrlOrd(k, w)$  on messages  $m_2 \neq m$ . By *Def. APdg*, both components are unchanged for messages  $m_2 \neq m$ , and hence  $APdg(w', k) \setminus \{m\} = APdg(w, k) \setminus \{m\}$ .  $\square$

**Theorem 5.3.4** ( $uBv$  after EVENTUAL-FULL-FORWARD). *Let  $s, w \in S$  with  $sBw$ ,  $fp(s)$ ,  $mp(w)$ ,  $m \in w(i).$ Seen, and  $m = APdg(w, i).$ front. Let  $v$  be the state produced by EVENTUAL-FULL-FORWARD( $w, i, m$ ), so that  $fullyFwd(v, i, m)$  holds. Let  $u = FForward(s, i)$ . Then  $w \rightarrow_{Meshsub}^+ v$  and  $uBv$ .*

*Proof.* **Reachability.** EVENTUAL-FULL-FORWARD executes **MHBM** transitions (Phase 1) and one-step **Ctrl** transitions (Phase 2), each of which is a valid **Meshsub** transition. By Theorem 5.3.3 the algorithm terminates, so  $w \rightarrow_{Meshsub}^+ v$ .

$m2f(v) = u$ . We show this in three steps.

**Step 1: Phase 2 commits only  $m$ .** By Lemma 5.3.0.3,  $m$  is the  $ctrlOrd$ -front and is therefore fully forwarded before any  $m_2 \neq m$ . Phase 2 terminates as soon as  $fullyFwd(v, i, m)$  holds. At that point, no  $m_2 \neq m$  has yet been fully forwarded, so  $m2f(v)$  advances only  $m$ . By Lemma 5.3.0.2, any duplicate **IWANT-for- $m$**  (arising from a long recently-seen window) is a no-op once  $hasMsgp(j, m, v)$  holds.

**Step 2:  $fullyFwd(v, i, m)$  yields  $m2f(v)$ .** Since  $fullyFwd(v, i, m)$ , every active neighbor  $j \in v(i).$ Nbrs( $m.Topic$ ) satisfies  $hasMsgp(j, m, v)$ . Therefore  $m \in ASeen(v, i)$  (*Def. ASeen*). By Lemma 5.3.3.1, for all  $m_2 \neq m$  and all peers  $k$ ,  $fullyFwd(k, m_2, v) = fullyFwd(k, m_2, w)$  and  $APdg(v, k) \setminus \{m\} = APdg(w, k) \setminus \{m\}$ . Since  $m$  is now fully forwarded in  $v$ , it moves from the first component of **APdg** into **ASeen**, so  $APdg(v, i) = APdg(w, i) \setminus \{m\}$ .

**Step 3: match with  $u = FForward(s, i)$  via the  $sBw$  constraint.** Since  $sBw$  with  $fp(s)$  and  $mp(w)$ , we have  $s = m2f(w)$ , and in particular  $s(i).$ Pdg =  $APdg(w, i)$ . The  $ctrlOrd$  tiebreaker was chosen to match the peer-id arrival order of  $s(i).$ Pdg, so  $m = APdg(w, i).$ front =  $s(i).$ Pdg.front: the concrete and abstract queues agree on which message is next. The abstract transition  $FForward(s, i)$  dequeues  $m$  from  $s(i).$ Pdg, adds  $m$  to  $s(i).$ Seen, and records that all neighbors have received  $m$ . By Steps 1 and 2,  $m2f(v)$  reflects exactly these changes:  $m$  removed from  $APdg(v, i)$ ,  $m$  added to  $ASeen(v, i)$ , and  $hasMsgp(j, m, v)$  for all active neighbors  $j \in v(i).$ Nbrs( $m.Topic$ ). Hence  $m2f(v) = FForward(s, i) = u$ .

Since  $u \in S_{Floodsub}$  and  $m2f(v) = u$ , Definition 5.3.4 gives  $uBv$ .  $\square$

We emphasize that the RWFSK obligations are discharged deterministically, given bounded exposure; probability is used only to bound the number of gossip rounds required to achieve that exposure at each reachable state.

**Lemma 5.3.4.1** (High-probability RWFSK satisfaction). *For any reachable state  $w \in S_{\text{Meshsub}}$  and any  $\varepsilon > 0$ ,*

$$\Pr_w[\text{the RWFSK obligations for } \lesssim_{m2f} \text{ are satisfied}] \geq 1 - \varepsilon.$$

*Proof.* Fix a reachable state  $w \in S_{\text{Meshsub}}$ . Let  $F = |\mathcal{F}(w)|$  be the number of messages in the frontier at  $w$ . If  $F = 0$  the RWFSK obligations are trivially satisfied, so assume  $F \geq 1$ .

**Step 1: per-message-peer termination.** Let  $N = |\text{dom}(w)|$  be the number of peers. Fix any message  $m \in \mathcal{F}(w)$  and any active peer  $i$ . By Theorem 5.3.3, for every  $\delta > 0$ , EVENTUAL-FULL-FORWARD( $w, i, m$ ) terminates within  $t(\delta) = O(n \log n)$  steps with probability at least  $1 - \delta$ . Setting  $\delta = \varepsilon/(F \cdot N)$ , the algorithm terminates for the specific pair  $(m, i)$  with probability at least  $1 - \varepsilon/(F \cdot N)$ .

**Step 2: union bound over frontier.** There are at most  $F \cdot N$  message-peer pairs. Applying Step 1 to each pair with failure probability  $\varepsilon/(F \cdot N)$  and taking a union bound, the probability that EVENTUAL-FULL-FORWARD fails to terminate for *any* pair is at most

$$F \cdot N \cdot \frac{\varepsilon}{F \cdot N} = \varepsilon.$$

Hence with probability at least  $1 - \varepsilon$ , EVENTUAL-FULL-FORWARD terminates for every message  $m \in \mathcal{F}(w)$  and every active peer  $i$ .

**Step 3: apply branching-time lemma.** By Theorem 5.3.4, whenever EVENTUAL-FULL-FORWARD( $w, i, m$ ) terminates it produces a finite path  $w \xrightarrow{+}_{\text{Meshsub}} v$  with  $\text{fullyFwd}(v, i, m)$ . Step 2 establishes that termination holds with probability at least  $1 - \varepsilon$  for every  $(m, i)$  pair. Hence with probability at least  $1 - \varepsilon$ , for every reachable state  $w$ , every  $m \in \mathcal{F}(w)$ , and every active peer  $i$  with  $m \in w(i).\text{Seen}$  and  $\neg \text{fullyFwd}(i, m, w)$ , there exists a finite path  $w \xrightarrow{+}_{\text{Meshsub}} v$  with  $\text{fullyFwd}(v, i, m)$ . This is exactly the hypothesis of Lemma 5.3.1.1, which then gives  $B$  witnesses  $\mathcal{M}_{\text{Meshsub}} \lesssim_{m2f} \mathcal{M}_{\text{Floodsub}}$ , and the result follows.  $\square$

**Theorem 5.3.5** (Probabilistic skipping refinement of Meshsub). *Under uniform random peer selection via select and fair heartbeat scheduling, for any  $\varepsilon > 0$ ,*

$$\mathcal{M}_{\text{Meshsub}} \lesssim_{m2f}^{\varepsilon} \mathcal{M}_{\text{Floodsub}}.$$

*Proof.* By Lemma 5.3.4.1, from every reachable state of  $\text{Meshsub}$  the RWFSK obligations for  $\lesssim_{m2f}$  are satisfied with probability at least  $1 - \varepsilon$ . By Definition 5.3.3, this is exactly the statement  $\mathcal{M}_{\text{Meshsub}} \lesssim_{m2f}^{\varepsilon} \mathcal{M}_{\text{Floodsub}}$ .  $\square$

**Theorem 5.3.6** (Meshsub refines Broadcastsub). *For any  $\varepsilon > 0$ ,*

$$\mathcal{M}_{\text{Meshsub}} \lesssim_{\text{f2b} \circ \text{m2f}}^{\varepsilon} \mathcal{M}_{\text{Broadcastsub}}$$

*Proof.* Follows from Theorem 5.3.5, Theorem 3.5.1, and the Compositional Skipping Refinement theorem (Chapter 2).  $\square$

To the best of our knowledge, this is the first probabilistic refinement proof connecting a mesh-based gossip protocol to an abstract broadcast model using refinement.

**Theorem 5.3.7** (Meshsub Probabilistic Commitment Implies Delivery). *Under weak fairness, temporal connectedness, and gossip coverage assumptions, for every  $\varepsilon > 0$ , the  $\text{Commitment} \Rightarrow \text{Delivery}$  property holds for  $\mathcal{M}_{\text{Meshsub}}$  with probability at least  $1 - \varepsilon$ .*

*Proof.* By Theorem 3.1.1, Broadcastsub satisfies the commitment-to-delivery property. By Theorem 5.3.6,

$$\mathcal{M}_{\text{Meshsub}} \lesssim_{\text{f2b} \circ \text{m2f}}^{\varepsilon} \mathcal{M}_{\text{Broadcastsub}}.$$

Since  $\text{Commitment} \Rightarrow \text{Delivery}$  is expressible in  $ACTL^* \setminus X$ , and probabilistic skipping refinement preserves all  $ACTL^* \setminus X$  properties with probability at least  $1 - \varepsilon$  [27, 28], the property holds for  $\mathcal{M}_{\text{Meshsub}}$  with probability at least  $1 - \varepsilon$ .  $\square$

## 5.4 Discussion: Meshsub Refines Broadcastsub

We conclude this chapter by summarizing the refinement relationship established between Meshsub and Broadcastsub, and by clarifying the conditions under which this result holds.

The proof proceeds in two stages. First, we show that Meshsub is an  $\varepsilon$ -probabilistic skipping refinement of Floodsub under the refinement map  $\text{m2f}$  (Theorem 5.3.5). This result captures the intuition that Meshsub preserves the observable dissemination behavior of Floodsub, up to finite skipping arising from gossip-based recovery and mesh maintenance. The use of a commitment-based refinement map  $\text{m2f}$  ensures that only fully forwarded message deliveries are exposed at the abstract level, while transient protocol mechanisms are abstracted away. Probabilistic reasoning is employed solely to bound the number of gossip rounds required to achieve exposure at each reachable state; once exposure is guaranteed, all RWFSK obligations are discharged deterministically.

Second, Floodsub is shown to be a skipping refinement of Broadcastsub (3.5.1) under the refinement map  $\text{f2b}$ , reflecting the fact that Floodsub implements broadcast by eagerly forwarding messages to all neighbors without introducing additional observable behavior. By composing these two skipping refinement steps via the Compositional Skipping Refinement theorem (Chapter 2), we obtain a direct probabilistic skipping refinement from Meshsub to Broadcastsub (Theorem 5.3.6).

The resulting refinement statement is strong: it preserves all branching-time properties expressible in  $ACTL^* \setminus X$ . Consequently, any safety or liveness property verified on the abstract Broadcastsub model, excluding next-time properties, also holds for Meshsub, modulo

the probabilistic bound  $\varepsilon$ . This establishes Meshsub as a correct and efficient implementation of Broadcastsub, justifying its use in large-scale decentralized systems where bandwidth efficiency and robustness are paramount.

Crucially, the refinement from Meshsub to Broadcastsub holds under the same correctness conditions required for the intermediate refinements. These include weak fairness of heartbeat transitions, sufficient coverage of neighbors by eager and lazy peerings, a non-empty recently-seen window to support gossip-based recovery, and temporal connectedness of the underlying network. Under these assumptions, the  $\text{Commitment} \Rightarrow \text{Delivery}$  property holds for Meshsub with probability at least  $1 - \varepsilon$ .



## Chapter 6

# Formalization and Property-Based Testing of Gossipsub v1.1

Gossipsub v1.1 [49] extends Meshsub with security-oriented mechanisms intended to improve robustness against adversarial behavior in permissionless networks. These extensions preserve the underlying mesh-and-gossip dissemination structure of Meshsub, while adapting peer selection and mesh maintenance decisions based on observed peer behavior. In this dissertation, we model and analyze a restricted subset of Gossipsub v1.1, focusing on those mechanisms that directly affect mesh formation and message dissemination. Our model of Gossipsub is written in ACL2S and is publicly available, open, extensible, formal, official and executable [36]. It supports reasoning about protocol properties and components, as well as running simulations that allow precise control over the network, peers and event ordering.

At the core of Gossipsub v1.1 is a *peer scoring* mechanism, whereby each peer locally assigns scores to its neighbors based on their observed behavior during message propagation and gossip exchange. These scores influence mesh maintenance decisions, including the retention of well-behaving peers and the pruning of peers whose behavior is deemed undesirable. As a result, scoring introduces a feedback loop between message dissemination outcomes and the evolving structure of the overlay mesh.

Other features described in the Gossipsub v1.1 specification—such as outbound mesh quotas, adaptive gossip dissemination, prune peer exchange, and application-level message validation—are not modeled in our formalization. By excluding these orthogonal extensions, our analysis isolates the impact of peer scoring on the Meshsub protocol core, enabling precise reasoning about its security implications.

Our model captures the essential distinction between Meshsub and Gossipsub v1.1 relevant to this work: while Meshsub relies on randomized peer selection and static degree bounds, Gossipsub v1.1 adapts mesh membership based on peer behavior. As we demonstrate in later chapters, this behavioral adaptation—despite being intended as a security enhancement—introduces subtle interactions that can lead to invalidation of expected dissemination properties (as in Meshsub).

This chapter is organized as follows. Section 6.1 describes our formal model for Gossipsub in ACL2S. Section 6.2 discusses and reasons about the scoring function component of Gossipsub. It defines the fundamental property which should hold for any scoring function that claims to defend against malicious peers, and then shows that it does not hold for

Gossipsub v1.1. Finally Section 6.3 relates the fundamental property to the commitment-implies-delivery property which we proved for **Broadcastsub**, **Floodsub** and **Meshsub** in the previous chapters.

**Chapter provenance.** This chapter is based on work previously published in [41, 43]. The presentation has been revised and extended for inclusion in this dissertation.

## 6.1 GossipSub ACL2s Model Description

In this section, we describe our ACL2s model, while also giving an overview of how the Gossipsub protocol works. Interested readers are encouraged to walk through our ACL2s formalization whose presentation mirrors this section [80]. Consider the mesh shown in Figure 6.1.1. Full-message payloads are forwarded on the full-message peering edges, which

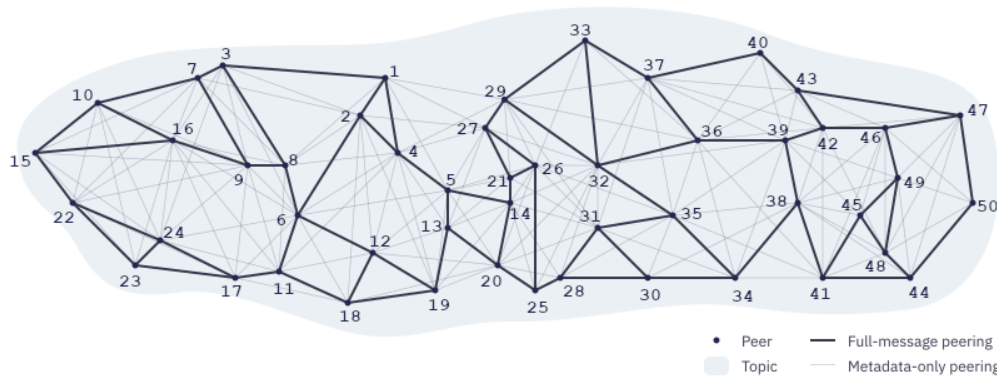


Figure 6.1.1: A mesh of peers subscribing to the same topic. This Figure was taken from [74].

consume more bandwidth, while the metadata-only peering edges are used only to advertise and request full-messages using corresponding “**IHAVE**” and “**IWANT**” Remote Procedure Calls (RPCs). These RPCs carry the metadata of the full-message being advertised or requested, which is considerably smaller than the message itself. In this way, network congestion is kept in check, and full-messages are supposed to be eventually disseminated to all peers who subscribe to the given topic. As an example, peers numbered 1 and 2 can send full-messages to each other, since they are mesh neighbors on the illustrated topic and share a full-message peering edge. However, peer 29 can receive a full-message from 1 only if it requests one, by sending an **IWANT** message in response to a corresponding **IHAVE** message received from peer 1. Note, Figure 6.1.1 only shows a single Gossipsub mesh. However, real world applications can have an arbitrary number of topics and corresponding meshes, *e.g.*, Eth2.0 supports up to 71 topics. Figure 6.1.2 illustrates a community graph of an actual Eth2.0 network from Li et. al. [47].

In order to model and analyze Gossipsub in ACL2s, we rely heavily on Defdata to easily model network components, and on cgen for property-based testing. Distributed systems are often described in terms of *state machines*, namely automata that encode the discrete behaviors of a peer in the system [44, 77]. In our model, we describe the state-space of a

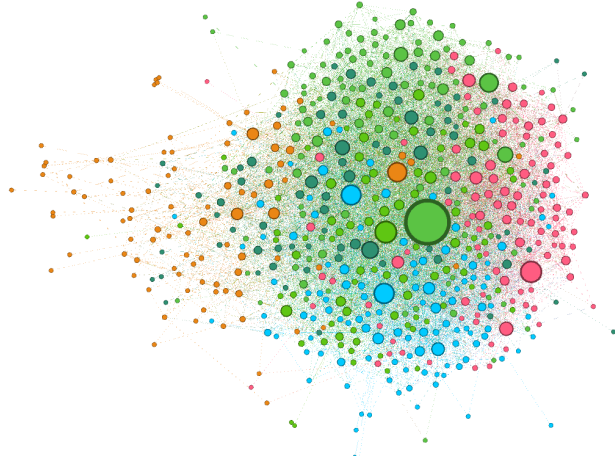


Figure 6.1.2: A community graph of Eth2.0 nodes where a bigger node implies greater degree. Similarly colored nodes have more edges among them than to the rest of the graph.

Gossipsub peer with a **peer-state** type, and then implement the Gossipsub state machine using a state transition function. We use a Defdata map from peers to their corresponding states to capture the state of an entire network.

```
(defdata group (map peer peer-state))
```

We use records whenever we need to store state because of the convenience of using named fields to access the internals of the state. While proving theorems referring to states, we noticed that we routinely had to prove helper lemmas about the types of the contents of those states. This motivated us to rewrite records in the ACL2S books to enable such helper lemmas automatically, leading to cleaner code.

The local state of a peer is modeled as a record using the following definition:

```
(defdata peer-state
  (record (nts      . nbr-topic-state)
          (mst      . msgs-state)
          (nbr-tctrs . pt-tctrs-map)
          (nbr-gctrs . p-gctrs-map)
          (nbr-scores . peer-rational-map)))
```

where (1) **nts** is a record that stores information about the peer's neighbors' subscriptions, the peer's mesh neighbors, and the peer's fanout (which we describe later); (2) **mst** is a record that stores the peer's messages and related state; (3) **nbr-tctrs** and **nbr-gctrs** are total maps that store counters used for computing a neighbor's topic-specific and general scores (respectively); and finally (4) **nbr-scores** is a total map from peers to their scores. Note that **peer** and **topic** are ACL2 symbols, while **tctrs** is a record that keeps track of a peer's behaviors in each topic:

```
(defdata tctrs
  (record (invalidMessageDeliveries . non-neg-rational)
          (meshMessageDeliveries   . non-neg-rational)
          (meshTime                 . non-neg-rational)))
```

```
(firstMessageDeliveries . non-neg-rational)
(meshFailurePenalty . non-neg-rational)))
```

Similarly, `gctrs` keeps track of a peer's general behaviors, not pertaining to any single topic:

```
(defdata gctrs
  (record (apco . rational) ;; application provided score
          (ipco . non-neg-rational) ;; ip-colocation factor
          (bhvo . non-neg-rational))) ;; track misbehavior
```

Notice that each of the counters is a rational, not a natural. This is because counters are supposed to fractionally decay at regular intervals. The rate of decay is dependent on the application running on top of Gossipsub. We define maps for each of these counters:

```
(defdata pt (cons peer topic))
(defdata pt-tctrs-map (map pt tctrs))
(defdata p-gctrs-map (map peer gctrs))
(defdata peer-rational-map (map peer rational))
```

For each map, we define a lookup function with default values. ACL2s automatically proves termination and input-output contracts, which suffices to show that the maps are total. As an example, the following is the lookup function for scores:

```
(defined lookup-score (p :peer prmap :peer-rational-map) :rational
  (let ((x (mget p prmap)))
    (match x
      (nil 0) ;; default value
      (& x))))
```

We explore each component of the `peer-state`.

**Neighbor topics state.** A key objective of the Gossipsub protocol is to reduce network congestion, while forwarding data as quickly as possible. To achieve this, a peer forwards full messages only to a subset of its neighboring peers. A Gossipsub peer keeps track of the topics its neighbors subscribe to, using the `nbr-topicsubs` field in `nbr-topic-state`. Using this information, it is able to build up a picture of the topics around it and which peers are subscribed to each topic. The peers to which it forwards full messages in topics it does not itself subscribe to constitute a list called a *fanout*, and is stored in the `topic-fanout` field, which is a map from topics to lists of peers (`lop`). The peer forwards full messages to other peers with whom it shares mesh membership. Mesh memberships are stored in the `topic-lop-map` field `topic-mesh`. Finally, the peer stores the last time it published a message to its fanout. This is used to expire a peer's fanout, if it has been too long since the peer last published.

```
(defdata peer symbol)
(defdata lop (listof peer))
(defdata topic-lop-map (map topic lop))
(defdata topic-nnr-map (map topic non-neg-rational))

(defdata nbr-topic-state
  (record (nbr-topicsubs . topic-lop-map)
```

```
(topic-fanout . topic-lop-map)
(last-pub     . topic-nnr-map)
(topic-mesh   . topic-lop-map)))
```

**Messages state.** `msgs-state` is a record type used to store information about messages received, requested, seen, or forwarded by a peer. First we define the types `pid-type`, which is just an alias for the type `symbol`, and the record type `payload-type`.

```
(defdata-alias pid-type symbol)
(defdata payload-type (record (content . symbol)
                              (pid     . pid-type)
                              (top     . topic)
                              (origin  . peer)))
```

`payload-type` is a record used to represent full messages, carrying the message content, payload id, the topic of this message and the peer who originated it. `pid-type` represents payload ids, a hash of a message payload which can identify the full message content. The `msgs-state` is defined as follows:

```
(defdata msg-peer (v (cons payload-type peer)
                    (cons pid-type peer)))
(defdata msgpeer-rat (map msg-peer rational))
(defdata msgs-waiting-for (map pid-type peer))
(defdata mcache (alistof payload-type peer))

(defdata msgs-state
  (record (recently-seen . msgpeer-rat)
         (pld-cache     . mcache)
         (hwindows      . lon)
         (waitingfor     . msgs-waiting-for)
         (served         . msgpeer-rat)
         (ihaves-received . nat)
         (ihaves-sent    . nat)))
```

`msgs-state` stores (1) `recently-seen`, a map from either a full-message or a message hash to the time since receipt; (2) `pld-cache`, an association list of full messages and their senders; (3) `hwindows`, or history windows, a list of naturals where each is the number of messages received in an interval; (4) `waitingfor`, a map from message ids of messages that haven't been received yet, to peers one has sent corresponding `IWANT` requests to; and (5) `served`, a map from either a full-message or a message hash to a rational, denoting the count of the number of times this message was served. The `served` map helps to detect peers sending too many `IWANT` messages. Finally, `msgs-state` contains (6) `ihaves-received` and `ihaves-sent`, which store the number of `IHAVE` messages received and sent, respectively.

**Events.** Our model includes events that can occur in a network. Events include peers sending or receiving (1) control messages like `GRAFT` or `PRUNE` for mesh control, `SUB`, `UNSUB`, `JOIN` and `LEAVE` for topics, or `CONNECT1` or `CONNECT2` messages to edit neighbors; (2) `PAYLOAD` for carrying message payload, or `IHAVE` for advertising or `IWANT` for requesting message payloads; and (3) `HBM` for heart-beat events occurring at each peer at regular intervals.

A list of events will have type `loev`. Events are defined as follows:

```
(defdata verb (enum '(SND RCV)))
(defdata rpc (v (list 'CONNECT1 lot)
                  (list 'CONNECT2 lot)
                  (list 'PRUNE topic)
                  (list 'GRAFT topic)
                  (list 'SUB topic)
                  (list 'UNSUB topic)))
(defdata data (v (list 'IHAVE lopic)
                  (list 'IWANT lopic)
                  (list 'PAYLOAD payload-type)))
(defdata mssg (v rpc data))
(defdata evnt (v (cons peer (cons verb (cons peer mssg)))
                  (list peer 'JOIN topic)
                  (list peer 'LEAVE topic)
                  (list peer verb peer 'CONNECT1 lot)
                  (list peer verb peer 'CONNECT2 lot)
                  (list peer 'HBM pos-rational)
                  (list peer 'APP payload-type)))
(defdata hbm-evnt (list peer 'HBM pos-rational))
(defdata-subtype hbm-evnt evnt)

(defdata loev (listof evnt))
```

Heart-beat events occur at regular intervals at each peer in a Gossipsub network. Several maintenance activities are performed during each such event. Scores are updated for neighboring peers, which are then used to update mesh memberships. Counters are multiplied by some application-specific decay factors. Neighboring peers that are not part of any mesh are sent **GRAFT** messages at regular intervals, provided that their addition could improve the average score of peers in the corresponding mesh. Several of these actions depend on application-specific weights (used for calculating scores) and parameters.

**Parameters and Scoring.** We store the weights and parameters relevant for scoring in a record called a **twp**. This record totally captures the application-specific configuration of a Gossipsub instance, *e.g.*, we can uniquely specify the configuration used by Eth2.0 in a **twp**. Thus, in order to simulate an application running on top of Gossipsub with our model, all we need to know is the application-specific **twp**. This is what we mean when we say our model is “pluggable”.

```
(defdata weights
  (record (w1 . non-neg-rational)
          (w2 . non-neg-rational)
          (w3 . non-pos-rational)
          (w3b . non-pos-rational)
          (w4 . neg-rational)
          (w5 . non-neg-rational)
          (w6 . neg-rational)
          (w7 . neg-rational)))
```

```

(defdata params
  (record (activationWindow . nat)
    (meshTimeQuantum . pos)
    (p2cap . nat)
    (timeQuantaInMeshCap . nat)
    (meshMessageDeliveriesCap . pos-rational)
    (meshMessageDeliveriesThreshold . pos-rational)
    (topiccap . rational)
    (grayListThreshold . rational)
    (d . nat)
    (dlow . nat)
    (dhigh . nat)
    (dlazy . nat)
    (hbmInterval . pos-rational)
    (fanoutTTL . pos-rational)
    (mcacheLen . pos)
    (mcacheGsp . non-neg-rational)
    (seenTTL . non-neg-rational)
    (opportunisticGraftThreshold . non-neg-rational)
    (topicWeight . non-neg-rational)
    (meshMessageDeliveriesDecay . frac)
    (firstMessageDeliveriesDecay . frac)
    (behaviourPenaltyDecay . frac)
    (meshFailurePenaltyDecay . frac)
    (invalidMessageDeliveriesDecay . frac)
    (decayToZero . frac)
    (decayInterval . pos-rational)))

(defdata wp (cons weights params))
(defdata twp (map topic wp))

```

Note that Gossipsub required weights  $w_3$  and  $w_{3b}$  to be negative. However, the use of Defdata allowed us to automatically find that FileCoin used an invalid  $\mathbf{twp}$  because it set  $w_3$  and  $w_{3b}$  to zero. We discussed this with the Gossipsub developers who then agreed to allow zero values. Given  $T$ , a set of topics which the neighbor subscribes to;  $\mathbf{tctrs}$ , the neighbor's topic specific counters;  $\mathbf{gctrs}$ , the neighbor's global counters; and a  $\mathbf{twp}$  containing entries for each topic our neighbor subscribes to, the score function calculates a neighbor's score as shown below.

$$score(q) = \min(TC, \sum_{\tau \in T} tw^\tau \times \sum_{i \in \{1,2,3,3b,4\}} w_i^\tau P_i^\tau) + w_5 P_5 + w_6 P_6 + w_7 P_7$$

where

```

(weights . params) = (mget  $\tau$  twp)
 $w_i^\tau$  = (weights- $w_i$  weights)
 $P_1^\tau$  = (calcP1(tctrs-meshTime tctrs) (params-meshTimeQuantum params))

```



```

        (params-timeQuantaInMeshCap params))
 $P_2^\tau = (\text{calcP2}(\text{tctrs-firstMessageDeliveries tctrs}) (\text{params-p2cap params}))$ 
 $P_3^\tau = (\text{calcP3}(\text{tctrs-meshTime tctrs}) (\text{params-activationWindow params})$ 
        (tctrs-meshMessageDeliveries tctrs)
        (params-meshMessageDeliveriesCap params)
        (params-meshMessageDeliveriesThreshold params))
 $P_{3b}^\tau = (\text{calcP3b}(\text{tctrs-meshTime tctrs}) (\text{params-activationWindow params})$ 
        (tctrs-meshFailurePenalty tctrs)
        (tctrs-meshMessageDeliveries tctrs)
        (params-meshMessageDeliveriesCap params)
        (params-meshMessageDeliveriesThreshold params))
 $P_4^\tau = (\text{calcP4}(\text{tctrs-invalidMessageDeliveries tctrs}))$ 
 $P_5 = (\text{gctrs-apco gctrs})$ 
 $P_6 = (\text{gctrs-ipco gctrs})$ 
 $P_7 = (\text{calcP7} (\text{gctrs-bhvo gctrs}))$ 
 $\text{tw}^\tau = (\text{params-topicweight params})$ 
 $\text{TC} = (\text{params-topiccap} (\text{cddar twp}))$ 

```

TC does not depend on any topic, but since it is stored in a `twp` which is indexed by `topic`, its value is replicated in each of the corresponding `params`. So, it is fine to extract its value from the first entry of a `twp`. Note that for score calculations, we require a non-empty `twp`.

Each of the `calcPi` functions where  $i \in \{1, 2, 3, 3b, 4, 7\}$  is used to calculate contributions to the score by one or more of counter values from `tctrs`. `calcP1` calculates the contribution to a neighbor's score based on the time spent in common meshes. `calcP2` awards score for being one of the first few to forward a message. `calcP3` calculates penalties due to the mesh message transmission rate being below a given threshold of `(params-meshMessageDeliveriesThreshold params)`. Whenever a peer is pruned, its corresponding `tctrs` counter `meshFailurePenalty` is augmented by the mesh message transmission rate deficit. This counter is not cleared even after the peer has been pruned. `calcP3b` scores mesh message delivery failures based on the value of this counter. Hence,  $P_{3b}^\tau$  is a “sticky” value which is supposed to discourage a peer that was pruned because of under-delivery from quickly getting re-grafted in a mesh. `calcP4` calculates the penalty on score due to sending invalid messages. `calcP7` calculates penalties due to several kinds of misbehaviors described by the Gossipsub specification. An example of such misbehaviors includes spamming with too many `IHAVE` messages which are either bogus and/or not following up to the corresponding `IWANT` requests.

**The Transition Function.** We define a transition function `run-network`, which, given an initial `Group` state and a list of `evnt`, produces a trace of type `egl`, which is an alist of `evnt` and `group`. Hence, after running a simulation, we have access to the state of the `Group` after each `evnt` was processed. `run-network` depends on the transition function for the `peer-state` (`transition`), which depends on the transition functions for `nbr-topic-state` (`update-nbr-topic-state`) and for `msgs-state` (`update-msgs-state`). For brevity, we mention only the signatures of each of these functions



below. Notice that each of these signatures represents neatly and concisely the types of the formal arguments and the function return type, which is very useful in a large code base.

```
(defdata egl (alistof evnt group)) ;; simulation trace

(definecd run-network (gr :group evnts :loev i :nat r :twp s :nat) :egl
...)

(defdata peer-state-ret
  (record (pst . peer-state)
    (evs . loev)))

(definecd transition
  (self :peer pstate :peer-state evnt :evnt r :twp s :nat) :peer-state-ret
...)

(defdata msgs-state-ret
  (record (mst . msgs-state)
    (evs . loev)
    (tcm . pt-tctrs-map)
    (gcm . p-gctrs-map)))

(definecd update-msgs-state (mst :msgs-state evnt :evnt pcm :pt-tctrs-map
  gcm :p-gctrs-map r :twp) :msgs-state-ret
...)

(defdata nbr-topic-state-ret
  (record (nts . nbr-topic-state)
    (evs . loev)
    (tcm . pt-tctrs-map)
    (gcm . p-gctrs-map)
    (sc . peer-rational-map)))

(definecd update-nbr-topic-state (nts :nbr-topic-state
  nbr-scores :peer-rational-map
  tcm :pt-tctrs-map gcm :p-gctrs-map
  evnt :evnt r :twp s :nat) :
  nbr-topic-state-ret
...)
```

A Gossipsub peer can select a random subset of its fanout and promote them as mesh members. It can also select a random subset of its neighbors to advertise with “IHAVE” messages. Such non-determinism is handled by sending a random seed *s* as a formal parameter to `run-network`, which is then propagated to the other transition functions it depends on. Observe that a single event like full message forwarding can trigger several more forwards, causing a cascade of events. Such events are represented by the `evs` field in the return types of `update-msgs-state` and `update-nbr-topic-state`. In order to limit the total number of events processed, we send a natural number *i* as a formal parameter to the `run-network`

function.

When proving contract theorems for the transition functions, we needed to prove the types of terms returned by utility functions, like `shuffle`, or ACL2 functions like `set-difference-equal`. For this, we used polymorphism and automated type-based reasoning provided by `Defdata`, as shown below:

```
(sig set-difference-equal ((listof :a) (listof :a)) => (listof :a))
(sig shuffle ((listof :a) nat) => (listof :a))
```

We made heavy use of higher-order macros written by Manolios [32] to improve the readability of our code. For example, `create-map*` is a list functor. Given an admitted function name or a lambda expression `f` of type  $a \rightarrow b$ , `create-map*` defines a function `map*-*f` of type  $(\text{listof } a) \rightarrow (\text{listof } b)$ . In the following code snippet, we show theorems proving that it obeys the functor laws, and give an example of its usage. `map*` is a syntactic sugar that maps function `f` onto a list without referring to the generated function name `map*-*f`.

```
(definec id (x :all) :all
  x)
;; Proof that create -map* is a list functor
;; 1) functor mapping preserves the identity function
(property functor-id (xs :tl)
  (= (map* id xs) ;; id is an identity function for lists
     (id xs)))

;; 2) functor mapping preserves function composition. f and g are declared
;; using defstub. gof is defined as a composition of g and f
(property functor-comp (xs :tl)
  (= (map* gof xs)
     (map* g (map* f xs))))

;; function to create a list of SND GRAFT events from peer p to a list of peers
(create-map* (lambda (tp p) '(,p SND ,(cdr tp) GRAFT ,(car tp)))
  lotopicpeerp
  loevp
  (:name mk-grafts)
  (:fixed-vars ((peerp p))))

(check= (map* mk-grafts '((FM . A) (DS . B)) 'P)
  '((P SND A GRAFT FM) (P SND B GRAFT DS)))
```

Given an admitted function name or a lambda expression `f` of type  $a \times b \rightarrow b$ , the higher order function `create-reduce*` defines a function `reduce*-*f` which accepts a list of elements of type `a`, an initial accumulator value of type `b`, and returns a reduction of the list using `f`, from left to right.

```
;; function to extract all the subscribers (neighboring peers) from a
  topic-lop-map
(create-reduce* (lambda (tp-ps tmp) (app tmp (cdr tp-ps)))
  lopp
  topic-lop-mapp)
```

```

(:name subscribers))

(check= (reduce* subscribers '()
  '((T1 P1 P2 P3)
    (T2 P4 P5 P1)))
  '(P4 P5 P1 P1 P2 P3))

```

## 6.2 Reasoning about the scoring function

Based on the observation that honest peers can be distinguished from malicious ones based on their observable behaviors (using local counters and scores), and thus, the overall network can be made more secure and performant if every honest peer promotes their well-behaving neighbors and demotes poorly-behaved ones, we came up with the following informal fundamental property.

**Fundamental Property of Gossipsub Defense Mechanisms.** *Peers who behave poorly will be demoted by their neighbors. Peers who behave better-than-average will be promoted by their neighbors. Promotion/demotion is entirely based on local peer behavior.*

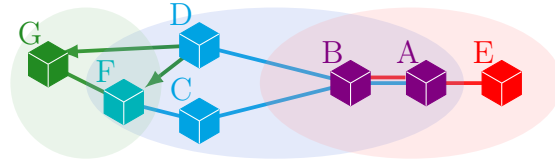


Figure 6.2.1: An example network

Before reasoning about the formalization of this fundamental property later in the chapter, we discuss the importance of this fundamental property. Consider the simple example shown in Figure 6.2.1, where peers A and B subscribe to, and are mesh neighbors in both Red and Blue topics. It might be possible for B to observe that A behaves perfectly well in the Blue topic while simultaneously misbehaving in the Red topic. This is not good for B because it depends on A for all of its messages in the Red topic. Note that this is a very simplified example. In an actual network, B could have several other neighbors subscribed to the Red topic. But as we will show later, it is equally trivial to have a scenario where all of B's neighbors isolate it from communications in the Red topic. In this example, we want B to prune A from its Red mesh in hopes of finding a better mesh neighbor later on. Reasoning about this fundamental property directly would be difficult due to the massive search-space of possible attack vectors. Hence, we focus on the following liveness property capturing the essence of the fundamental property:

**Property 6.2.1.** If a peer's score relating to its performance in any topic is continuously non-positive, then the peer's overall score should eventually be non-positive:

$$\forall q, \tau :: \langle G(\text{score}(q) \text{ for topic } \tau \leq 0) \Rightarrow F(\text{score}(q) \leq 0) \rangle$$

where  $\text{score}(q)$  for topic  $t$  is defined below.

$$\text{tw}^\tau \times \sum_{i \in \{1,2,3,3b,4\}} w_i^\tau P_i^\tau$$

Notice that Property 6.2.1 is temporal. We write the non-temporal version of this property in context of Eth2.0 (using Eth2.0 **twp**) in ACL2s as shown below, and disprove the temporal version using an induction argument later in the dissertation.

### Property 6.2.2.

```
(property (ptc :pt-tctrs-map pcm :p-gctrs-map p :peer top :topic)
  :hyps (^ (member-equal '(,p . ,top) (acl2::alist-keys ptc))
    (> (lookup-score p (calc-nbr-scores-map ptc pcm *eth-twp*)) 0))
  (> (calcScoreTopic (lookup-tctrs p top ptc) (mget top *eth-twp*)) 0))
```

The following is one of the counter-examples to the above property, generated by cgen in ACL2s:

```
((top 'agg)
 (p 'p4)
 (pcm '((p3449 (:0tag . gctrs) (:apco . 0) (:bhvo . 0) (:ipco . 0))
  (p3450 (:0tag . gctrs) (:apco . 0) (:bhvo . 0) (:ipco . 0))
  (p3451 (:0tag . gctrs) (:apco . 0) (:bhvo . 0) (:ipco . 0))))
 (ptc '(((p4 . agg)
  (:0tag . tctrs)
  (:firstmessagedeliveries . 0)
  (:invalidmessagedeliveries . 0)
  (:meshfailurepenalty . 0)
  (:meshmessagedeliveries . 1)
  (:meshtime . 42))
 ((p4 . blocks)
  (:0tag . tctrs)
  (:firstmessagedeliveries . 324)
  (:invalidmessagedeliveries . 0)
  (:meshfailurepenalty . 0)
  (:meshmessagedeliveries . 330)
  (:meshtime . 377))
 ((p4 . sub1)
  (:0tag . tctrs)
  (:firstmessagedeliveries . 371)
  (:invalidmessagedeliveries . 0)
  (:meshfailurepenalty . 0)
  (:meshmessagedeliveries . 377)
  (:meshtime . 324))
 ((p4 . sub2)
  (:0tag . tctrs)
  (:firstmessagedeliveries . 318)
  (:invalidmessagedeliveries . 0))
```

```

(:meshfailurepenalty . 0)
(:meshmessagedeliveries . 324)
(:meshtime . 371))
... ))

```

For brevity, we omit entries for peer-topic key values for peers other than `p4`. In property-based testing, the free variables of a property under test are assigned values using a synergistic combination of theorem proving and random assignments computed using type-based enumerators (generators) in an effort to discover counterexamples to the property. Observe that Property 6.2.2 depends on `ptc` and `pcm` which do not have trivial types. These are maps containing records which themselves consist of several numerical values, which makes the search space of possible counter-examples immensely large. In order to make it easier for `cgen` to find counter-examples, we wrote custom enumerators to enumerate restricted values for `topic`, `tctrs`, `gctrs`, `pt-tctrs-map` and `p-gctrs-map`. Specifically, we limit the penalties on the score due to some counters, such that the negative contributions due to misbehavior are comparable to the positive contributions due to good behavior. Below we define custom enumerators for `topic` and `tctrs`.

```

(definec topics () :tl
  ;; valid topics used in Ethereum
  '(AGG BLOCKS SUB1 SUB2 SUB3))

(definec nth-topic-custom (n :nat) :symbol
  (nth (mod n (len (topics))) (topics)))
(defdata lows (range integer (0 <= _ <= 1))) ;; high values
(defdata-subtype lows nat)
(defdata highs (range integer (300 < _ <= 400))) ;; low values
(defdata-subtype highs nat)

(defun nth-bad-counters-custom (n)
  ;; setting invalidMessageDeliveries and meshFailurePenalty to 0 due to high
  penalty
  (defun nth-good-counters-custom (n)
    (tctrs 0 (nth-highs (+ n 2)) (nth-highs (+ n 3)) (nth-highs (+ n 4)) 0))

  (defun nth-counters-custom (n) ;; custom enumerator for tctrs
    (if (== 0 (mod n 4))
        (nth-bad-counters-custom n)
        (nth-good-counters-custom n)))

```

Besides Property 6.2.2, we formalize three safety properties for GossipSub, stated below. Together, these four are the most general properties of the score function, which must hold in order for the fundamental property to hold.

**Property 6.2.3.** Increasing bad-performance counters (which are multiplied with negative weights) should decrease the overall score.

**Property 6.2.4.** Increasing good-performance counters (which are multiplied with positive

weights) will not decrease the score for a mesh peer that has been in the mesh for a sufficiently long time.

**Property 6.2.5.** If two peers subscribe to the same topics, and achieve identical per-topic counters, and identical global counters, then they achieve identical scores.

We are able to find counterexamples to Property 6.2.3 in much the same way as 6.2.2, as shown in Table 6.1.

| Topic  | MT  | FMD | FMD' | MMD | MMD' | IMD | MFP | Score | Score' |
|--------|-----|-----|------|-----|------|-----|-----|-------|--------|
| Blocks | 147 | 194 | 3    | 200 | 10   | 0   | 0   | 22.21 | 6.21   |
| Agg    | 150 | 194 | 194  | 230 | 230  | 0   | 0   | 13.83 | 13.83  |
| Sub1   | 141 | 188 | 188  | 194 | 194  | 0   | 0   | 7.80  | 7.80   |
| Sub2   | 110 | 180 | 180  | 232 | 232  | 0   | 0   | 7.80  | 7.80   |
| Sub3   | 135 | 182 | 182  | 188 | 188  | 0   | 0   | 7.78  | 7.78   |
| Total  |     |     |      |     |      |     |     | 32.72 | 32.72  |

Table 6.1: Perturbations in **topic-counters** values for an Eth2.0 peer that violate Property 2. Abbreviations denote score components: *FMD* (FirstMessageDeliveries), *FMD'* (FirstMessageDeliveries penalty), *MMD* (MeshMessageDeliveries), *MMD'* (MeshMessageDeliveries penalty), *IMD* (InvalidMessageDeliveries), *MFP* (MeshFailurePenalty), and *MT* (MeshTime). Both totals = 32.72, even though the count of FirstMessageDeliveries and MeshMessageDeliveries drops.

We manually prove that Property 6.2.4 holds over all configurations (available with the artifacts). The proof that Property 6.2.5 holds over all configurations follows directly from referential transparency of ACL2S.

## 6.3 Discussion: Relation to Floodsub and Meshsub Correctness Arguments

Property 6.2.2 formalizes a fundamental consistency expectation between local topic behavior and global peer reputation in Gossipsub v1.1. Informally, it states that a peer assigned a strictly positive neighbor score should also have strictly positive topic-specific score contributions. In other words, a peer’s global reputation should not be supported by topic contexts in which its behavior is locally penalized.

This expectation aligns with the correctness intuitions underlying Floodsub and Meshsub. Although those protocols do not maintain explicit peer scoring, their refinement-based proofs establish that peers which publish and forward messages according to the protocol rules preserve the safety and liveness properties of the dissemination overlay. Implicitly, these arguments assume behavioral coherence: peers that contribute positively to dissemination do not simultaneously violate topic-level assumptions.

From this perspective, Property 6.2.2 expresses a compatibility requirement between Gossipsub’s scoring layer and the Commitment⇒Delivery property already established for

Floodsub and Meshsub. The scoring mechanism is intended to ensure that peers exhibiting persistently poor topic-specific behavior eventually receive a non-positive overall score, thereby limiting their influence on mesh formation and message propagation.

Our counterexamples demonstrate that this property does not hold in Gossipsub v1.1. Peers with persistently non-positive topic scores may retain positive overall standing and remain in the dissemination mesh. This breakdown in the scoring mechanism directly undermines the commitment-implies-delivery guarantee: there exist executions which lead to peer states, we found in our counter-examples, in which a message is committed at its origin, yet some peers subscribed at commit time never receive it. Thus, the failure of the score property provides a structural explanation for the violation of dissemination correctness. In the next section, we will synthesize such executions, as well as attacks that exploit vulnerabilities exposed by the failure of Gossipsub’s scoring system.





## Part III

# Effectiveness of the Combined Methodology



## Chapter 7

# Model based Security Analysis of Gossipsub v1.1

Protocols intended for deployment in large networks must provide not only scalability and performance, but also strong resilience against adversarial behavior. As such systems grow in complexity, informal reasoning and testing alone are insufficient to establish confidence in their security, motivating the need for systematic verification techniques that can reason about all possible protocol executions. This need is particularly acute for Gossipsub, which was explicitly developed as a security-enhanced pubsub protocol designed to mitigate a wide range of known attacks.

The Gossipsub developers specified their protocol in English prose and reference implementations [49, 73, 82], and validated its security primarily through unit testing and network emulation of pre-programmed scenarios [51, 83]. These evaluations are intended to demonstrate that misbehaving peers in a Gossipsub network are eventually identified and pruned. However, as Dijkstra famously observed, “Program testing can be a very effective way to show the presence of bugs, but it is hopelessly inadequate for showing their absence.” This motivated us to develop our own model of Gossipsub in ACL2S (as described in the previous chapter) that allows us to write and reason about properties about the protocol and its components, as well as to run simulations with precise control over network dynamics and states.

We synthesize and validate a collection of attacks on actual Eth2.0 topologies. Our attacks range from simple attacks with a single attacker and victim, to eclipse attacks, to attacks that partition Eth2.0 networks. All of the attacks are based on our attack gadgets, which encapsulate the essence of the vulnerabilities we discovered. Our attack gadgets can be applied to any network topology and allow an attacker to block or throttle certain message transmissions, without being discovered and without incurring any penalties. In fact, we show that for all of these attacks the scores assigned to the attackers by the victims stabilize; hence, by induction, they remain positive forever.

The counter-example 6.2 which we obtained in the previous section is a specification for an unsafe state that does not satisfy Property 6.2.2 for an Eth2.0 network. However, we are also interested in characterizing and generating attacks against an Eth2.0 Gossipsub network for three main reasons: (1) to show that an unsafe state is **reachable** from a reasonable start state, (2) **invalidation** of our temporal Property 6.2.1 using the trace generated from

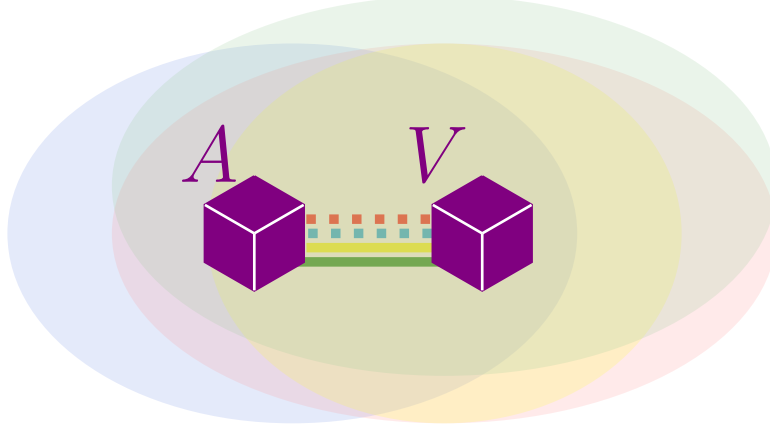


Figure 7.1.1: Example of an  $AG_2$  attack gadget, where out of four common mesh topics,  $A$  is attacking  $V$  in the red and blue (dashed line) topics.

the attack, and finally (3) demonstration of **scalability** of our attack to large networks, typically of the size and shape used by real world applications.

The rest of this chapter is organized as follows. Section 7.1 defines attack gadgets and shows how to use them to create havoc in Gossipsub v1.1 networks. Section 7.2 describes real-world Eth2.0 topologies used in our attack simulation. Section 7.3 details the experimental setup, while Section 7.4 interprets these attacks as refinement counterexamples—valid execution sequences that exploit Gossipsub v1.1’s scoring mechanism to reach states unreachable in Floodsub and Meshsub.

**Chapter provenance.** This chapter is based on work previously published in [41, 43]. The presentation has been revised and extended for inclusion in this dissertation.

## 7.1 Attack Gadgets

An attack gadget is a tuple  $\langle A, V, S \rangle$ , where  $A, V$  are peers ( $A$  is the attacker and  $V$  is the victim),  $S$  is a set of subnet topics (the attacked topics), and  $A, V$  are mesh neighbors over a set of topics that is a superset of  $S$ . For each  $i \in \mathbb{N}$ , we define  $AG_i$  to be the set of attack gadgets where  $|S| = i$ . Figure 7.1.1 illustrates an example attack gadget in  $AG_2$ . The attack gadget allows  $A$  to maintain an overall positive score while misbehaving with respect to  $S$ , by behaving honestly with respect to the other topics. Therefore, if  $T$  is the set of subnet topics in the given Eth2.0 network, an attack gadget  $\langle A, V, S \rangle$  in  $AG_i$  is only possible if  $|T \setminus S|$  is large enough. Using the Eth2.0  $GS$  parameters, we can calculate the number of other topics  $A$  and  $V$  have to subscribe to as follows.

$$\min\{t \in \mathbb{N} \mid (7.2 + 3.2 \frac{t}{T} > 24.7 \frac{i}{T}) \wedge (t + i \leq T)\} \quad (7.1)$$

The above only has a solution for some values of  $T$ . Also, note that to derive a corresponding formula for a different  $GS$  application, one has to use that application’s parameters.

## 7.2 Eth2.0 Topologies

We validate our attacks using the full-network topologies of the Eth2.0 testnets Ropsten, Goerli and Rinkeby, as measured by Li et. al. [47] Table 7.1 shows basic characteristics of these Eth2.0 network topologies.

| Network | Nodes | Degree |     |       | Diameter |
|---------|-------|--------|-----|-------|----------|
|         |       | min    | max | avg   |          |
| Ropsten | 588   | 1      | 418 | 25.49 | 5        |
| Goerli  | 1355  | 1      | 712 | 28.26 | 5        |
| Rinkeby | 446   | 1      | 191 | 68.96 | 6        |

Table 7.1: Eth2.0 Network Characteristics

Figure 7.2.1 shows a visualization of the Ropsten network topology, where each node is represented by a circle whose diameter is proportional to its degree. Similarly colored nodes are in the same community *i.e.*, share more edges among themselves than with the rest of the network.

## 7.3 Experimental Setup

Our experiments use the above topologies as follows. Given a topology and the number of subnet topics,  $T$ , we construct a corresponding ACL2S model (of type **Group**), with topics **BLOCKS**, **AGG** and  $T$  subnet topics, for  $n = T + 2$  topics in total. Furthermore, every peer is a mesh member of every topic. We then generate attacks, as outlined below, and use ACL2S to check that the attacks are successful, *i.e.*, that the attackers *continuously* limit messages of the targeted topics without *ever* being discovered or penalized by any of the victims.

**Single attacker/victim attacks** We synthesize attacks involving a single attacker and a single victim by instantiating a single attack gadget. For each of the network topologies and for  $i \in \{1, 2, 3\}$ , where  $i$  corresponds to the number of attacked topics, we generate a corresponding ACL2S model. In the model, we determine the value of  $T$ , the number of subnet topics, using Equation 7.1. Once we have the ACL2S model, we generate a sequence of events consisting of message transmission events from  $A$  to  $V$  as well as heartbeat events at  $V$  (heartbeats are when  $V$  updates the scores of its peers). The shape of the events we generate is described by the following regular expressions:

$$\begin{aligned}
 Msgs &:= M_1^b \dots M_i^b M_{i+1}^f \dots M_n^f \\
 Events &:= (Msgs H)^+
 \end{aligned}$$

where (1)  $M_k^l$  denotes sending and receiving  $l$  payload messages from  $A$  to  $V$  for topic  $k$ , (2)  $b \in \{0, 1\}$ , as discussed below, (3)  $i$  is the number of attacked topics, as mentioned above,

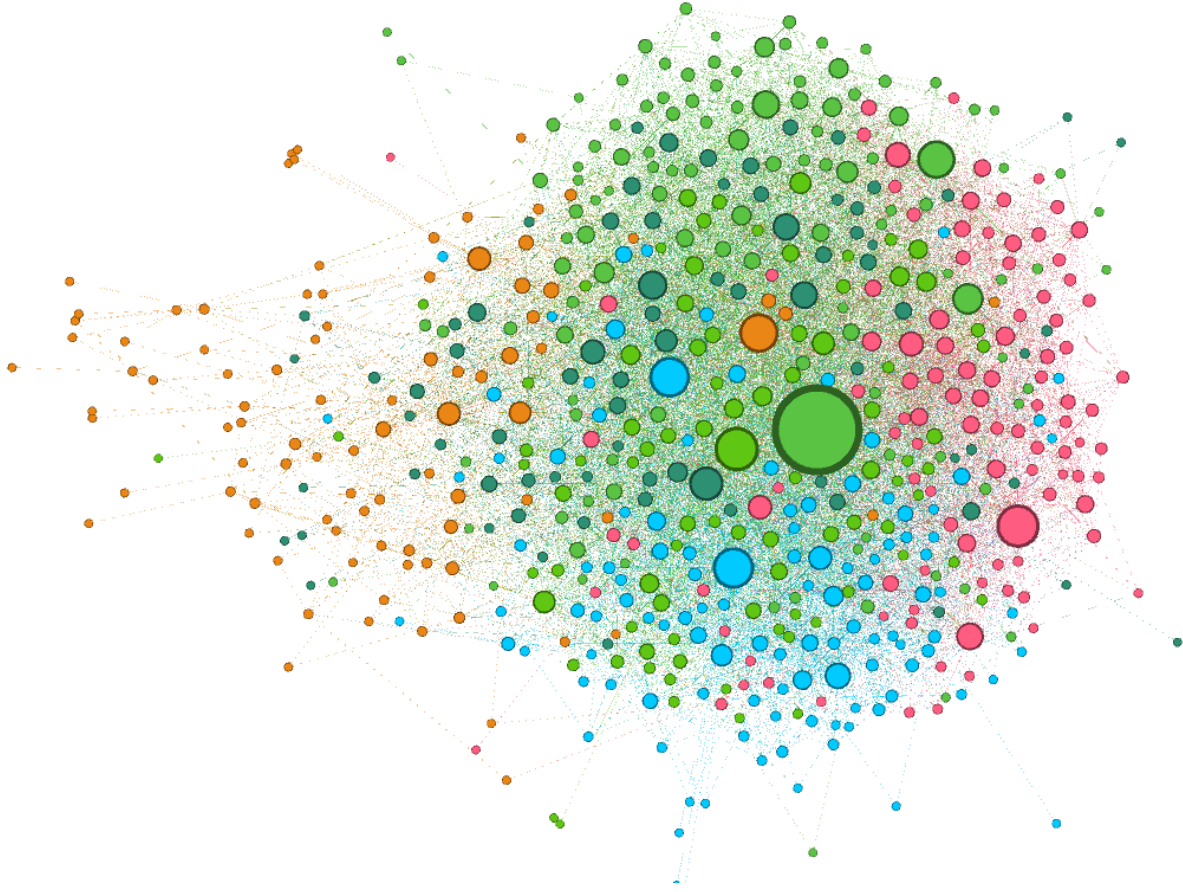


Figure 7.2.1: Visualization of the Ropsten topology of Eth2.0 nodes illustrating node degree distribution and community structure.

(4)  $f$  is the number of messages sent for the topics which are not attacked, as justified below, (5)  $n = T + 2$  is the total number of topics, as described above, and (6)  $H$  is a heartbeat event at  $V$ .

The ordering of events is not important because any permutation of  $Msgs$  occurring between  $V$ 's heartbeats will have the same effect on the network state. Also, we set  $f=10$  for each topic because according to Eth2.0  $GS$  parameters, under normal operation, this is at least 10% of the expected mesh message deliveries per topic and sending more than  $f$  messages can never decrease the score assigned to an attacker by the victim.

We synthesize two types of attacks. The first are *throttling attacks*, which reduce the mesh message transmission rate in the attacked topics to be below the threshold set by the Eth2.0 parameters by setting  $b=1$ . The second are *blocking attacks*, which completely block mesh message transmission for all of the attacked topics by setting  $b=0$ . We validate the attacks by checking that Property 1 is eventually always violated by the output trace of our experiments.

To test our model's performance on actual topologies, we ran our experiments on 100,000 events. Note that processing one event typically generates a cascade of other events because when  $V$  receives a message, it forwards it to its peers, who then forward it to their peers and

so on. Table 7.2 shows the time required to simulate and validate our attacks in ACL2s, on a 16GB M1 Macbook Air machine.

| Network | Throttling      | Blocking        |                 |                 |
|---------|-----------------|-----------------|-----------------|-----------------|
|         | AG <sub>1</sub> | AG <sub>1</sub> | AG <sub>2</sub> | AG <sub>3</sub> |
| Ropsten | 1.3             | 1.3             | 1.3             | 1.3             |
| Goerli  | 1.3             | 1.4             | 1.8             | 1.1             |
| Rinkeby | 1.6             | 1.9             | 2.0             | 2.1             |

Table 7.2: Wallclock time (mins) for attack simulation

We observed that for all experiments, the first violation of Property 1 is observed at event  $H$  occurring right after the activation period (an Eth2.0 parameter) has passed. Hence, an attacker peer can start its attack quickly after it joins a mesh. Experimentally, we observed that this attack is not transient as attack scores (assigned by victims) eventually converge to a positive number that stays the same in successive heartbeats at  $V$ . By induction, this establishes that our attacks are perpetual. The time taken by our models to process 100,000 events gives a sense of the performance one can expect for other kinds of simulations. Simulation times for the remaining attacks are similar and for brevity, we only simulate these attacks until stability which is achieved, which never takes more than 5 seconds.

### 7.3.1 Eclipse Attacks

We can use attack gadgets to construct other eclipse attacks by just instantiating an attack gadget per neighbor of a victim such that if they collude, they can target and completely isolate the victim *i.e.*, the victim will never receive any messages in the  $i$  attacked topics. We tested this attack in the Ropsten topology by identifying a victim node with four neighboring peers (about 12% nodes have degree less than 5; see Figure 7.2.1) and instantiating its neighbors as attackers using AG<sub>3</sub> gadgets. We verified that the victim’s message cache contained no message received in any of the attacked topics while containing messages received in non-attacked topics, that the attackers were continuously assigned positive scores by the victim, and that this behavior was perpetual.

### 7.3.2 Partition Attacks

Attack gadgets can also be used to isolate a subset of the peers in a network by carrying out a network partition attack, which we describe as follows. Given a network graph  $G = \langle V, E \rangle$  and a set of victims  $S$ , we want to identify a set  $X$ , preferably of minimal cardinality, such that  $X$  is a *vertex cut* of  $G$  that partitions  $G$  into disconnected components  $\{S, V \setminus \{S \cup X\}\}$ . The elements of  $X$  are the misbehaving peers, *i.e.*, each peer in  $X$  attacks all of its non- $X$  neighbors, using our attack gadgets to throttle or block messages over the attacked

topics. Hence, no message in an attacked topic can be sent to a peer in  $S$  from a peer outside of the partition and vice versa. In addition to targeting specific victims, interesting partition attacks include setting  $S$  to be a community of the network (since partitions along community lines are expected to require smaller vertex cuts than are required for other sets of peers; see Figure 7.2.1) and choosing  $S$  such that  $|S|$  is  $\approx \frac{|V|}{2}$ , which will bisect the graph. Finding minimal vertex cuts is an NP-hard problem [6], and can be reduced either to a Pseudo-Boolean problem or to 0-1 Integer Linear Programming problem. We synthesized and evaluated partition attacks using the Ropsten topology by selecting a set of victims,  $S$ , where  $|S| = 6$ , finding a minimal vertex cut,  $X$  (in our case  $|X| = 2$ ) and creating the appropriate attack gadgets. We verified that each of the victim’s message caches contained no message in any of the attacked topics that originated outside of  $S$ , but did contain messages in non-attacked topics received from outside of  $S$ , that victim nodes continuously assigned positive scores to their attackers (the nodes in  $X$ ) and that this behavior was perpetual, *i.e.*, maintained forever.

## 7.4 Discussion: Attacks as Refinement Counterexamples

The attacks described above can be understood as concrete witnesses to a breakdown in behavioral assumptions that are implicitly relied upon by earlier publish–subscribe protocols. Although these attacks are constructed against Gossipsub v1.1, they exploit executions that would be unreachable under the correctness arguments developed for Floodsub and Meshsub.

Refinement-style reasoning for Floodsub and Meshsub establishes that well-behaving peers preserve the dissemination guarantees of the overlay under all admissible executions. Since these protocols do not maintain adaptive reputation or scoring state, they cannot reach configurations in which a peer is structurally favored while simultaneously degrading topic-level dissemination. As a result, the classes of executions exploited by our attacks are ruled out by construction.

Gossipsub v1.1, by contrast, introduces peer scoring as an additional layer of state that directly influences mesh maintenance decisions. This additional flexibility admits executions in which mesh membership becomes decoupled from topic-level behavior. The attacks exploit this decoupling, driving the protocol into states that violate the behavioral invariants implicitly preserved by Floodsub and Meshsub.

Viewed through this lens, the attacks function as refinement counterexamples. They arise not from malformed inputs or undefined behavior, but from sequences of locally valid protocol transitions. Property-based testing enables the systematic discovery of such executions without requiring the construction of an explicit refinement proof, providing a lightweight means of identifying refinement-level failures in security-augmented protocol designs.



## Chapter 8

# Lean Gossip - Simulation-Based Optimization of Gossipsub

The formal refinement analysis of Meshsub identifies the mesh degree  $D$  and the gossip fanout parameter  $D_{\text{lazy}}$  as performance-critical parameters governing the trade-off between dissemination latency, bandwidth consumption, and robustness under churn. In particular, the refinement proof establishes that these parameters directly bound the number of gossip rounds required to simulate abstract Floodsub forwarding with high probability, and hence control the extent of probabilistic stuttering in the concrete execution. While the proof provides asymptotic guarantees and high-probability bounds, it does not prescribe concrete parameter values for realistic network sizes and workloads. In this chapter, we complement the formal analysis with large-scale simulation to explore the quantitative impact of  $D$  and  $D_{\text{lazy}}$  on Gossipsub performance, using simulation-based optimization to identify parameter regimes that achieve favorable trade-offs between delivery rate, bandwidth cost, and latency.

This chapter introduces and evaluates what we call *Lean Gossip*: a configuration regime for gossip-based dissemination protocols in which recovery via gossip is deliberately kept minimal. Concretely, Lean Gossip refers to Gossipsub configurations with sparse eager meshes ( $D$  small) and a minimal gossip degree ( $D_{\text{lazy}} = 1$ ), relying on gossip as a lightweight recovery mechanism rather than a primary dissemination channel. As we show, this minimal use of gossip is sufficient to compensate for churn-induced message loss while avoiding the bandwidth explosion associated with aggressive gossiping or redundant eager-push flooding.

While these parameters are known to influence typical performance, our results show that Gossipsub with minimal gossip ( $D_{\text{lazy}}=1$ ) dominates Floodsub on delivery rate and bandwidth cost across all churn levels, achieving comparable reliability at 29–35 $\times$  lower bandwidth cost. This provides the first principled guidance for parameter selection: sparse mesh connectivity with gossip-based recovery outperforms redundant eager-push connections, and Ethereum’s deployed mesh degree lies near the Pareto frontier of delivery–cost tradeoffs.

We make the following contributions:

**First systematic parameter study of Gossipsub under churn.** We present the first comprehensive simulation-based evaluation of Gossipsub parameterization across mesh degree  $D$  and gossip degree  $D_{\text{lazy}}$  under five churn regimes (0–40%) in a 1000-node network.

Our simulator implements Gossipsub v1.0 semantics and is validated against published Protocol Labs measurements. We release our simulation framework and analysis scripts for reproducibility [39].

**Floodsub is strictly dominated.** We show that simple flooding (Floodsub) is strictly dominated by Gossipsub in the delivery–bandwidth tradeoff across 0–40% churn. For every churn level, there exist Gossipsub configurations that achieve higher delivery at substantially lower bandwidth cost. Floodsub’s only advantage is lower latency.

**Lean Gossip strictly dominates Ethereum’s deployed configuration.** We identify *Lean Gossip* configurations ( $D_{\text{lazy}}=1$ ) that lie on the Pareto frontier and strictly dominate Ethereum’s deployed Gossipsub parameters in the delivery–bandwidth tradeoff under worst-case performance aggregated over 0–40% churn.

**Recovery dominates redundancy.** We demonstrate that gossip-based recovery—not eager-push redundancy—is the dominant mechanism for reliable dissemination under churn. Increasing  $D$  beyond approximately 8 yields diminishing delivery returns, while increasing  $D_{\text{lazy}}$  beyond 1 sharply increases bandwidth cost with negligible improvement.

**Pareto dominance framework for protocol evaluation.** We introduce a two-dimensional Pareto dominance analysis over worst-case delivery and normalized bandwidth cost, enabling principled, multi-objective comparison of dissemination strategies.

The formal model developed in Chapter 5 establishes that Meshsub correctly implements Broadcastsub under the stated correctness conditions. However, that model intentionally abstracts away timing, network latency, bandwidth consumption, and realistic churn patterns. The simulation study in this chapter complements the formal analysis by quantifying these performance characteristics under realistic network conditions.

The rest of this chapter is organized as follows. Section 8.1 describes our simulation based methodology. Section 8.2 details our implementation of Gossipsub v1.0 used for simulation analysis. Section 8.3 presents the simulation results, and Section 8.4 discusses their implications. Section 8.5 discusses limitations of our work and scope for future work. Section 8.6 discusses related work.

**Chapter provenance.** A version of this chapter is currently under review.

## 8.1 Methodology

This section describes the simulation framework used to evaluate Gossipsub dissemination as a function of mesh degree  $D$  and gossip degree  $D_{\text{lazy}}$  under churn. We isolate protocol-level delivery–bandwidth–latency tradeoffs by modeling a single-topic overlay and excluding scoring and multi-topic interactions.

### 8.1.1 Gossipsub Overview

Gossipsub combines eager push via a maintained mesh overlay with lazy pull via metadata gossip. Each peer maintains a neighbor set  $p.nbrs$  and, for a subscribed topic  $t$ , an eager mesh  $p.\mathcal{M}(t)$  of size approximately  $D$  to which full payloads are forwarded. Periodically, peers gossip message identifiers to up to  $D_{lazy}$  non-mesh neighbors, enabling recovery via IHAVE/IWANT exchanges. Heartbeat maintenance enforces mesh degree bounds and emits gossip.

For a subscribed topic  $s \in p.\mathcal{S}$ , peer  $p$  maintains a *mesh*  $p.\mathcal{M}(s) \subseteq \{q \in p.nbrs : s \in q.\mathcal{S}\}$ , a subset of  $s$ -subscribing neighbors to which  $p$  eagerly forwards full message payloads upon first receipt. The mesh degree is bounded:  $D_{lo} \leq |p.\mathcal{M}(s)| \leq D_{hi}$ , where  $D$  is the target degree and by default  $D_{lo} = D - 2$ ,  $D_{hi} = D + 2$ . Likewise, for an unsubscribed topic  $u \in p.\mathcal{U}$ , peer  $p$  maintains a *fanout*  $p.\mathcal{F}(u) \subseteq \{q \in p.nbrs : u \in q.\mathcal{S}\}$ , enabling message origination on topics without subscription. Meshes, fanouts, and subscriptions are all mutable: a peer may unsubscribe from a topic, delete its corresponding mesh, and build a fanout; or vice versa.

**Metadata dissemination.** Each peer maintains a bounded message cache  $p.mcache$  mapping recent message identifiers to payloads. Metadata about recently received messages are periodically broadcast to up to  $D_{lazy}$  randomly selected neighbors outside the mesh, called *gossip peers*. These IHAVE announcements contain message identifiers from  $p.mcache$ , allowing metadata to disseminate quickly with low overhead. Upon receiving IHAVE( $M$ ) from peer  $q$ , a peer  $p$  identifies missing messages  $M' = M \setminus p.seen$  and responds with IWANT( $M'$ ), after which  $q$  transmits the requested payloads. This lazy pull mechanism enables recovery of messages missed due to mesh disconnection.

**Heartbeat maintenance.** Gossipsub executes a periodic heartbeat procedure at configurable intervals (default 1 s) to maintain mesh structure and disseminate metadata. For each topic  $t \in p.\mathcal{S}$ , the heartbeat performs the following operations in sequence:

**Mesh maintenance.** Enforce degree bounds by grafts and prunes.

$$\begin{aligned} |p.\mathcal{M}(t)| < D_{lo} &\Rightarrow \text{select } k = D - |p.\mathcal{M}(t)| \text{ peers from} \\ &\quad p.nbrs \setminus p.\mathcal{M}(t), \text{ send GRAFT to each, add to } p.\mathcal{M}(t) \\ |p.\mathcal{M}(t)| > D_{hi} &\Rightarrow \text{select } k = |p.\mathcal{M}(t)| - D \text{ peers from} \\ &\quad p.\mathcal{M}(t), \text{ send PRUNE to each, remove from } p.\mathcal{M}(t) \end{aligned}$$

**Fanout maintenance.** For each topic  $u \in p.\mathcal{U}$  with non-empty fanout, remove peers that have unsubscribed and replenish if  $|p.\mathcal{F}(u)| < D$ . Expire fanouts inactive beyond a configurable TTL.

**Gossip emission.** Select up to  $D_{lazy}$  peers from  $p.nbrs \setminus p.\mathcal{M}(t)$  and emit IHAVE announcements containing message identifiers from  $p.mcache$ .

**Cache shift.** Advance the message cache window, expiring entries older than the gossip history length (default 3 heartbeat intervals). This bounds memory usage while retaining sufficient history for IWANT responses.

### 8.1.2 Model Simplifications

To isolate dissemination dynamics, we apply the following simplifications:

**Single topic.** All peers subscribe to one topic ( $|\mathcal{T}| = 1$ ), eliminating fanout interactions and cross-topic effects.

**No peer scoring.** We omit v1.1 scoring, as it addresses adversarial behavior rather than benign churn.

**Static neighbor sets.** Peer tables are initialized once and change only due to churn.

**No Floodsub compatibility.** We model pure Gossipsub semantics.

Under these assumptions, peer state reduces to mesh neighbors, gossip peers, pending queue, seen set, bounded message cache, availability state, and delivery record. The parameters  $D$  and  $D_{lazy}$  fully determine redundancy.

**Simplified peer state.** Under these simplifications, peer state reduces to a septuple. For the single topic  $t \in \mathcal{T}$ , we write:  $p.\mathcal{M} = p.\mathcal{M}(t)$ : the eager mesh neighbors,  $p.\mathcal{L} \subseteq p.nbrs \setminus p.\mathcal{M}$ : the gossip peers selected for IHave emission,  $p.pending$ : the queue of inbound messages yet to be processed,  $p.seen$ : the set of received message identifiers (cleared on rejoin),  $p.mcache$ : the bounded message cache,  $p.\omega \in \{\text{online}, \text{offline}\}$ : the availability state,  $p.delivered$ : a persistent record of all messages received by peer  $p$ , which survives peer rejoin events.

The parameters  $D$  (target mesh degree) and  $D_{lazy}$  (maximum gossip degree) fully determine the protocol’s redundancy–efficiency tradeoff in this simplified model. Note that  $p.seen$  is used for duplicate detection during protocol execution and is cleared when a peer rejoins, while  $p.delivered$  is used solely for delivery measurement and persists across churn events to ensure accurate delivery rate statistics.

These simplifications preserve the core dissemination dynamics: eager mesh forwarding provides primary delivery, while lazy pull serves as a recovery mechanism for messages missed due to mesh disconnection. Our single-topic model essentially captures the behavior of either a mesh overlay (when peers subscribe) or a fanout overlay (when peers originate messages without subscription) in a full multi-topic Gossipsub network. Since both structures share identical eager-push semantics and gossip recovery mechanisms, our model isolates the fundamental dissemination dynamics common to all Gossipsub overlays.

We implement Gossipsub v1.0 semantics, omitting the peer scoring mechanism introduced in v1.1 [50]. Peer scoring is primarily for building attack resilience by penalizing misbehaving peers and is orthogonal to dissemination dynamics. This simplification isolates the core delivery–efficiency tradeoffs arising from mesh structure and gossip parameters.

## 8.2 Protocol Implementation

Having formally described the protocol in the previous section, we now explain key design decisions, evaluation metrics, and protocol parameter values used in our simulation.

### 8.2.1 Aggregate Message Workload

Ethereum’s Gossipsub layer disseminates a heterogeneous mix of beacon blocks, attestations, aggregated attestations, and protocol control messages (Table 8.1). Beacon blocks (blobs)

Table 8.1: Ethereum message sizes

| Category               | Size (Bytes) |
|------------------------|--------------|
| Attestation            | 512          |
| Aggregated attestation | 1536         |
| Beacon block / Blob    | 131072       |
| Message identifier     | 32           |
| IHAVE / IWANT          | 32 / 16      |
| GRAFT / PRUNE          | 100          |

are large (approximately 128 KiB) but rare, with only one block produced per 12-second slot [19]. In contrast, attestation-related messages dominate by count. Empirical measurements of the Ethereum P2P network show that attestation-derived messages dominate gossip traffic by count, while beacon blocks are comparatively rare [25]. Individual attestations are aggregated prior to gossip dissemination, making aggregated attestations the primary message type processed by the Gossipsub layer. Accordingly, we model message dissemination using a single representative aggregated message size of 1.5 KiB which dominates Ethereum gossip traffic by volume (Table 8.1).

According to Ethereum’s attestation workload, each peer publishes messages according to a Poisson process with rate  $\lambda_{\text{pub}} = v/384$ , where  $v$  denotes the number of validators colocated with the peer. We primarily evaluate the case  $v = 1$ , yielding roughly one message every 384 seconds per peer. We use a Poisson process to smooth slot-synchronous effects while preserving Ethereum’s long-run attestation rate.

### 8.2.2 Network Model

**Topology.** We simulate a network of 1000 peers. We verified that qualitative trends remain unchanged for larger network sizes; we use 1000 peers to enable exhaustive parameter sweeps. Each peer maintains a neighboring peer table of 40–80 connections drawn from this population. The peer table size is consistent with measurements of Ethereum consensus nodes showing approximately 60 average connections per node [35]. Eager and lazy neighbors are sampled uniformly from this table.

**Network Delay Models.** We evaluate our simulator under two network delay models:

**Constant delay model.** All per-hop latencies are fixed at 25 ms, matching Protocol Labs’ 50 ms RTT assumption [75]. We use this model for validation (Section 8.2.9).

**Geographic delay model.** Nodes are partitioned into three regions reflecting Ethereum’s geographic distribution [72]: 30% Europe, 40% North America, and 30% Asia. Per-hop latencies are drawn from region-pair-specific normal distributions (Table 8.2). **We use this model for all parameter sweep experiments (Section ??), as it better reflects production Ethereum network conditions.**

|            | Europe                  | N. America              | Asia                     |
|------------|-------------------------|-------------------------|--------------------------|
| Europe     | $\mathcal{N}(30, 10^2)$ | $\mathcal{N}(90, 20^2)$ | $\mathcal{N}(180, 30^2)$ |
| N. America | —                       | $\mathcal{N}(40, 10^2)$ | $\mathcal{N}(150, 25^2)$ |
| Asia       | —                       | —                       | $\mathcal{N}(35, 10^2)$  |

Table 8.2: Per-hop latency distributions (ms) by region pair

The effective per-hop mean under this model is:

$$\mu_h^{\text{eff}} = \sum_{r,s} p_{rs} \cdot \mu_{rs} \quad (8.1)$$

where  $p_{rs}$  is the proportion of edges between regions  $r$  and  $s$ . With random peer selection, the effective per-hop mean latency is  $\mu_h^{\text{eff}} \approx 105$  ms.

**Churn.** Based on real-world measurements by Kiffer *et al* [35], we model churn using a bi-modal peer classification. A fraction  $c$  of peers are designated as “churny” and cycle between online and offline states with transition probabilities  $\lambda_{\text{leave}} = 0.01$  and  $\lambda_{\text{rejoin}} = 0.05$  per tick, resulting in rapid cycling with mean connection duration of 10 seconds, consistent with the short-lived connections observed in Ethereum’s P2P network [35]. The remaining  $1 - c$  “stable” peers remain online throughout the experiment ( $\lambda_{\text{leave}} = 0$ ). Note that when an offline peer comes back online, it loses all its state except its churn type and peer table (ambient peer discovery). At steady state, churny peers maintain approximately 83% availability, resulting in an overall network availability of  $(1 - c) + 0.83c$ . We evaluate configurations with  $c \in \{0.0, 0.1, 0.2, 0.3, 0.4\}$ .

### 8.2.3 Simulation Timing

The simulator advances in discrete ticks of duration  $\Delta t = 0.1$  seconds. Gossipsub heartbeat routines are executed every 10 ticks, corresponding to a 1 sec default heartbeat interval. At each tick the following events occur: (1) token-bucket refills for all peers, (2) message production according to a Poisson process with rate  $\lambda_{\text{pub}} = 1/384$  per peer, (3) processing of scheduled message deliveries according to per-link delays, (4) churn transitions (online/offline state changes of churny peers), (5) bandwidth enforcement via send accounting, which drops messages exceeding per-peer bandwidth limits, and (6) execution of heartbeat routines (every 10 ticks) including I HAVE emission and mesh maintenance.

**Message processing model.** Messages arriving within the same tick are processed sequentially in arbitrary order. With  $\Delta t = 0.1$  s and per-hop delays of 30-200 ms, simultaneous arrivals are rare; this discrete-time approximation preserves the invariant that each message is processed at most once per peer via the *seen* set.

**Processing rate.** We assume a conservative processing capacity of one payload and one control message per 100 ms tick (approximately 10 messages/sec). Higher service rates reduce absolute latency but do not change relative ordering across configurations.

We discard the first 2000 ticks (200 seconds) as warmup to allow the mesh to stabilize before measuring metrics. Algorithm 5 formalizes this tick-based execution model.

## 8.2.4 Bandwidth Model

**Token-bucket approximation of TCP** We model upload bandwidth using a token-bucket abstraction of TCP congestion control. Each peer has capacity  $B$  (25 Mbps), bucket size  $B/8$  bytes, and refill rate  $B \cdot \Delta t/8$  per tick. A message of size  $s$  is transmitted only if sufficient tokens are available. This permits short bursts while enforcing long-term throughput limits. Across all experiments, bandwidth exhaustion did not cause delivery loss; message loss arises solely from churn.

## 8.2.5 Evaluation Metrics

We evaluate Gossipsub configurations along three axes: delivery, efficiency, and latency. For each metric, we define both per-message and aggregate measures.

Let  $V$  denote the set of all peers in the network. At any time  $t$ , a subset of peers are online and participating in the protocol; we denote this set as  $V_t \subseteq V$ .

### Definition 28 (Delivery Set)

The *delivery set*  $R_{m,t}$  is the set of peers that received message  $m$  by time  $t$  during the simulation:

$$R_{m,t} = \{p \in V : m \in p.\textit{delivered} \text{ by time } t\} \quad (8.2)$$

where *delivered* is a persistent record of all messages received by peer  $p$ , which survives peer rejoin events.

### Definition 29 (Delivery rate)

We define the time-dependent delivery rate  $\delta_{m,t}$  for  $t \geq t_m$  as:

$$\delta_{m,t} = \frac{|R_{m,t}|}{|V|} \quad (8.3)$$

This captures the evolution of message dissemination through the network. At publication time,  $\delta_{m,t_m} = 0$  since no peer has yet received the message—not even the publisher  $v_m$ , which inserts  $m$  into its *pending* queue before  $m$  is processed, forwarded to mesh peers, and added to  $v_m.\textit{seen}$ . As the protocol propagates  $m$  through eager-push and gossip-based recovery,  $\delta_{m,t}$  increases monotonically, *i.e.*,  $\langle \forall t_i, t_j :: t_i \leq t_j \Rightarrow \delta_{m,t_i} \leq \delta_{m,t_j} \rangle$ . Let  $f_m$  be the first time after  $t_m$  when  $m$  is no longer in the *pending* queue of any peer and all gossip-based recovery attempts have completed. After  $f_m$ ,  $m$  will not be forwarded again. In a stable network without churn, we expect  $\delta_{m,f_m} = 1$ . However, under churn, some peers in  $V$  may go offline before receiving  $m$ , causing  $\delta_{m,f_m}$  to stabilize below 1. We define the *final delivery rate*  $\delta_m = \delta_{m,f_m}$ .

## Aggregate Metrics

### Definition 30 (Mean Delivery Rate)

For a simulation producing message set  $M$ , the mean delivery rate is:

$$\bar{\delta} = \frac{1}{|M|} \sum_{m \in M} \delta_m \quad (8.4)$$

Our simulation computes  $\delta_m$  at the end of the measurement period  $T$  rather than waiting for stabilization at  $f_m$ . Messages published near the end of the measurement period may still have  $m$  in some peer's *pending* queue when statistics are computed, yielding  $\delta_{m,T} \leq \delta_m$ . This slightly underestimates delivery rates, particularly for configurations with high tail latency. However, since this effect applies uniformly across all configurations, relative comparisons remain valid.

### Definition 31 (Total Bandwidth)

Total bytes transmitted during the measurement period:

$$B_{total} = \sum_{p \in V} (B_p^{\text{Payload}} + B_p^{\text{Ctrl}}) \quad (8.5)$$

where  $B_p^{\text{Payload}}$  counts message payload bytes and  $B_p^{\text{Ctrl}}$  counts control message (IHAVE, IWANT, GRAFT, PRUNE) bytes sent from peer  $p$ .

### Definition 32 (Bytes Per Delivery)

Quantifies bandwidth efficiency by measuring the average bytes transmitted per successful delivery:

$$\beta = \frac{B_{total}}{\sum_{m \in M} |R_m|} \quad (8.6)$$

This metric captures protocol overhead from multiple sources: redundant payload transmissions (*e.g.*, eager push to multiple mesh neighbors), control messages for lazy push or mesh maintenance, and wasted bandwidth from transmissions to peers that subsequently depart. Lower values indicate better resource utilization, with the theoretical minimum corresponding to perfect amortization of a single payload transmission across all recipients.

### Definition 33 (Normalized Cost)

We normalize by payload size  $P = 1536$  bytes:

$$\beta/P \quad (8.7)$$

This normalization yields a dimensionless measure capturing the average multiple of a payload's worth of traffic required per successful delivery, enabling direct comparison across



parameter settings independent of payload size. A normalized cost of 10 indicates that, on average, 10 payload-equivalents of total traffic are transmitted per successful delivery.

**Definition 34 (Per-Delivery Latency)**

For each successful delivery  $(m, p)$  where  $p \in R_m$ , the latency is:

$$\ell_{m,p} = (t_{m,p} - t_m) \cdot \Delta t \quad (8.8)$$

where  $t_{m,p}$  is the tick at which peer  $p$  first received message  $m$  in its pending queue.

**Definition 35 (P99 Latency)**

The 99th percentile latency is computed over all successful deliveries:

$$p99 = \text{Percentile}_{99}(\{\ell_{m,p} : m \in M, p \in R_m\}) \quad (8.9)$$

High  $p99$  may indicate suboptimal mesh connectivity, high churn, or protocol mechanisms (such as lazy pull) that delay message propagation under certain conditions. Latency is used only as a relative ordering metric for Pareto dominance; we do not claim that absolute  $p99$  values predict production behavior.

Together, these metrics characterize the delivery-efficiency-latency tradeoff space that distinguishes gossip protocol designs. An effective protocol must balance high delivery rates against bandwidth costs while maintaining acceptable latency bounds. All results are averaged across three independent random seeds to ensure statistical confidence.

## 8.2.6 Simulation Algorithm

Algorithm 5 describes the discrete-event simulation loop. Each tick processes events in a fixed order to ensure deterministic execution.

The full algorithm, along with description of each of the functions it depends on are available as auxiliary material [39].

## 8.2.7 Pareto Dominance Analysis

**Definition 36 (Pareto Dominance)**

Configuration  $A$  *dominates* configuration  $B$  on delivery rate and bandwidth cost if  $\delta_A \geq \delta_B$  and  $(\beta/P)_A \leq (\beta/P)_B$ , with at least one strict inequality.

We exclude latency from the dominance computation because Floodsub achieves the lowest latency (0.4s p99). Including latency would prevent any configuration from strictly dominating Floodsub, obscuring the clear delivery-cost advantage of Gossipsub. Latency is instead encoded as marker shape in Figure 8.3.3 for visual inspection. The algorithm description for pareto analysis is available as auxiliary material [39].

---

**Algorithm 5** Gossipsub Simulation Main Loop

---

**Require:** Network parameters  $(n, D, D_{\text{lazy}}, c)$ , simulation length  $T$ , warmup  $T_w$

**Ensure:** Metrics  $(\bar{\delta}, \beta_{\text{norm}}, p99)$

```
1: Initialize  $n$  peers with peer tables of size  $[40, 80]$ 
2: Initialize eager mesh  $\mathcal{M}_p$  and gossip peers  $\mathcal{L}_p$  for each peer  $p$ 
3:  $t \leftarrow 0$ 
4: while  $t < T_w + T$  do
5:   REFILLTOKENBUCKETS ▷ Bandwidth accounting
6:   PROUCEMESSAGES( $t$ ) ▷ Poisson arrivals
7:   PROCESSDELIVERIES( $t$ ) ▷ Scheduled network arrivals
8:   PROCESSCHURN( $t$ ) ▷ Bimodal leave/rejoin
9:   ENFORCEBANDWIDTH( $t$ ) ▷ Drop excess messages
10:  if  $t \bmod 10 = 0$  then ▷ Heartbeat every 1 second
11:    EMITGOSSIP( $t$ ) ▷ I HAVE announcements
12:    MAINTAINMESH( $t$ ) ▷ GRAFT/PRUNE
13:  end if
14:   $t \leftarrow t + 1$ 
15:  if  $t = T_w$  then
16:    Reset all counters ▷ End of warmup
17:  end if
18: end while
19: return COMPUTEMETRICS
```

---

### 8.2.8 Simulation Parameters

Table 8.3 lists all simulation parameters used in our experiments.

### 8.2.9 Validation Against Published Measurements

To validate our simulator, we compare against Protocol Labs’ Gossipsub v1.1 evaluation [75], which tested a 1000-node network with  $D = 8$ ,  $D_{\text{lazy}} = 8$ , and no churn. We use a fixed 25 ms per-hop delay to match Protocol Labs’ 50 ms RTT assumption. Our validation aims to establish the correctness of Gossipsub’s dissemination dynamics and the relative ordering of configurations, rather than to predict exact absolute performance in production deployments. We therefore validate against published testbed results under controlled conditions and emphasize qualitative agreement and dominance relationships, which are robust to simulation artifacts.

### Theoretical Latency Analysis

We first derive expected latency bounds from graph-theoretic properties of the mesh overlay.

Table 8.3: Simulation parameter values

| Parameter           | Symbol               | Value                    |
|---------------------|----------------------|--------------------------|
| <i>Network</i>      |                      |                          |
| Number of peers     | $n$                  | 1000                     |
| Peer table size     |                      | [40, 80]                 |
| Validators per peer | $v$                  | 1                        |
| <i>Timing</i>       |                      |                          |
| Tick duration       | $\Delta t$           | 0.1 s                    |
| Heartbeat interval  |                      | 1.0 s                    |
| Warmup period       | $T_w$                | 2000 ticks               |
| Measurement period  | $T$                  | 10000 ticks              |
| <i>Protocol</i>     |                      |                          |
| Mesh degree         | $D$                  | $\{2, 4, \dots, 14\}$    |
| Gossip degree       | $D_{lazy}$           | $\{1, 3, \dots, 33\}$    |
| Gossip history      |                      | 3 heartbeats             |
| <i>Messages</i>     |                      |                          |
| Payload size        | $P$                  | 1536 bytes               |
| Message ID size     |                      | 32 bytes                 |
| IHAVE overhead      |                      | 32 bytes                 |
| IWANT overhead      |                      | 16 bytes                 |
| GRAFT/PRUNE size    |                      | 100 bytes                |
| <i>Bandwidth</i>    |                      |                          |
| Global upload cap   | $B$                  | 25 Mbps                  |
| Token bucket size   |                      | $B/8$ bytes              |
| Refill rate         |                      | $B \cdot \Delta t/8$     |
| <i>Churn</i>        |                      |                          |
| Churny fraction     | $c$                  | $\{0, 0.1, \dots, 0.4\}$ |
| Churny leave rate   | $\lambda_{leave}^c$  | 0.01 / tick              |
| Churny rejoin rate  | $\lambda_{rejoin}^c$ | 0.05 / tick              |
| Stable leave rate   | $\lambda_{leave}^s$  | 0                        |
| <i>Replication</i>  |                      |                          |
| Seeds per config    |                      | 3                        |

Table 8.4: Validation against Protocol Labs Gossipsub evaluation (1000 nodes,  $D = 8$ ,  $D_{\text{lazy}} = 8$ , 0% churn, 10 ms ticks)

| Metric           | Ours    | Protocol Labs | Notes                       |
|------------------|---------|---------------|-----------------------------|
| Delivery rate    | 99.7%   | 100%          | Primary metric: matches     |
| p50 latency      | 0.240 s | 0.100 s       | $2.4\times$ (tick overhead) |
| p99 latency      | 0.390 s | 0.165 s       | $2.4\times$ (tick overhead) |
| Maximum latency  | 0.540 s | 0.350 s       | $1.5\times$                 |
| ETH2 req. (<3 s) | ✓       | ✓             | Both satisfy                |

**Average shortest-path length.** The eager mesh forms a random regular graph with degree  $D$ . For such graphs, the average shortest-path length (ASPL) scales logarithmically:

$$\text{ASPL} \approx \frac{\ln n}{\ln D} = \frac{\ln 1000}{\ln 8} \approx 3.32 \text{ hops} \quad (8.10)$$

Empirical measurements from our simulation refine this to  $\text{ASPL} \approx 3.0$  hops, consistent with ProbeLab’s DEthna measurements on the Goerli testnet ( $\text{ASPL} \approx 2.7$  for  $\sim 1\text{--}2\text{k}$  nodes) [89].

**Expected end-to-end latency.** Given one way latency per-hop with mean  $\mu_h$  and standard deviation  $\sigma_h$ , the expected end-to-end latency is:

$$\mu_{\text{e2e}} \approx \text{ASPL} \times \mu_h \quad (8.11)$$

The variance combines within-path variability and between-path hop-count variability:

$$\sigma_{\text{e2e}}^2 \approx \text{ASPL} \times \sigma_h^2 + \text{Var}(\text{hops}) \times \mu_h^2 \quad (8.12)$$

Assuming Gaussian approximation, the 99th percentile is:

$$\text{p99} \approx \mu_{\text{e2e}} + 2.326 \times \sigma_{\text{e2e}} \quad (8.13)$$

For Protocol Labs’ parameters ( $\mu_h \approx 25$  ms from 50 ms RTT, negligible variance), predicted p99 is  $3.0 \times 25\text{ms} \approx 75$  ms. Protocol Labs reports 165 ms, with the  $\approx 2\times$  difference attributable to protocol processing overhead (deserialization, validation, forwarding decisions).

## Validation Results

Table 8.4 compares our simulator against Protocol Labs measurements. We test with per-hop delay of 25 ms (matching Protocol Labs’ 50 ms RTT assumption) using 10 ms tick resolution.

**Delivery rate.** Our simulator achieves 99.7% delivery rate compared to Protocol Labs’ 100%. This minor difference is an artifact of our measurement methodology: as discussed in Section 8.2.5, we compute delivery rate at the end of the measurement period  $T$  rather than waiting for full stabilization. Messages published near the end of the simulation may still be in transit when statistics are computed, yielding  $\delta_{m,T} < \delta_m$ . With sufficient settling time, we expect 100% delivery rate under zero churn, confirming that our implementation correctly captures Gossipsub’s dissemination mechanics.

**Latency.** Absolute latencies are approximately  $2.4\times$  higher than Protocol Labs due to tick-based simulation overhead. With  $\text{ASPL} \approx 3\text{--}4$  hops and effective per-hop latency of  $\sim 60$  ms (25 ms network + tick overhead), predicted p50 is  $3.5 \times 60\text{ms} = 210\text{ms}$ , consistent with our measured 240 ms. Protocol Labs’ event-driven simulator avoids this overhead, yielding latencies closer to theoretical network delay. Crucially, both simulators satisfy Ethereum’s informal requirement of full attestation propagation within 3 seconds with substantial margin. The latency difference reflects simulation methodology rather than protocol behavior. Our discrete-time model (Algorithm 5) uses 100 ms ticks for the parameter sweep, introducing quantization overhead of  $\lceil \text{delay}/\Delta t \rceil + 1$  ticks per hop; we reduce it by using 10 ms ticks for validation.

**Implications for parameter sweep.** Since delivery rate, our primary metric, matches Protocol Labs, and latency differences arise from consistent simulation overhead, relative comparisons across configurations remain valid. The parameter sweep (Section ??) uses 100 ms ticks for computational efficiency; all 500 configurations are evaluated under identical conditions, ensuring fair comparison.

## Qualitative Trend Validation

Beyond absolute metric values, we validate that our simulation reproduces the qualitative behavior documented in prior work:

**Monotonic delivery improvement with  $D$ :** Protocol Labs observes that higher mesh degree improves delivery, consistent with our Figure 8.3.1(a) appearing later in the Results section.

**Delivery rate–Bandwidth cost tradeoff:** The evaluation report notes that increased connectivity comes at bandwidth cost, matching our finding in Figure 8.3.1(b).

**Gossip as recovery mechanism:** Protocol Labs describes lazy pull (gossip) as enabling message recovery when mesh paths fail, consistent with our observation that  $D_{\text{lazy}}$  improves delivery under churn (Figure 8.3.2(a)).

**Latency sensitivity to gossip:** The report notes that gossip propagation adds latency (up to one heartbeat interval per hop), explaining the tail latency behavior we observe.

## Limitations of Validation

We acknowledge several limitations in our validation approach:

**Churn validation:** No published measurements characterize Gossipsub behavior under controlled churn conditions comparable to our simulation. Our churn model is based on connection duration measurements from Kiffer *et al* [35], but delivery rate degradation under churn has not been validated against production data.

**Scale gap:** Our 1000-node simulation is substantially smaller than production Ethereum networks. While we verified that qualitative tradeoffs and the relative ordering of parameter configurations persist at larger scales, absolute metric values may differ.

**Simplified protocol model:** We omit peer scoring, multi-topic overlays, and Floodsub compatibility. These simplifications may affect behavior under adversarial conditions but should minimally impact benign churn scenarios.

Despite these limitations, the consistency between our simulation and Protocol Labs’ testbed results, particularly for baseline latency, and the qualitative structure of delivery-bandwidth tradeoffs, provides confidence that our methodology captures the essential dissemination dynamics of Gossipsub.

## 8.3 Simulation Results

We now evaluate how Gossipsub parameters influence the delivery–efficiency–latency trade-off defined in Section 8.2.5. Our analysis focuses on two protocol parameters: the mesh degree  $D$ , which controls eager-push connectivity, and the gossip degree  $D_{\text{lazy}}$ , which controls lazy dissemination. We simulate Floodsub ( $D=60$ ,  $D_{\text{lazy}}=0$ ) as a baseline representing pure eager-push flooding without gossip-based recovery. Since Floodsub is modeled via Gossipsub parameters, it inherits mesh maintenance behavior; rejoining peers establish mesh connections within one heartbeat interval ( $\sim 1$  s). This approximates realistic Floodsub behavior, as establishing 60 TCP connections after rejoin would incur comparable latency.

For each parameter configuration, we report results across all five churn regimes (0–40%) with error bars indicating the range across the other parameter dimension. This visualization reveals both typical behavior and the envelope of outcomes under varying network conditions. The separation is critical in churned networks, where churn-induced message loss can significantly impact protocol performance.

### 8.3.1 Effect of Mesh Degree

Figure 8.3.1 shows the impact of mesh degree  $D$  on delivery rate, bandwidth cost, and tail latency. Each line corresponds to a churn regime, with vertical bars indicating the range across gossip degrees  $D_{\text{lazy}}$ .

**delivery.** Delivery rate improves with increasing  $D$  but exhibits strong diminishing returns, plateauing beyond  $D \approx 8$ . At 0% churn, all Gossipsub configurations achieve  $\sim 100\%$  delivery. At 10–20% churn, delivery remains above 98% for  $D \geq 4$ . At 40% churn, configurations with  $D \geq 4$  maintain  $>98\%$  delivery, while Floodsub ( $D=60$ ) degrades to 91.8%. The narrow error bars indicate that  $D_{\text{lazy}}$  has modest impact on delivery compared to  $D$  and churn level.

**Efficiency.** Normalized cost ( $\beta/P$ ) grows monotonically with  $D$ . Floodsub incurs  $\sim 60\times$  overhead, while Gossipsub ranges from  $2\times$  at low  $D$  and churn to  $\sim 300\times$  at high  $D$ ,  $D_{\text{lazy}}$ , and churn. The wide vertical bars reveal that  $D_{\text{lazy}}$  substantially impacts cost—higher gossip degrees increase bandwidth consumption significantly. Low- $D$  configurations with minimal gossip achieve comparable delivery at substantially lower cost.

**Latency.** Tail latency decreases with increasing  $D$ , reflecting faster propagation in well-connected meshes. Floodsub achieves the lowest latency ( $\sim 0.4$  s) due to pure eager-push with no gossip delays. Gossipsub latency ranges from  $\sim 0.5$ –2 s at 0% churn to 10–100 s under high churn, with higher  $D$  reducing latency at all churn levels.

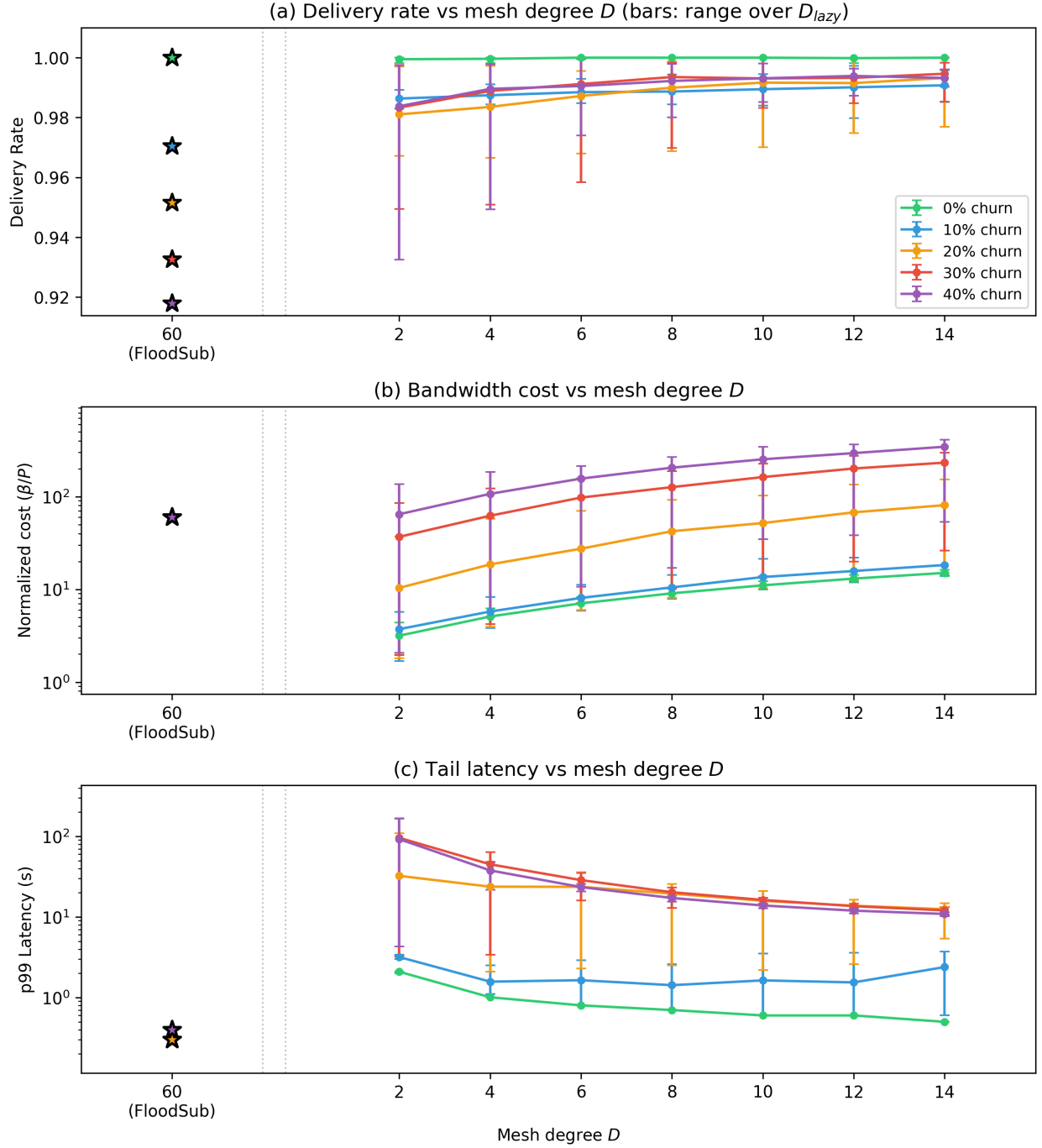


Figure 8.3.1: Effect of mesh degree  $D$  under varying churn regimes. Each line represents a churn level (0–40%), plotting the mean across all  $D_{lazy}$  values; error bars show the range (min–max) over  $D_{lazy} \in \{1, 3, \dots, 33\}$ . Floodsub ( $D=60$ ,  $D_{lazy}=0$ ) shown as stars. (a) Delivery rate vs.  $D$ . (b) Normalized cost ( $\beta/P$ ) vs.  $D$ . (c) Tail latency vs.  $D$ .

**Key insight:** Under churn, Gossipsub dominates Floodsub on both delivery and cost. Floodsub’s only advantage is sub-second latency. Gossip-based recovery ( $D_{\text{lazy}} \geq 1$ ) provides delivery that additional eager-push connections alone cannot achieve.

### 8.3.2 Effect of Gossip Degree

Figure 8.3.2 evaluates the impact of gossip degree  $D_{\text{lazy}}$ . Each line corresponds to a churn regime, with vertical bars indicating the range across mesh degrees  $D \in \{2, 4, \dots, 14\}$ .

**delivery.** Even minimal gossip ( $D_{\text{lazy}}=1$ ) achieves  $>96\%$  delivery across all churn levels, matching or exceeding Floodsub’s 91.8–100%. Increasing  $D_{\text{lazy}}$  provides marginal improvement: delivery at 40% churn rises from  $\sim 97\%$  ( $D_{\text{lazy}}=1$ ) to  $\sim 99\%$  ( $D_{\text{lazy}}=21$ ). The wide vertical bars indicate that  $D$  has substantial impact on delivery, particularly under high churn, while  $D_{\text{lazy}}$  has relatively modest effect beyond the minimal value.

**Efficiency.** Normalized cost increases sharply with  $D_{\text{lazy}}$ , from 2–30 $\times$  at  $D_{\text{lazy}}=1$  to 10–300 $\times$  at  $D_{\text{lazy}}=21$  depending on churn. This reflects IHAVE/IWANT control message overhead. High- $D_{\text{lazy}}$  configurations under churn exceed Floodsub’s  $\sim 60\times$  cost while providing only marginal delivery improvement.

**Latency.** Tail latency shows high variability under churn, with  $D_{\text{lazy}}$  having modest direct effect. At 0–10% churn, latency remains stable ( $<3$  s). Under 20–40% churn, latency ranges from  $\sim 10$  s to  $\sim 100$  s depending primarily on  $D$  (shown by wide vertical bars). Floodsub achieves the lowest latency ( $\sim 0.4$  s) due to pure eager-push.

Pure gossip without an eager mesh  $D=0$  performs poorly. Even with  $D_{\text{lazy}}=33$ , delivery reached only 41–43% with  $p99$  latency up to 469 s. At  $D_{\text{lazy}}=3$ , delivery was just 4–6%. This confirms that eager push provides essential baseline connectivity; gossip serves as a recovery mechanism, not a primary dissemination path.

### 8.3.3 Best-effort Trade-offs and Dominance

Figure 8.3.3 plots worst-case delivery against normalized cost, with marker shape encoding  $p99$  latency. Floodsub achieves the lowest latency (0.4 s), so it is not strictly Pareto-dominated in three dimensions. However, on the delivery-cost plane, it lies below the frontier: multiple Gossipsub configurations achieve both higher delivery and lower cost.

**Gossipsub dominates Floodsub on delivery and cost.** Floodsub ( $D=60$ ,  $D_{\text{lazy}}=0$ ) achieves 91.8% worst-case delivery at 60 $\times$  cost. Every  $D_{\text{lazy}}=1$  configuration (blue markers) exceeds this delivery at lower cost. Table 8.5 shows representative configurations: at 10% churn, (2, 1) achieves 98.4% delivery at 35 $\times$  lower cost; at 40% churn, (2, 1) achieves 93.3% at 29 $\times$  lower cost.



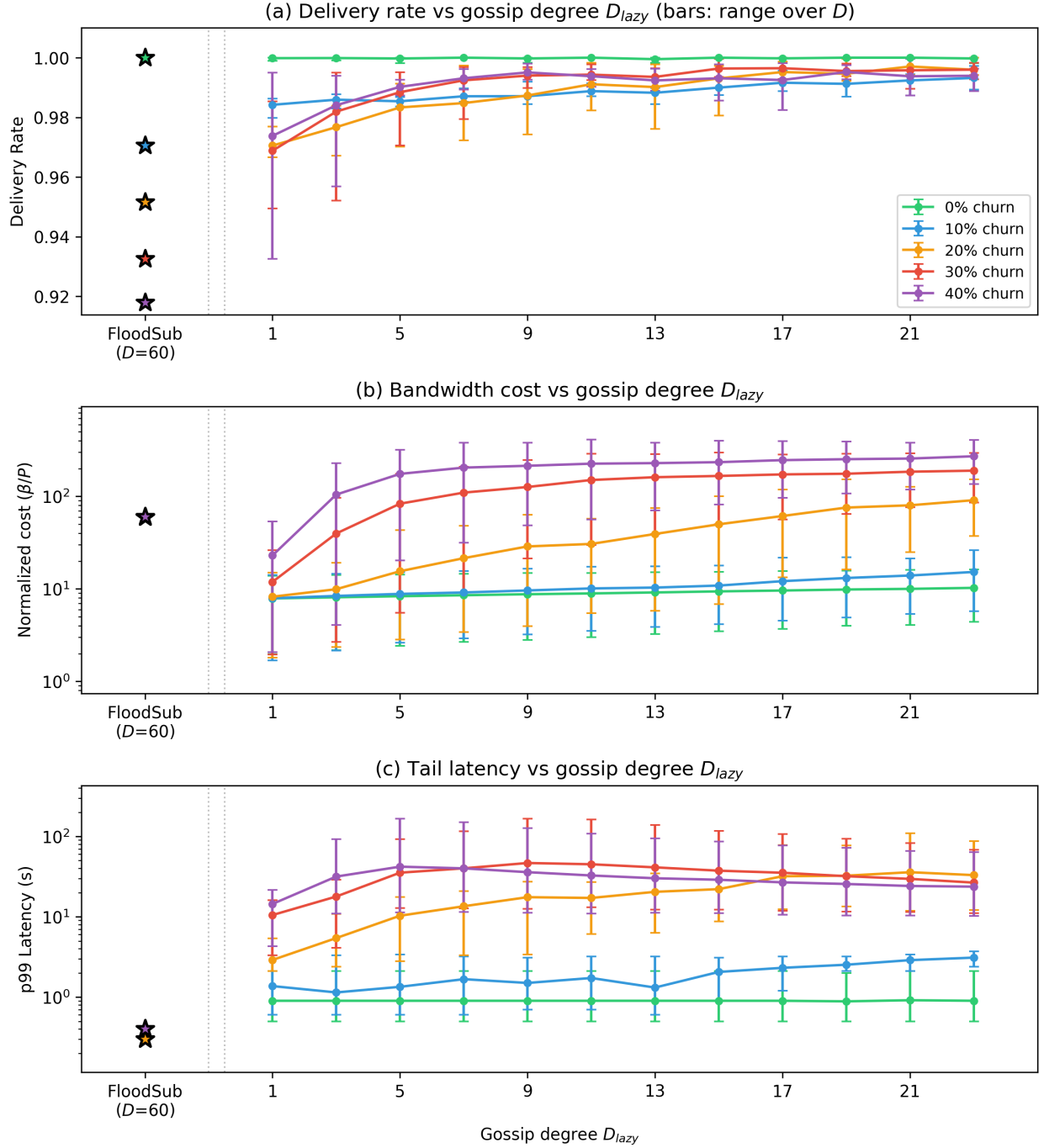


Figure 8.3.2: Effect of gossip degree  $D_{lazy}$  under varying churn regimes. Each line represents a churn level (0–40%), plotting the mean across all  $D$  values; error bars show the range (min–max) over  $D \in \{2, 4, \dots, 14\}$ . Floodsub ( $D=60$ ,  $D_{lazy}=0$ ) shown as stars. (a) Delivery rate vs.  $D_{lazy}$ . (b) Normalized cost ( $\beta/P$ ) vs.  $D_{lazy}$ . (c) Tail latency vs.  $D_{lazy}$ .

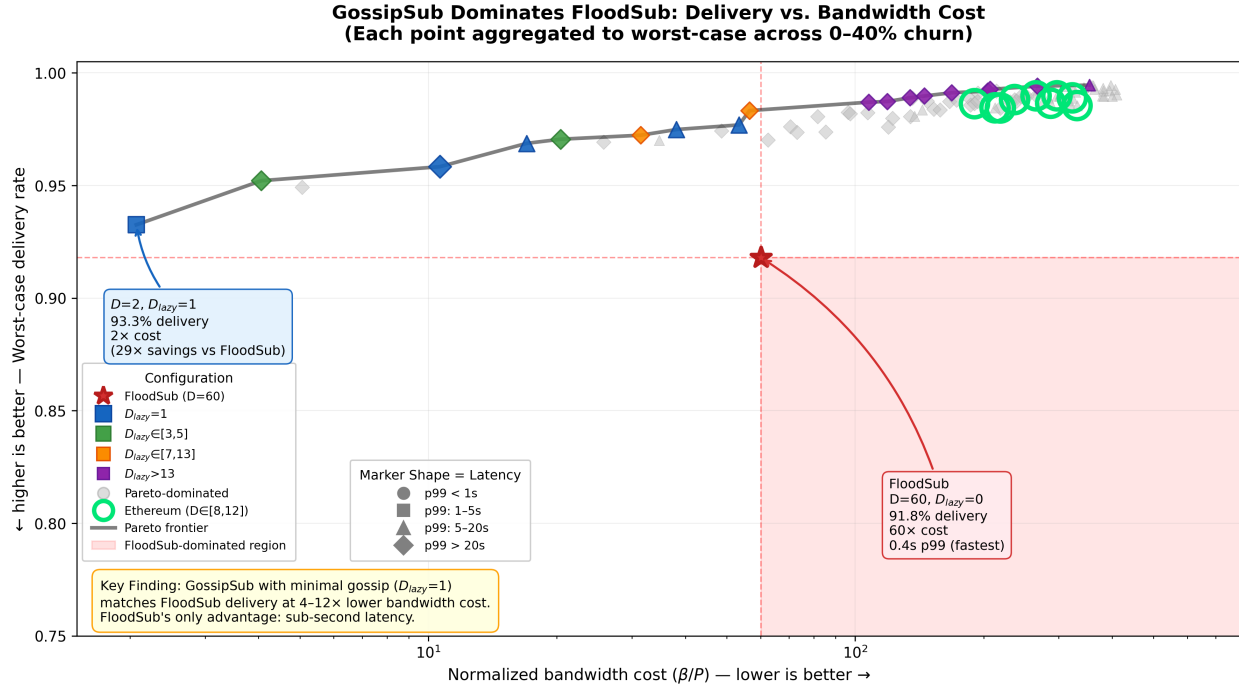


Figure 8.3.3: Pareto dominance: worst-case delivery vs. normalized cost. Each point is a  $(D, D_{lazy})$  configuration aggregated across 0–40% churn. Marker color indicates  $D_{lazy}$ ; shape indicates  $p99$  latency tier. Floodsub ( $D=60$ , red star) achieves 91.8% delivery at 60× cost. Gossipsub ( $D=2, D_{lazy}=1$ ) achieves 93.3% delivery at 2× cost, providing 29× savings. Red region: configurations strictly inferior to Floodsub.

Table 8.5: Gossipsub configurations dominating Floodsub at each churn level. Minimal gossip ( $D_{\text{lazy}}=1$ ) with sparse mesh exceeds Floodsub delivery at every churn level while achieving substantial cost savings.

| Churn | Floodsub | Gossipsub | GS Delivery | Cost Savings |
|-------|----------|-----------|-------------|--------------|
| 0%    | 100.0%   | (4, 1)    | 100.0%      | 15×          |
| 10%   | 97.1%    | (2, 1)    | 98.4%       | 35×          |
| 20%   | 95.2%    | (2, 1)    | 96.9%       | 33×          |
| 30%   | 93.3%    | (2, 1)    | 94.9%       | 31×          |
| 40%   | 91.8%    | (2, 1)    | 93.3%       | 29×          |

**Floodsub’s only advantage is sub-second latency.** Most  $D_{\text{lazy}}=1$  configurations (blue markers) fall in the 5–20s latency tier—acceptable for Ethereum’s 12s slots. At this latency, they dominate Floodsub on both delivery and cost. Floodsub’s 0.4s latency matters only for applications requiring sub-second propagation, at 60× the bandwidth cost.

### 8.3.4 Dominance Over Ethereum’s Deployed Configuration

Ethereum deploys Gossipsub with mesh degree  $D=8$  and gossip degree  $D_{\text{lazy}}=8$ . We evaluate this configuration under worst-case performance aggregated across 0–40% churn and compare it against the Pareto frontier.

Figure 8.3.3 shows that Ethereum’s deployed configuration lies strictly below the Pareto frontier in the delivery–bandwidth plane. In particular, Lean Gossip configurations with  $D_{\text{lazy}}=1$  achieve equivalent or higher worst-case delivery at substantially lower normalized cost. For example, configurations such as  $(D=8, D_{\text{lazy}}=1)$  and even  $(D=4, D_{\text{lazy}}=1)$  provide comparable delivery while reducing bandwidth consumption by a large factor relative to  $(D=8, D_{\text{lazy}}=8)$ . Under worst-case aggregation, no setting with  $D_{\text{lazy}}>1$  lies on the frontier. This establishes that Ethereum’s deployed gossip degree is over-provisioned for benign churn scenarios. delivery scales primarily with mesh degree  $D$ , while increasing  $D_{\text{lazy}}$  beyond 1 incurs steep bandwidth cost with negligible delivery benefit.

Thus, Lean Gossip strictly dominates Ethereum’s deployed configuration in the worst-case delivery-bandwidth tradeoff across 0–40% churn.

## 8.4 Discussion

Our results reveal three structural insights about gossip-based dissemination under churn.

### 8.4.1 Why Floodsub Is Strictly Dominated

Floodsub attempts delivery through network-layer redundancy: maintaining 60 eager-push connections to maximize delivery probability. However, under churn, if a message fails to reach a node because its mesh peers are offline, there is no recovery mechanism. Gossipsub employs protocol-layer repair via IHAVE/IWANT exchanges, enabling missed messages to be recovered even with sparse meshes.

As a result, Floodsub delivers fewer messages at dramatically higher bandwidth cost. Its only advantage is sub-second latency.

### 8.4.2 Why Lean Gossip Dominates Ethereum’s Configuration

Ethereum deploys  $D=8$ ,  $D_{\text{lazy}}=8$ , reflecting a conservative balance between redundancy and repair. Our Pareto analysis shows that  $D_{\text{lazy}}=1$  configurations achieve equivalent or higher worst-case delivery at substantially lower cost.

This suggests that widely deployed production parameters may be suboptimal even under benign churn. delivery scales primarily with mesh connectivity, not gossip fanout. Reducing  $D_{\text{lazy}}$  preserves delivery while significantly reducing bandwidth.

### 8.4.3 Recovery Dominates Redundancy

A central conceptual contribution of this work is identifying that gossip-based recovery—not eager-push redundancy—is the dominant mechanism for delivery under churn.

Increasing  $D$  beyond approximately 8 yields diminishing delivery returns. Increasing  $D_{\text{lazy}}$  beyond 1 sharply increases bandwidth cost with minimal benefit. Pure gossip without eager mesh connectivity performs poorly, confirming that eager push provides baseline reachability while gossip provides repair. The true tradeoff is therefore latency versus cost—not delivery versus cost. Sparse meshes combined with minimal recovery dominate brute-force flooding in the delivery–bandwidth plane.

### 8.4.4 Practical Recommendations

Based on our simulations, we offer guidance for Gossipsub deployments.

**For delivery-critical applications.** Use  $D \geq 8$  with  $D_{\text{lazy}}=1$ . These achieve approximately 99% delivery even at 40% churn while maintaining cost advantages over Floodsub (Table 8.6).

**For bandwidth-constrained environments.** Configurations with  $D_{\text{lazy}}=1$  achieve delivery *exceeding* Floodsub at 4–30× lower cost. We recommend  $D=8$ ,  $D_{\text{lazy}}=1$  for home validators.

**For latency-critical applications.** If sub-second propagation is required, Floodsub ( $D=60$ ,  $D_{\text{lazy}}=0$ ) achieves 0.4 s  $p99$  latency—but at 60× cost and 2% lower delivery.

Table 8.6: Representative configurations across the reliability-efficiency-latency spectrum (worst-case across churn regimes). Gossipsub configurations with  $D_{\text{lazy}}=1$  achieve higher delivery than Floodsub at lower cost; Floodsub’s only advantage is latency.

| $D$                                                              | $D_{\text{lazy}}$ | Delivery | p99 Latency | Cost ( $\beta/P$ ) |
|------------------------------------------------------------------|-------------------|----------|-------------|--------------------|
| 2                                                                | 1                 | 93.3%    | 4.3 s       | $2\times$          |
| 4                                                                | 1                 | 94.9%    | 21.6 s      | $5\times$          |
| 8                                                                | 1                 | 96.9%    | 17.0 s      | $17\times$         |
| 12                                                               | 1                 | 97.5%    | 12.9 s      | $38\times$         |
| 14                                                               | 1                 | 97.7%    | 10.8 s      | $53\times$         |
| <i>Floodsub reference (<math>D=60, D_{\text{lazy}}=0</math>)</i> |                   |          |             |                    |
| 60                                                               | 0                 | 91.8%    | 0.4 s       | $60\times$         |

**Ethereum’s parameters.** Ethereum’s deployed mesh degree ( $D=8$ ) achieves 100% delivery at 0% churn and 99% at 40% churn; our results show that equivalent delivery can be achieved with substantially lower bandwidth by reducing gossip to  $D_{\text{lazy}}=1$  (Table 8.7).

**Gossip explosion under churn.** At  $D=2$  with  $D_{\text{lazy}}=23$  and 40% churn, traffic reaches 106 GB—a  $68\times$  increase over  $D_{\text{lazy}}=1$  (1.6 GB)—while improving delivery by only 5.6 percentage points (93.3% to 98.9%). This occurs because churning peers repeatedly request messages via I HAVE/I WANT that never complete. Protocol designers should consider gossip backoff under detected churn.

## 8.5 Limitations and Future Work

Our analysis has several limitations. First, we model single-topic networks, while production Gossipsub deployments often involve multiple overlapping topics with shared mesh connections. Second, our churn model assumes uniform leave/join probabilities, whereas real networks may exhibit correlated churn (*e.g.*, geographic outages affecting multiple peers simultaneously). Third, we do not model adversarial behavior; while our parameter recommendations optimize for benign churn, attack resilience under these parameters requires separate evaluation [42, 75]. Fourth, our network topology assumes a random graph with fixed degree, whereas real peer discovery mechanisms may produce different connectivity patterns.

Future work should extend this analysis to multi-topic scenarios, incorporate peer scoring mechanisms from Gossipsub v1.1 [50], and validate recommendations through testbed experiments or instrumentation of production networks. Additionally, our Pareto optimization

Table 8.7: Delivery rate by churn level for Gossipsub ( $D=8$ ,  $D_{lazy}=1$ ) compared to Floodsub ( $D=60$ ,  $D_{lazy}=0$ ). Gossipsub achieves higher delivery at every churn level  $>0\%$ .

| Churn | GS(8,1)<br>Delivery | GS(8,1)<br>p99 | Floodsub<br>Delivery | Floodsub<br>p99 |
|-------|---------------------|----------------|----------------------|-----------------|
| 0%    | 100.0%              | 0.7 s          | 100.0%               | 0.3 s           |
| 10%   | 98.5%               | 0.7 s          | 97.1%                | 0.3 s           |
| 20%   | 96.9%               | 2.5 s          | 95.2%                | 0.3 s           |
| 30%   | 97.0%               | 13.0 s         | 93.3%                | 0.4 s           |
| 40%   | 98.0%               | 17.0 s         | 91.8%                | 0.4 s           |

framework could be applied to emerging protocol variants such as Gossipsub v2.0 [71], which introduces probabilistic lazy forwarding within the mesh itself. Finally, integrating our parameter selection methodology into adaptive protocols that dynamically adjust  $D$  and  $D_{lazy}$  based on observed network conditions represents a promising direction for self-optimizing gossip network.

## 8.6 Bibliographic Notes

One of the earliest investigations into P2P communication was conducted by Paul Baran [4] during the late 1950s, who proposed flooding as a mechanism for building resilient, low-data-rate networks capable of withstanding both equipment failures and intentional disruptions. He quantified connectivity vs. survivability by simulating random node failures and measuring the percentage of the network that remained reachable. The first challenge that arises in naive flooding is redundancy: the same message may be received multiple times by the same node, leading to bandwidth waste and network congestion. This issue was formalized in the broadcast storm problem by Ni *et al* [67] who identified the three core causes: contention, collision, and redundant rebroadcasts, through simulations in Mobile Ad hoc NETworks (MANETs). Gopal and Kutten [22] proposed protocols like BASIC and RELIEF that achieve efficient broadcast in networks by relying on timing guarantees: for example, nodes forward a message for a bounded duration before halting. While efficient, such approaches raise concerns about reliability of message delivery. More recently, selective rebroadcast schemes [79] reconsider the conventional suppression approach by actively promoting controlled redundancy. Rather than minimizing rebroadcasts, these schemes encourage additional transmissions to enhance reliability in lossy or highly dynamic network environments.

Lynch’s study on reliable broadcast in networks with non-programmable servers [21] presents an early and influential algorithm for disseminating messages, explicitly analyzing

the trade-offs between reliability and message transmission costs. While links may fail or recover dynamically, the protocol assumes that the network remains continuously connected. Beyond this work, Lynch and Vaandrager developed a refinement framework for verifying distributed systems using forward and backward simulations. Their untimed framework [53] provides techniques for relating concrete implementations to abstract specifications via trace inclusion, enabling modular correctness proofs for asynchronous message-passing protocols. This framework was later extended to timing-based systems [52, 54], introducing simulation relations capable of reasoning about real-time constraints, timed actions, and clocks.

Parallel to these classical reliable-broadcast results, a substantial body of work developed probabilistic epidemic or gossip-based dissemination. Demers *et al* [16] demonstrated that a fanout of  $\log(n)$  is sufficient to spread messages to nearly all nodes in  $\mathcal{O}(\log(n))$  rounds with high probability, assuming uniform random peer selection. Analytical models [5, 20] of protocols where nodes periodically exchange information with randomly chosen peers showed that coverage increases exponentially with fanout. These protocols trade deterministic guarantees for simplicity, resilience, and scalable expected performance, and they form the conceptual foundation underlying modern P2P publish-subscribe (pubsub) protocols such as Gossipsub.

Previous work by Cristo *et al* [15] presents a systematic evaluation to characterize and explain Gossipsub’s behavior across several axes when integrated with XRP’s consensus layer. The study examines nine key dimensions—including message propagation delay, mesh stability, scoring dynamics, resilience under churn, fault tolerance, and adversarial conditions to understand performance and reliability trade-offs.

Orthogonally, Vyzovitis *et al* [75, 76, 83] presented a detailed analysis of the resilience of Gossipsub implementations against Sybil and Eclipse attacks. On the contrary, Kumar *et al* [42] used formal model driven analysis to show that there exist Ethereum 2.0 configurations of Gossipsub prone to sybil attacks that can lead to eclipse or even network partitions.





# Part IV

## Conclusion



# Chapter 9

## Conclusion

This dissertation developed a principled framework for analyzing gossip-based publish–subscribe protocols through compositional refinement, property-driven counterexample generation, and simulation-guided optimization. By studying Floodsub, Meshsub, and Gossipsub in succession, we showed how increasingly sophisticated protocol mechanisms can be related back to a simple broadcast specification, and how deviations from that specification can be precisely identified and explained.

At the heart of this work is a uniform dissemination guarantee: once a message is committed at its origin, every peer subscribed at commit time eventually receives it. We first proved this guarantee for Broadcastsub, then lifted it compositionally to Floodsub, and finally to Meshsub via probabilistic skipping refinement. These results demonstrate that selective mesh-based forwarding preserves semantic dissemination guarantees while enabling bandwidth reduction, provided explicit fairness and gossip assumptions hold.

Crucially, the refinement proofs expose the structural role of mesh degree  $D$  and gossip fanout  $D_{\text{lazy}}$  as parameters governing the extent of probabilistic stuttering and dissemination delay. Rather than treating performance and correctness as separate concerns, the analysis makes their interaction explicit: the same parameters that bound refinement also determine real-world trade-offs between latency and bandwidth efficiency.

Extending the analysis to Gossipsub v1.1 revealed a fundamental mismatch between control-plane scoring and dissemination invariants. Property-based testing and automated counterexample generation showed that peer scoring can violate behavioral consistency assumptions required for dissemination correctness. These counterexamples were not merely theoretical; they exposed concrete attack gadgets and informed public vulnerability disclosures. The framework therefore serves not only as a vehicle for certification, but also as a diagnostic tool for identifying semantic weaknesses in deployed systems.

Finally, simulation-based exploration of the Gossipsub parameter space demonstrated that the structural insights derived from refinement analysis translate directly into practical guidance. Pareto analysis over delivery, bandwidth, and latency confirmed that gossip-based recovery—not eager redundancy—is the dominant mechanism for robustness under churn. In this way, formal reasoning and empirical evaluation reinforce one another.

Taken together, this work advocates a layered methodology for distributed protocol analysis: establish strong semantic guarantees for core abstractions, lift those guarantees compositionally, test the compatibility of extensions, and use simulation to guide deployment

decisions. Rather than attempting end-to-end verification of production-scale systems, this approach isolates correctness arguments where they are most effective while retaining the ability to uncover subtle dissemination failures in real-world protocols.

## 9.1 Future Work

Several directions remain open.

**Refinement with scoring.** A natural next step is to integrate peer scoring into the refinement framework. This would require modeling score evolution explicitly and characterizing conditions under which scoring preserves, rather than undermines, dissemination guarantees.

**Adversarial churn and strategic behavior.** Our analysis assumes benign churn and fair scheduling. Extending the models to incorporate adversarial churn patterns and strategic peer behavior would strengthen the connection between formal guarantees and security analysis.

**Multi-topic and cross-layer interactions.** We focused primarily on single-topic dissemination. Extending the formal and simulation models to capture cross-topic interactions, shared mesh dynamics, and resource contention remains an important step toward characterizing behavior at scale.

More broadly, this work opens the door to hybrid analysis pipelines in which formal verification, counterexample generation, and empirical evaluation operate in concert. Such pipelines may provide a scalable path toward principled correctness reasoning for complex distributed systems.

# Bibliography

- [1] Martín Abadi and Leslie Lamport. The existence of refinement mappings. *Theor. Comput. Sci.*, 1991. doi:[10.1016/0304-3975\(91\)90224-P](https://doi.org/10.1016/0304-3975(91)90224-P).
- [2] Ioannis Aekaterinidis and Peter Triantafillou. Peer-to-peer publish-subscribe systems. In *Encyclopedia of Database Systems, Second Edition*. Springer, 2018. doi:[10.1007/978-1-4614-8265-9\\_1221](https://doi.org/10.1007/978-1-4614-8265-9_1221).
- [3] Noran Azmy, Stephan Merz, and Christoph Weidenbach. A rigorous correctness proof for pastry. In *Abstract State Machines, Alloy, B, TLA, VDM, and Z*, 2016. doi:[10.1007/978-3-319-33600-8\\_5](https://doi.org/10.1007/978-3-319-33600-8_5).
- [4] Paul Baran. On distributed communications: I. introduction to distributed communications networks. Technical Report RM-3420-PR, RAND Corporation, 1964. URL [https://www.rand.org/pubs/research\\_memoranda/RM3420.html](https://www.rand.org/pubs/research_memoranda/RM3420.html). Accessed: 2025-06-08.
- [5] K. Birman, M. Hayden, O. Ozkasap, Z. Xiao, M. Budiu, and Y. Minsky. Bimodal multicast. *ACM Transactions on Computer Systems (TOCS)*, 1999. doi:[10.1145/317785.317786](https://doi.org/10.1145/317785.317786).
- [6] Paul Bonsma. Most balanced minimum cuts. *Discrete Applied Mathematics*, 2010. doi:<https://doi.org/10.1016/j.dam.2009.09.010>.
- [7] Michael C. Browne, Edmund M. Clarke, and Orna Grumberg. Characterizing finite kripke structures in propositional temporal logic. *Theoretical Computer Science*, 1988. doi:[10.1016/0304-3975\(88\)90098-9](https://doi.org/10.1016/0304-3975(88)90098-9).
- [8] Miguel Castro, Peter Druschel, A-M Kermarrec, and Antony IT Rowstron. Scribe: A large-scale and decentralized application-level multicast infrastructure. *IEEE Journal on Selected Areas in communications*, 2002.
- [9] Harsh Chamarthi, Peter C. Dillinger, Panagiotis Manolios, and Daron Vroon. The "acl2" sedan theorem proving system. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2011. doi:[10.1007/978-3-642-19835-9\\_27](https://doi.org/10.1007/978-3-642-19835-9_27).
- [10] Harsh Raju Chamarthi. *Interactive Non-theorem Disproving*. PhD thesis, Northeastern University, 2016.

- [11] Harsh Raju Chamarthi and Panagiotis Manolios. Automated specification analysis using an interactive theorem prover. In Per Bjesse and Anna Slobodová, editors, *International Conference on Formal Methods in Computer-Aided Design, FMCAD '11*, pages 46–53. FMCAD Inc., 2011. URL <http://dl.acm.org/citation.cfm?id=2157665>.
- [12] Harsh Raju Chamarthi, Dillinger Peter C., Matt Kaufmann, and Panagiotis Manolios. Integrating testing and interactive theorem proving. 2011. doi:[10.4204/EPTCS.70.1](https://doi.org/10.4204/EPTCS.70.1).
- [13] Harsh Raju Chamarthi, Dillinger Peter C., and Panagiotis Manolios. Data Definitions in the ACL2 Sedan. *ACL2*, 2014.
- [14] S. Coretti, Y. Gilad, and R. Pass. The generals’ scuttlebutt: Byzantine-resilient gossip protocols. *Theoretical Computer Science*, 2022. doi:[10.1016/j.tcs.2022.07.006](https://doi.org/10.1016/j.tcs.2022.07.006).
- [15] Flaviene Scheidt de Cristo, Jorge Augusto Meira, Jean-Philippe Eisenbarth, and Radu State. A 9-dimensional analysis of gossipsub over the XRP ledger consensus protocol. In *NOMS 2024 IEEE Network Operations and Management Symposium*, 2024. doi:[10.1109/NOMS59830.2024.10575688](https://doi.org/10.1109/NOMS59830.2024.10575688).
- [16] A. Demers, D. Greene, C. Hauser, W. Irish, and J. Larson. Epidemic algorithms for replicated database maintenance. In *Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing (PODC)*, 1987.
- [17] Edsger W. Dijkstra. Self-stabilizing systems in spite of distributed control. *Commun. ACM*, 1974. doi:[10.1145/361179.361202](https://doi.org/10.1145/361179.361202).
- [18] Peter C. Dillinger, Panagiotis Manolios, Daron Vroon, and J. Strother Moore. ACL2s: “the ACL2 sedan”. In *Proceedings of the 7th Workshop on User Interfaces for Theorem Provers (UITP 2006)*, 2007. doi:[10.1016/j.entcs.2006.09.018](https://doi.org/10.1016/j.entcs.2006.09.018).
- [19] Ethereum Foundation. Ethereum consensus layer networking specification: Phase 0 p2p interface. Technical report, Ethereum Foundation, 2020. URL <https://github.com/ethereum/consensus-specs/blob/master/specs/phase0/p2p-interface.md>.
- [20] A. Ganesh, A.-M. Kermarrec, and L. Massoulié. Peer-to-peer membership management for gossip-based protocols. *IEEE Trans. Computers*, 2003. doi:[10.1109/TC.2003.1176982](https://doi.org/10.1109/TC.2003.1176982).
- [21] Hector Garcia-Molina, Boris Kogan, and Nancy A. Lynch. Reliable broadcast in networks with nonprogrammable servers. In *Proceedings of the 8th International Conference on Distributed Computing Systems (ICDCS)*, 1988. URL <https://groups.csail.mit.edu/tds/papers/Lynch/icdcs88.pdf>.
- [22] Ajei S. Gopal, Inder S. Gopal, and Shay Kutten. Fast broadcast in high-speed networks. *IEEE/ACM Transactions on Networking*, 1999. doi:[10.1109/90.769773](https://doi.org/10.1109/90.769773).
- [23] B. Haeupler, J. Jin, and J. Sun. Adversarially-robust gossip algorithms for approximate quantile and mean computations. *arXiv preprint arXiv:2402.01792*, 2024. doi:[10.48550/arXiv.2402.01792](https://doi.org/10.48550/arXiv.2402.01792).

- [24] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R Lorch, Bryan Parno, Michael L Roberts, Srinath Setty, and Brian Zill. Ironfleet: proving practical distributed systems correct. In *Proceedings of the 25th Symposium on Operating Systems Principles*, 2015. doi:[10.1145/2815400.2815428](https://doi.org/10.1145/2815400.2815428).
- [25] Lioba Heimbach, Yann Vonlanthen, Juan Villacis, Lucianna Kiffer, and Roger Wattenhofer. Deanonymizing ethereum validators: The p2p network has a privacy issue. In *USENIX Security Symposium*, 2025. URL <https://arxiv.org/abs/2409.04366>.
- [26] Daniel Jackson. Alloy: a language and tool for exploring software designs. *Commun. ACM*, 2019. doi:[10.1145/3338843](https://doi.org/10.1145/3338843).
- [27] Mitesh Jain and Panagiotis Manolios. Skipping refinement. In *Computer Aided Verification*, 2015. doi:[10.1007/978-3-319-21690-4\\_7](https://doi.org/10.1007/978-3-319-21690-4_7).
- [28] Mitesh Jain and Panagiotis Manolios. Proving skipping refinement with acl2s. In *International Workshop on the ACL2 Theorem Prover and Its Applications*, 2015. doi:[10.4204/EPTCS.192.9](https://doi.org/10.4204/EPTCS.192.9).
- [29] Mitesh Jain and Panagiotis Manolios. Local and compositional reasoning for optimized reactive systems. In *Computer Aided Verification*, 2019. doi:[10.1007/978-3-030-25540-4\\_32](https://doi.org/10.1007/978-3-030-25540-4_32).
- [30] R. Kane, P. Manolios, and S.K. Srinivasan. Monolithic verification of deep pipelines with collapsed flushing. In *Proceedings of the Design Automation and Test in Europe Conference*, 2006. doi:[10.1109/DATE.2006.244077](https://doi.org/10.1109/DATE.2006.244077).
- [31] Matt Kaufmann and J Strother Moore. ACL2 homepage, 2022. URL <https://www.cs.utexas.edu/users/moore/acl2/>.
- [32] Matt Kaufmann and J Strother Moore. Acl2 sources. <https://github.com/acl2/acl2>, 2023. Accessed 12 June 2023.
- [33] Matt Kaufmann, Panagiotis Manolios, and J Strother Moore. *Computer-Aided Reasoning: Case Studies*. Kluwer Academic Publishers, July 2000. doi:[10.1007/978-1-4757-3188-0](https://doi.org/10.1007/978-1-4757-3188-0).
- [34] Matt Kaufmann, Panagiotis Manolios, and J Strother Moore. *Computer-Aided Reasoning: An Approach*. Kluwer Academic Publishers, July 2000. doi:[10.1007/978-1-4615-4449-4](https://doi.org/10.1007/978-1-4615-4449-4).
- [35] Lucianna Kiffer, Asad Salman, Dave Levin, Alan Mislove, and Cristina Nita-Rotaru. Under the hood of the ethereum gossip protocol. In *Financial Cryptography and Data Security*. Springer, 2021.
- [36] Ankit Kumar. Gossipsub model and attacks. <https://github.com/ankitku/gsacl2ws>, 2023. Accessed 22 July 2023.

- [37] Ankit Kumar and Panagiotis Manolios. Mathematical programming modulo strings. In *Formal Methods in Computer Aided Design, FMCAD*, 2021. doi:[10.34727/2021/ISBN.978-3-85448-046-4\\_36](https://doi.org/10.34727/2021/ISBN.978-3-85448-046-4_36).
- [38] Ankit Kumar and Panagiotis Manolios. Proof of refinement of floodsub, 2025. URL <https://github.com/ankitku/FloodsubRef>. In submission.
- [39] Ankit Kumar and Panagiotis Manolios. Lean gossip: Gossipsub parameter optimization simulator, 2025. URL <https://github.com/ankitku/lean-gossip>. Simulation framework accompanying “Lean Gossip: Big Gains From Small Talk”.
- [40] Ankit Kumar and Panagiotis Manolios. A formalization of the correctness of the floodsub protocol. *Electronic Proceedings in Theoretical Computer Science*, 2025. doi:[10.4204/eptcs.423.9](https://doi.org/10.4204/eptcs.423.9).
- [41] Ankit Kumar, Max von Hippel, Panagiotis Manolios, and Cristina Nita-Rotaru. Verification of gossipsub in acl2s. In *International Workshop on the ACL2 Theorem Prover and Its Applications*, 2023. doi:[10.4204/EPTCS.393.10](https://doi.org/10.4204/EPTCS.393.10).
- [42] Ankit Kumar, Max von Hippel, Pete Manolios, and Cristina Nita-Rotaru. Formal model-driven analysis of resilience of gossipsub to attacks from misbehaving peers, 2023. URL <https://arxiv.org/abs/2212.05197>.
- [43] Ankit Kumar, Max von Hippel, Panagiotis Manolios, and Cristina Nita-Rotaru. Formal model-driven analysis of resilience of gossipsub to attacks from misbehaving peers. In *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19-23, 2024*, 2024. doi:[10.1109/SP54263.2024.00017](https://doi.org/10.1109/SP54263.2024.00017).
- [44] Leslie Lamport. The implementation of reliable distributed multiprocess systems. *Computer Networks*, 1978. doi:[10.1016/0376-5075\(78\)90045-4](https://doi.org/10.1016/0376-5075(78)90045-4).
- [45] Leslie Lamport. *Specifying systems: the TLA+ language and tools for hardware and software engineers*. Addison-Wesley, 2002. ISBN 0-3211-4306-X.
- [46] João Leitão, José Pereira, and Luís E. T. Rodrigues. Epidemic broadcast trees. In *Symposium on Reliable Distributed Systems (SRDS)*, 2007. doi:[10.1109/DSN.2007.50](https://doi.org/10.1109/DSN.2007.50).
- [47] Kai Li, Yuzhe Tang, Jiaqi Chen, Yibo Wang, and Xianghong Liu. TopoShot. In *Internet Measurement Conference*, 2021. doi:[10.1145/3487552.3487814](https://doi.org/10.1145/3487552.3487814).
- [48] libp2p. gossipsub v1.0: An extensible baseline pubsub protocol. <https://github.com/libp2p/specs/blob/master/pubsub/gossipsub/gossipsub-v1.0.md>, 2020. GitHub Repository - libp2p specifications.
- [49] libp2p. gossipsub v1.1: Extensions for attack resistance. <https://github.com/libp2p/specs/blob/master/pubsub/gossipsub/gossipsub-v1.1.md>, 2020. GitHub Repository - libp2p specifications.



- [50] libp2p. gossipsub v1.1: Extensions for attack resistance. <https://github.com/libp2p/specs/blob/master/pubsub/gossipsub/gossipsub-v1.1.md>, 2020. GitHub Repository - libp2p specifications.
- [51] Dylan Lott. Audit of Gossipsub v1.1 protocol design + implementation for protocol labs. <https://leastauthority.com/blog/audit-of-gossipsub-v1-1-protocol-design-implementation-for-protocol-labs/>, 2020. Accessed 3 March 2022.
- [52] Nancy A. Lynch and Frits W. Vaandrager. Forward and backward simulations for timing-based systems. In *Real-Time: Theory in Practice, REX Workshop, Mook, 1991, Proceedings*, 1991. doi:[10.1007/BFB0032002](https://doi.org/10.1007/BFB0032002).
- [53] Nancy A. Lynch and Frits W. Vaandrager. Forward and backward simulations: I. untimed systems. *Inf. Comput.*, 1995. doi:[10.1006/INCO.1995.1134](https://doi.org/10.1006/INCO.1995.1134).
- [54] Nancy A. Lynch and Frits W. Vaandrager. Forward and backward simulations, II: timing-based systems. *Inf. Comput.*, 1996. doi:[10.1006/INCO.1996.0060](https://doi.org/10.1006/INCO.1996.0060).
- [55] Panagiotis Manolios. Correctness of pipelined machines. In *Formal Methods in Computer-Aided Design, FMCAD*, 2000. doi:[10.1007/3-540-40922-X\\_11](https://doi.org/10.1007/3-540-40922-X_11).
- [56] Panagiotis Manolios. *Mechanical Verification of Reactive Systems*. PhD thesis, The University of Texas at Austin, Department of Computer Sciences, Austin TX, 2001.
- [57] Panagiotis Manolios and Sudarshan K. Srinivasan. Automatic verification of safety and liveness for xscale-like processor models using WEB refinements. In *Design, Automation and Test in Europe Conference and Exposition, DATE*, 2004. doi:[10.1109/DATE.2004.1268844](https://doi.org/10.1109/DATE.2004.1268844).
- [58] Panagiotis Manolios and Sudarshan K. Srinivasan. Refinement maps for efficient verification of processor models. In *Design, Automation and Test in Europe Conference and Exposition, DATE*, 2005. doi:[10.1109/DATE.2005.257](https://doi.org/10.1109/DATE.2005.257).
- [59] Panagiotis Manolios and Sudarshan K. Srinivasan. Automatic verification of safety and liveness for pipelined machines using WEB refinement. *ACM Trans. Design Autom. Electr. Syst.*, 2008. doi:[10.1145/1367045.1367054](https://doi.org/10.1145/1367045.1367054).
- [60] Panagiotis Manolios and Daron Vroon. Algorithms for ordinal arithmetic. In *Conference on Automated Deduction CADE*, 2003. doi:[10.1007/978-3-540-45085-6\\_19](https://doi.org/10.1007/978-3-540-45085-6_19).
- [61] Panagiotis Manolios and Daron Vroon. Integrating reasoning about ordinal arithmetic into ACL2. In *Formal Methods in Computer-Aided Design FMCAD*, LNCS. Springer-Verlag, 2004. doi:[10.1007/978-3-540-30494-4\\_7](https://doi.org/10.1007/978-3-540-30494-4_7).
- [62] Panagiotis Manolios and Daron Vroon. Ordinal Arithmetic: Algorithms and Mechanization. *Journal of Automated Reasoning*, 2005. doi:[10.1007/s10817-005-9023-9](https://doi.org/10.1007/s10817-005-9023-9).

- [63] Panagiotis Manolios and Daron Vroon. Termination analysis with calling context graphs. In *Computer Aided Verification CAV*, 2006. doi:[10.1007/11817963\\_36](https://doi.org/10.1007/11817963_36).
- [64] Pete Manolios. Reasoning about programs, 2023. URL <https://www.ccs.neu.edu/home/pete/courses/Logic-and-Computation/2023-Fall/lectures.html>. Lecture notes, Northeastern University, CS 2800: Logic and Computation, Accessed 21 Apr 2025.
- [65] Robin Milner. An algebraic definition of simulation between programs. In *Proceedings of the 2nd International Joint Conference on Artificial Intelligence*, 1971. URL <http://ijcai.org/Proceedings/71/Papers/044.pdf>.
- [66] Robin Milner. Operational and algebraic semantics of concurrent processes. In *Handbook of Theoretical Computer Science, Volume B: Formal Models and Semantics*. Elsevier and MIT Press, 1990. doi:[10.1016/B978-0-444-88074-1.50024-X](https://doi.org/10.1016/B978-0-444-88074-1.50024-X).
- [67] Sze-Yao Ni, Yu-Chee Tseng, Yuh-Shyan Chen, and Jang-Ping Sheu. The broadcast storm problem in a mobile ad hoc network. In *Proceedings of the 5th Annual ACM/IEEE International Conference on Mobile Computing and Networking (MobiCom '99)*, 1999. doi:[10.1145/313451.313525](https://doi.org/10.1145/313451.313525).
- [68] Oded Padon, Kenneth L. McMillan, Aurojit Panda, Mooly Sagiv, and Sharon Shoham. Ivy: safety verification by interactive generalization. In *ACM SIG-PLAN Conference on Programming Language Design and Implementation, PLDI*, 2016. doi:[10.1145/2908080.2908118](https://doi.org/10.1145/2908080.2908118).
- [69] David Michael Ritchie Park. Concurrency and automata on infinite sequences. In *Theoretical Computer Science, 5th GI-Conference, Karlsruhe, Germany, March 23-25, 1981, Proceedings*, 1981. doi:[10.1007/BFB0017309](https://doi.org/10.1007/BFB0017309).
- [70] Amir Pnueli. Linear and branching structures in the semantics and logics of reactive systems. In *Proceedings of the 12th Colloquium on Automata, Languages and Programming*, 1985.
- [71] ppoph and libp2p contributors. Gossipsub v2.0 spec: Lower or zero duplicates by lazy mesh propagation. <https://github.com/libp2p/specs/pull/653>, 2024. GitHub Pull Request #653, libp2p/specs.
- [72] ProbeLab. Ethereum network topology. <https://probelab.io/ethereum/topology/>, 2024. Accessed: 2025-01-15.
- [73] Protocol Labs. GO-LIBP2P-PUBSUB. <https://github.com/libp2p/go-libp2p-pubsub>, 2020.
- [74] Protocol Labs. What is Publish/Subscribe. <https://docs.libp2p.io/concepts/pubsub/overview/>, 2023. Accessed 12 May 2023.
- [75] Protocol Labs Research. Gossipsub-v1.1 evaluation report. Technical report, Protocol Labs, 2020. URL <https://research.protocol.ai/publications/gossipsub-v1.1-evaluation-report/>. Protocol Labs Research Publication.

- [76] Protocol Labs Research. Gossipsub: An attack-resilient messaging-layer protocol for public blockchains. <https://research.protocol.ai/blog/2020/gossipsub-an-attack-resilient-messaging-layer-protocol-for-public-blockchains/>, 2020. Protocol Labs Research Blog.
- [77] Fred B. Schneider. Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Comput. Surv.*, 1990. doi:[10.1145/98163.98167](https://doi.org/10.1145/98163.98167).
- [78] Kumar Shivam, Vishnu Paladugu, and Yanhong A. Liu. Specification and runtime checking of derecho, A protocol for fast replication for cloud services. In *Advanced tools, programming languages, and PPlatforms for Implementing and Evaluating algorithms for Distributed systems, ApPLIED*, 2023. doi:[10.1145/3584684.3597275](https://doi.org/10.1145/3584684.3597275).
- [79] Yu-Chee Tseng, Sze-Yao Ni, Yuh-Shyan Chen, and Jang-Ping Sheu. The broadcast storm problem in a mobile ad hoc network. *Wireless Networks*, 2002. doi:[10.1023/A:1013763825347](https://doi.org/10.1023/A:1013763825347).
- [80] Max von Hippel. Code walk of the gossipsub acl2s formalization. <https://github.com/maxvonhippel/gossipSub-FM/blob/main/model/demo.lisp>, 2023. Accessed 30 May 2023.
- [81] Dimitris Vyzovitis. Gossipsub v1.0: An extensible baseline pubsub protocol. <https://github.com/libp2p/specs/blob/master/pubsub/gossipsub/gossipsub-v1.0-old.md>, 2020. Accessed 23 Jan 2025.
- [82] Dimitris Vyzovitis. gossipsub: An extensible baseline pubsub protocol. <https://github.com/libp2p/specs/blob/master/pubsub/gossipsub/README.md>, 2022. Accessed 28 Nov 2022.
- [83] Dimitris Vyzovitis, Yusef Nasirifard, Yiannis Psaras, and David Dias. Gossipsub: Attack-resilient message propagation in the filecoin and eth2.0 networks. *arXiv preprint arXiv:2007.02754*, July 2020. URL <https://arxiv.org/abs/2007.02754>. arXiv preprint.
- [84] Andrew T. Walter and Panagiotis Manolios. ACL2s systems programming. In *Workshop on the ACL2 Theorem Prover and its Applications*, 2022. doi:[10.4204/EPTCS.359.12](https://doi.org/10.4204/EPTCS.359.12).
- [85] Andrew T. Walter, Benjamin Boskin, Seth Cooper, and Panagiotis Manolios. Gamification of loop-invariant discovery from code. In *Proceedings of the Seventh AAAI Conference on Human Computation and Crowdsourcing, HCOMP*, 2019. doi:[10.1609/HCOMP.V7I1.5277](https://doi.org/10.1609/HCOMP.V7I1.5277).
- [86] Andrew T. Walter, David A. Greve, and Panagiotis Manolios. Enumerative data types with constraints. In *Formal Methods in Computer-Aided Design, FMCAD*, 2022. doi:[10.34727/2022/ISBN.978-3-85448-053-2\\_25](https://doi.org/10.34727/2022/ISBN.978-3-85448-053-2_25).
- [87] Andrew T. Walter, Ankit Kumar, and Panagiotis Manolios. Proving calculational proofs correct. In *Proceedings of the 18th International Workshop on the ACL2 Theorem Prover and Its Applications*, 2023. doi:[10.4204/EPTCS.393.11](https://doi.org/10.4204/EPTCS.393.11).

- [88] Pamela Zave. Using lightweight modeling to understand chord. *Comput. Commun. Rev.*, 2012. doi:[10.1145/2185376.2185383](https://doi.org/10.1145/2185376.2185383).
- [89] Chonghe Zhao, Yipeng Zhou, Shengli Zhang, Taotao Wang, Quan Z. Sheng, and Song Guo. DEthna: Accurate Ethereum network topology discovery with marked transactions. In *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, pages 1711–1720. IEEE, 2024. doi:[10.1109/INFOCOM52122.2024.10621281](https://doi.org/10.1109/INFOCOM52122.2024.10621281).

# Index

ACL2s, [32](#)

Abstractly silent suffix, [31](#)

$ACTL^* \setminus X$ , [32](#)

Attack Gadgets, [124](#)

Broadcastsub, [37](#)

Commitment implies Delivery, [33](#)

Floodsub, [37](#)

Gossipsub v1.0, [79](#)

Gossipsub v1.1, [105](#)

Lean gossip, [130](#)

Meshsub, [79](#)

Message frontier, [89](#)

Pareto dominance, [137](#)

Refinement

Compositional Simulation Refinement, [31](#)

Compositional Skipping Refinement, [31](#)

Skipping, [31](#)

Stuttering Simulation, [31](#)

Simulation

Reduced Well-Founded Skipping Simulation, [30](#)

Well-Founded Simulation, [30](#)

Temporal connectedness, [45](#)



# Vita

Ankit Kumar received his Bachelor of Technology degree in Electrical Engineering from the Indian Institute of Technology (BHU) Varanasi in 2010, and Master of Technology from Indian Institute of Technology, Kanpur in 2017. He joined Northeastern University in 2018 to pursue his doctoral studies in Computer Science under the supervision of Professor Panagiotis Manolios.

His research focuses on formal verification of distributed systems, with particular emphasis on peer-to-peer protocols. His work combines refinement-based reasoning, automated theorem proving, and security analysis to establish correctness guarantees for complex distributed systems.